

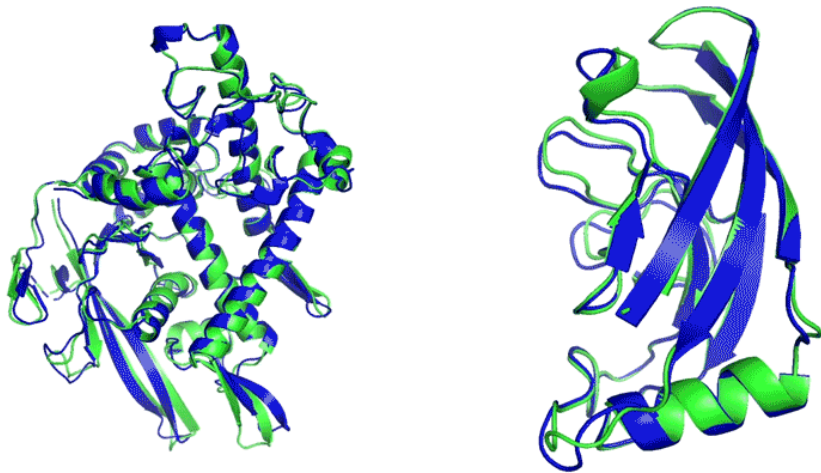
Artificial Intelligence in BIP

Dan Wu

danwu.bme@zju.edu.cn

Contents

- Introduction to AI in Medicine
- Machine learning in BIP
- Basic Neural Networks
- CNN, Attention, Generative Network...



b. Lipoma of breast

A 62-year-old woman presents for diagnostic mammography with a palpable mass in her right breast. Describe the finding.

Watson Inference

Infiltrating duct carcin...

Lipoma of breast

Fibroadenoma of bre...

Fat necrosis of breast

Eyes of Watson

Analyzing Text

Analyzing Images

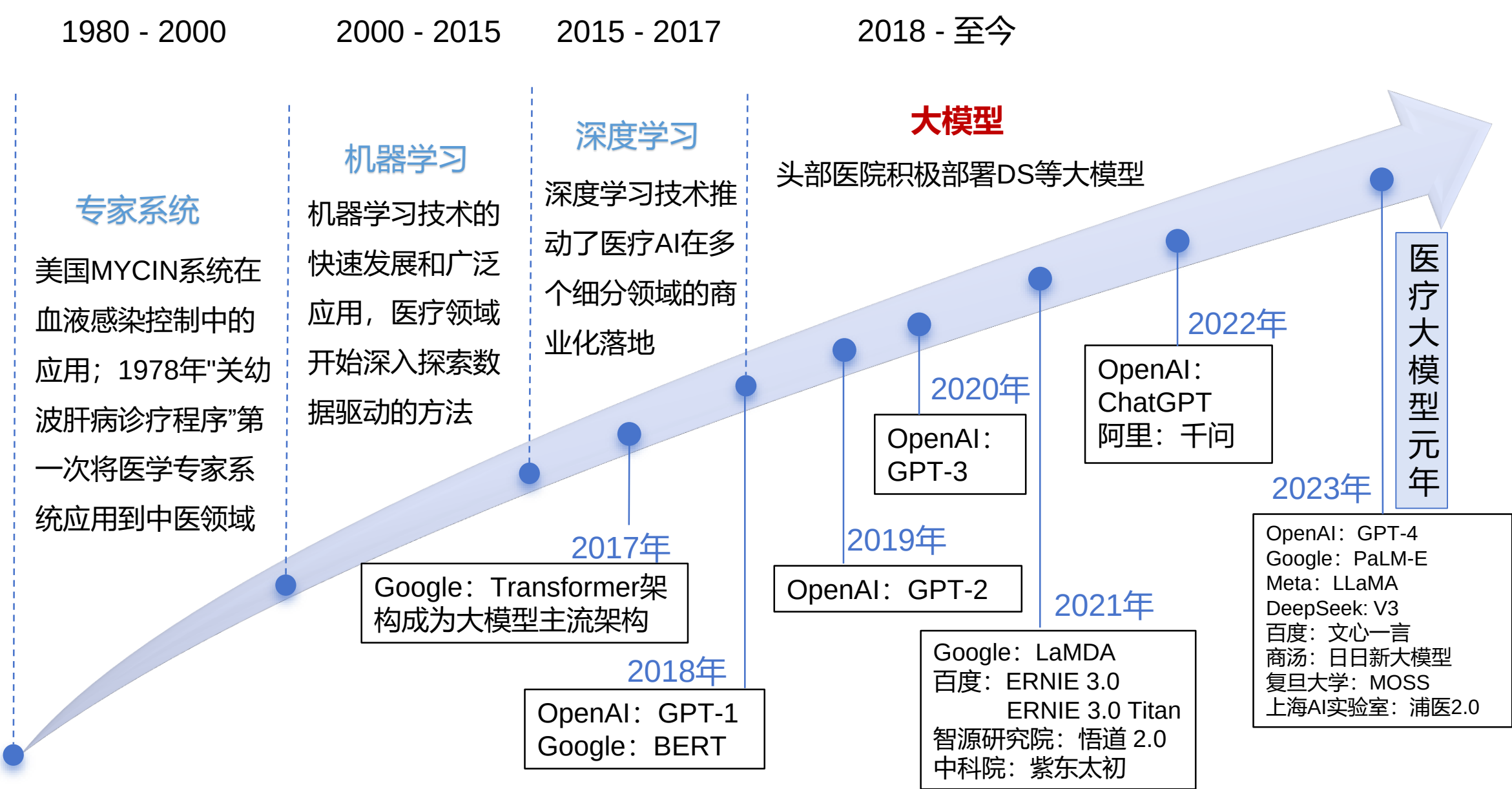
Reasoning

Selecting Answer



ACKNOWLEDGMENTS

医学人工智能

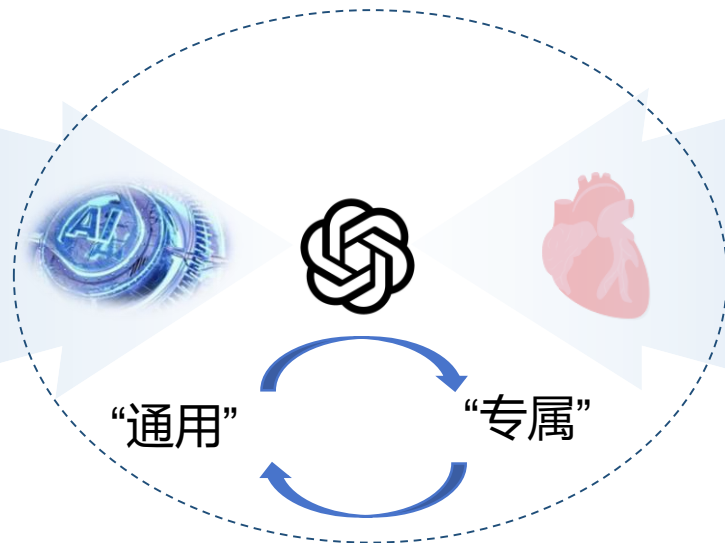


大模型技术引发医学人工智能“新一轮变革”

大模型：在海量数据上进行**大规模**预训练，然后通过**指令微调**以适应一系列下游任务的人工智能模型，被看做是一项人工智能技术迈向**通用智能**的里程碑式的进展

通用大模型

跨领域、跨任务的深度学习模型，具有**广泛适用性的模型**，可用于各种不同的任务和场景



领域大模型

针对特定领域或特定任务的大规模深度学习模型，**通用大模型+领域知识+领域数据**

LLaVA-Med (微软)

LLaVA-Med是一个**生物医学多模态对话模型**，是首次将多模态指令微调扩展到生物医学领域的尝试

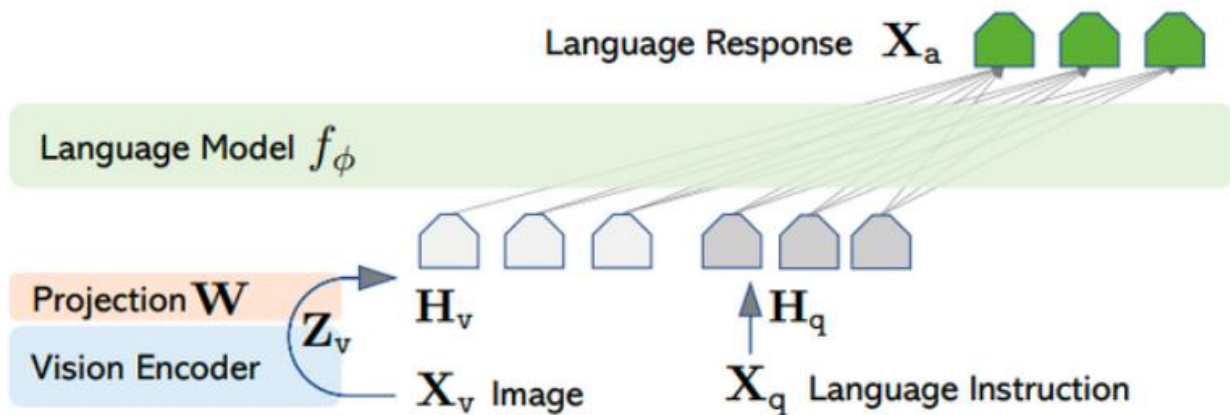


Figure 1: LLaVA network architecture.

数据集: **PMC-15M** 从PubMed Central的1500万生物医学**图像-文本对**

模型基座: 通用数据上训练的**LLaVA**

训练过程:

1. Stage1: 首先对LLaVA进行微调, 对齐文本和图像, **表征学习**
2. Stage2: 使用**instruction-following**数据学习开放式会话语义
3. 在下游数据集上进行微调

Biomedical Visual Instruction-Following Example

Caption:

Figure 4: Contrast-enhanced CT scan of the chest for patient #1. A large cavitating lesion was identified in the right upper lobe with pleural involvement and likely extension into the chest wall. In addition, right hilar lymphadenopathy was suspected. This was the primary malignant lesion in this patient.



Sentences with in-line figure mentions:

- Computerized tomography (CT) scans of the chest (Figure 4) were obtained.
- (Figure 4) and demonstrated a large cavitating lesion in the posterior aspect of the right upper lobe with probable pleural involvement likely representing the primary lesion.

Conversation:

User: Where is the lesion located?

Assistant: The lesion is located in the posterior aspect of the right upper lobe of the lung.

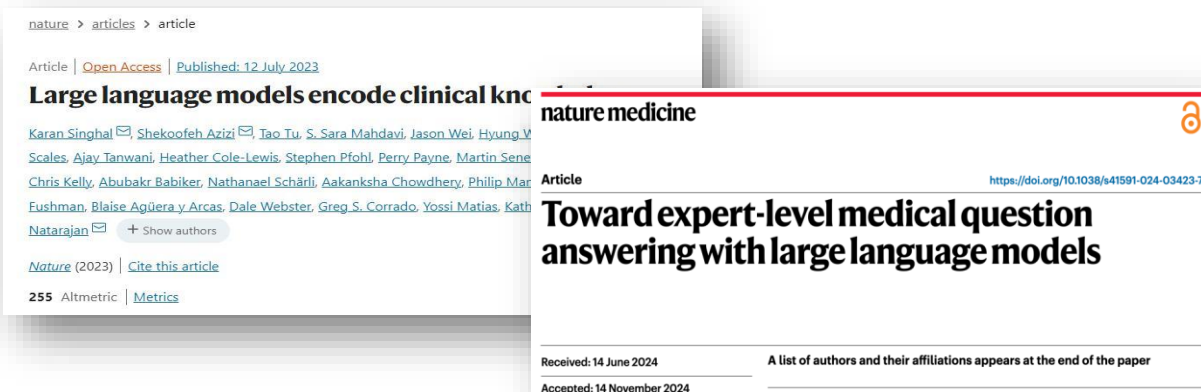
User: Is it extending into the chest wall?

Assistant: Yes, the CT scan suggests that the large cavitating lesion in the right upper lobe is likely extending into the chest wall.

User: Can you explain more about the cavitating lesion?

Assistant: Certainly. A cavitating lesion refers to an abnormal mass or growth within the lung tissue that has a cavity or hollow space inside it. This cavity can be formed due to the necrosis (death) of the tissue inside the lesion, leading to the breakdown and disintegration of the affected tissue. Cavitating lesions can be caused by various factors, such as infections (like tuberculosis or fungal infections), autoimmune diseases, or malignancies (cancer). In the context of the CT scan provided, the large cavitating lesion in the right upper lobe of the lung is suggestive of a malignant (cancerous) growth. The cavity within the lesion may be a result of the rapid and uncontrolled growth of cancer cells, which can lead to the destruction of surrounding healthy tissue. It's important to note that further diagnostic tests, such as a biopsy, would be required to confirm the malignancy and determine the specific type of cancer involved.

Med-PaLMM (谷歌)



- 构建了**MultiMedBench**，跨越多种模态(医学成像、临床文本和基因组学)，具有12个开源数据集和14种不同任务的超过100万个样本，用于训练和评估通才医疗大模型
- 可用相同的模型权值进行**医学图像分类、医学问答、视觉问答、放射学报告生成和总结、基因组变异调用**等。多个任务上达到与最先进的(SOTA)专业模型竞争或超过最先进的性能，而**无需任何特定于任务的定制**
- 246张回顾性胸部x光片盲选，其中40.50%病例，相比于放射科医生报告，临床医生更偏好选择模型生成的检查报告

Med-PaLMM 首个全科通才医疗大模型

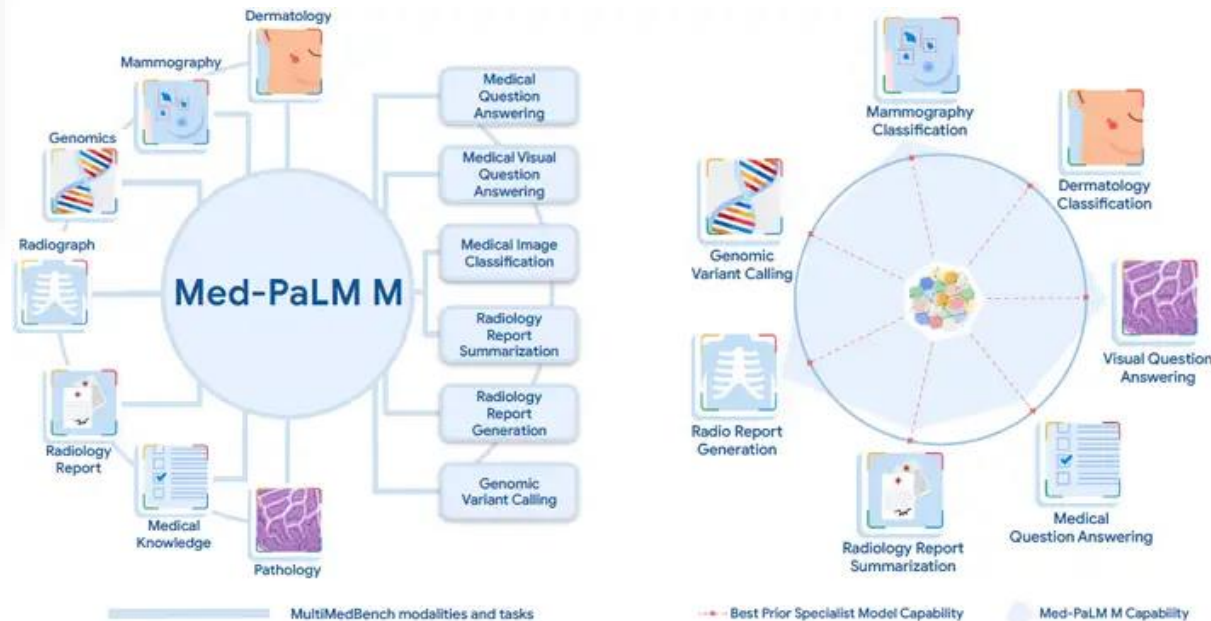
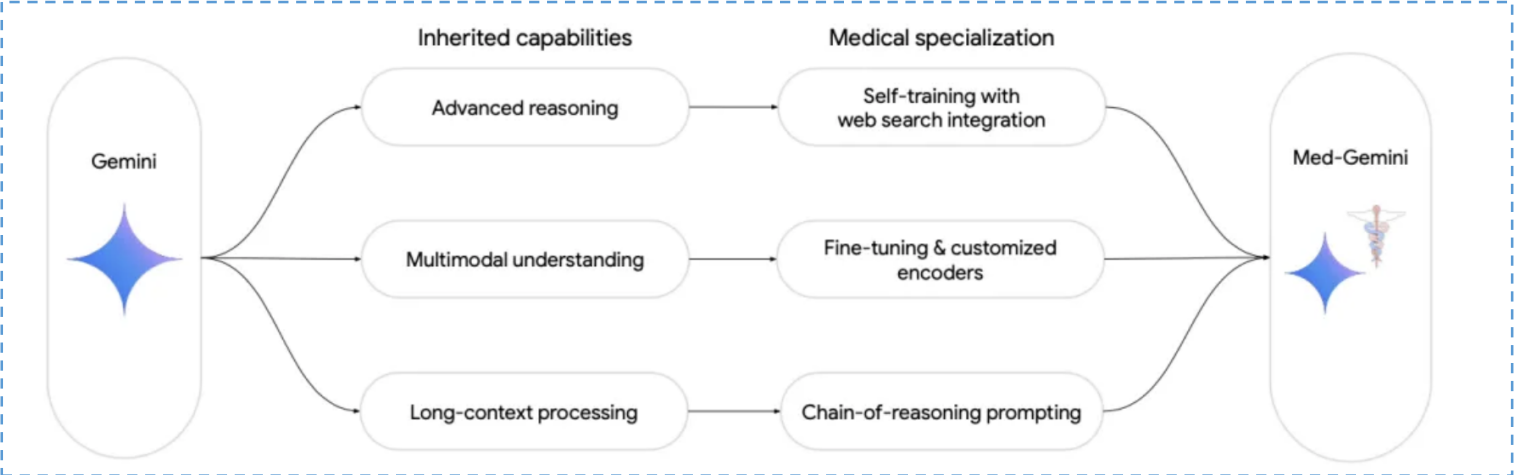


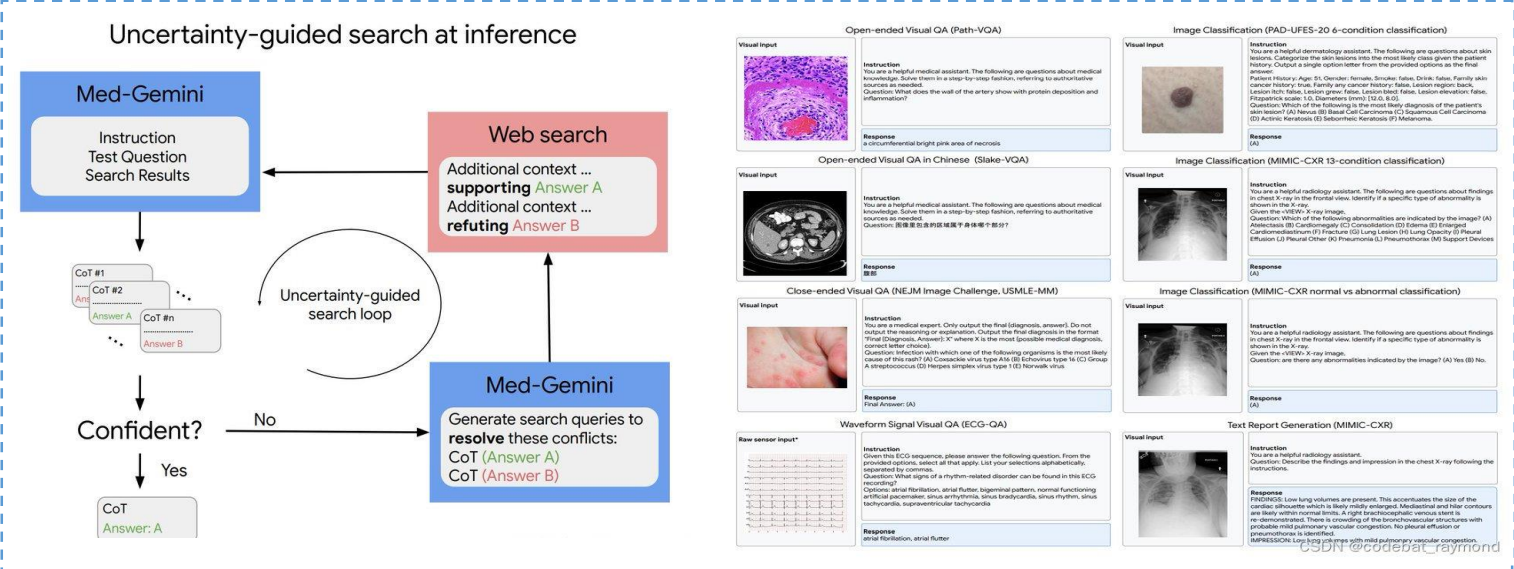
Figure 1 | Med-PaLM M overview. A generalist biomedical AI system should be able to handle a diverse range of biomedical data modalities and tasks. To enable progress towards this overarching goal, we curate MultiMedBench, a benchmark spanning 14 diverse biomedical tasks including question answering, visual question answering, image classification, radiology report generation and summarization, and genomic variant calling. Med-PaLM Multimodal (Med-PaLM M), our proof of concept for such a generalist biomedical AI system (denoted by the shaded blue area) is competitive with or exceeds prior SOTA results from specialists models (denoted by dotted red lines) on all tasks in MultiMedBench. Notably, Med-PaLM M achieves this using a single set of model weights, without any task-specific customization.

Med-Gemini (谷歌)

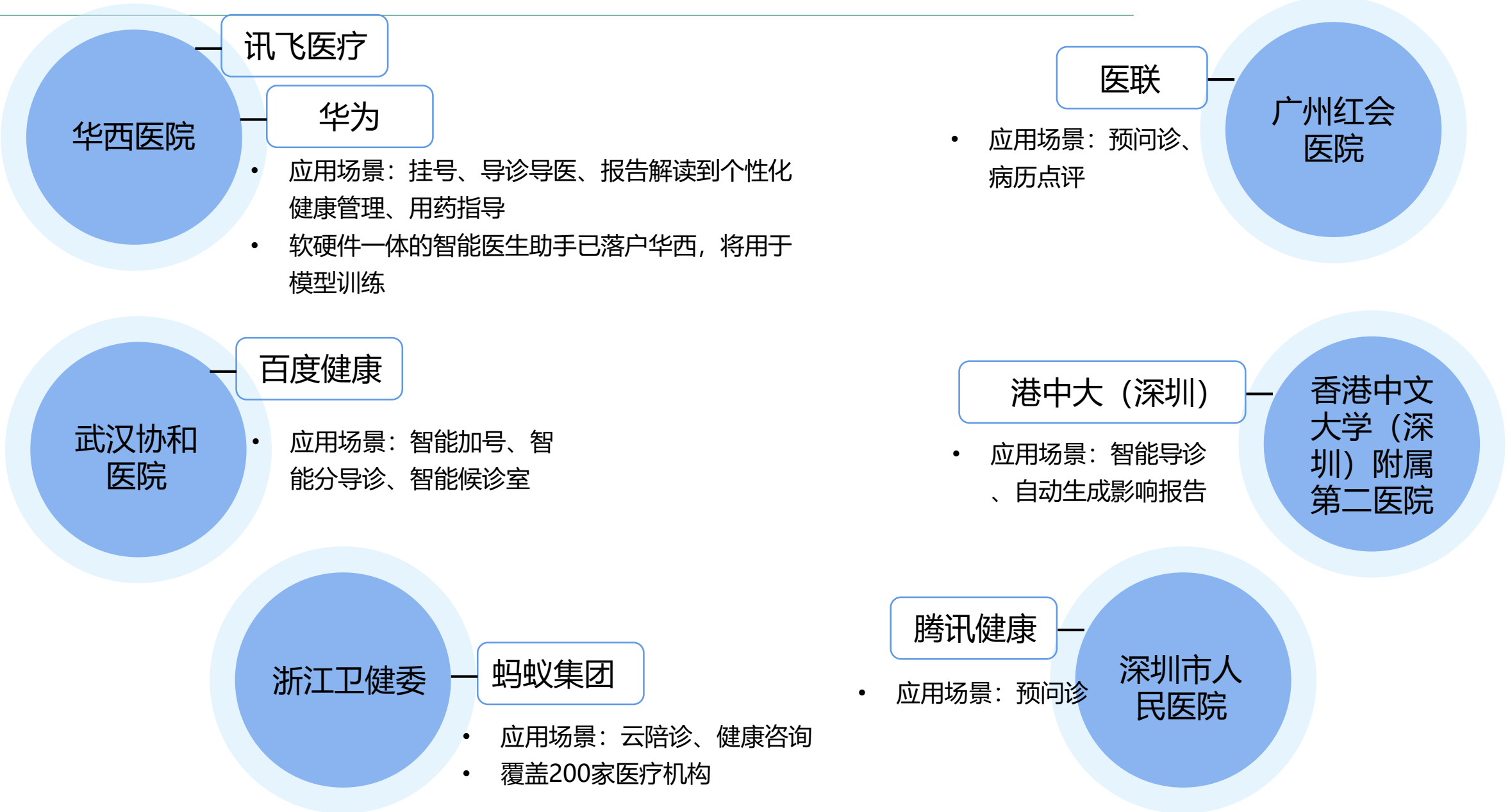
在基座模型基础上，引入高质量、多模态的真实世界医疗数据，通过**人在回路的思维链增强训练**，使大模型具备医疗领域的必要能力



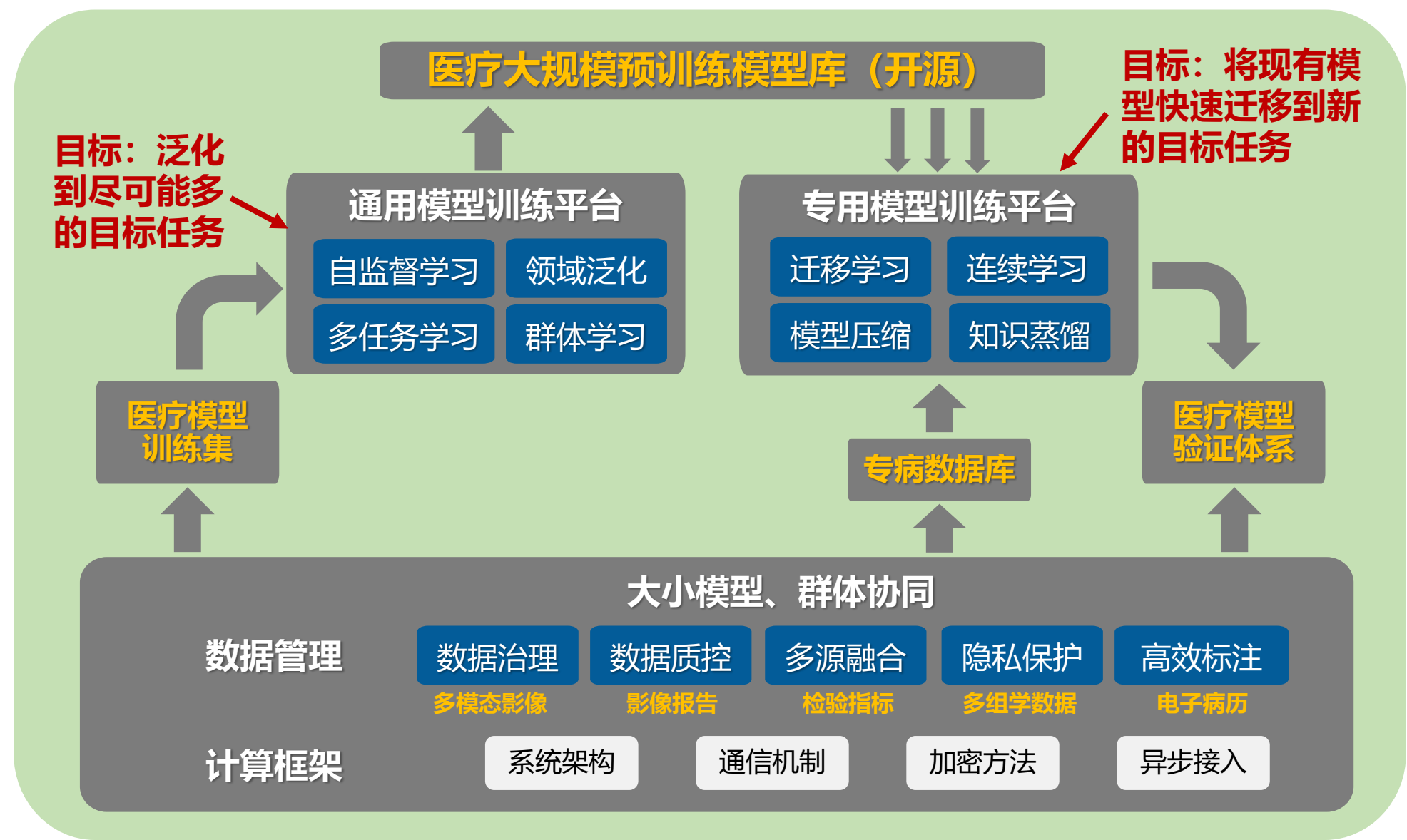
- 基于Google的Gemini大模型构建的多模态医学模型**Med-Gemini**，融合了高级推理、多模态理解和长文本处理能力
- 在**医学文本摘要、转诊信生成和医学简化任务**，Med-Gemini 的表现优于人类专家
- Med-Gemini 能够进行多模态医学对话，为临床医生和患者提供更自然、高效的交互方式



国内医疗大模型的应用实践



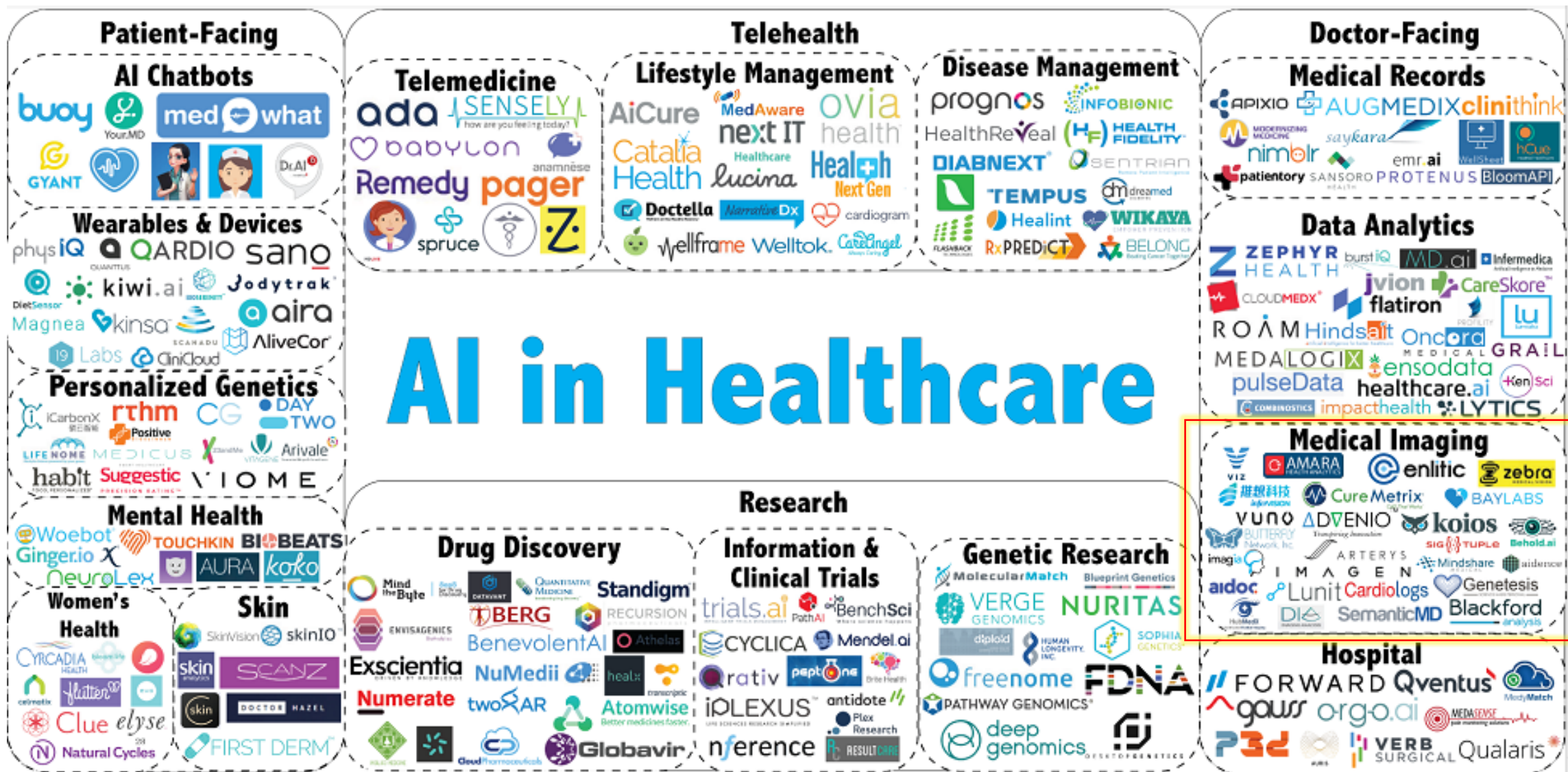
医疗大模型的发展目标



发展目标：构建隐私保护、用户友好框架下的大规模分布式医疗通用模型，形成**基于医疗多模态大模型的开放共享体系和模型效能综合评测体系**，实现高泛化表征学习以及其向下游任务的低成本迁移

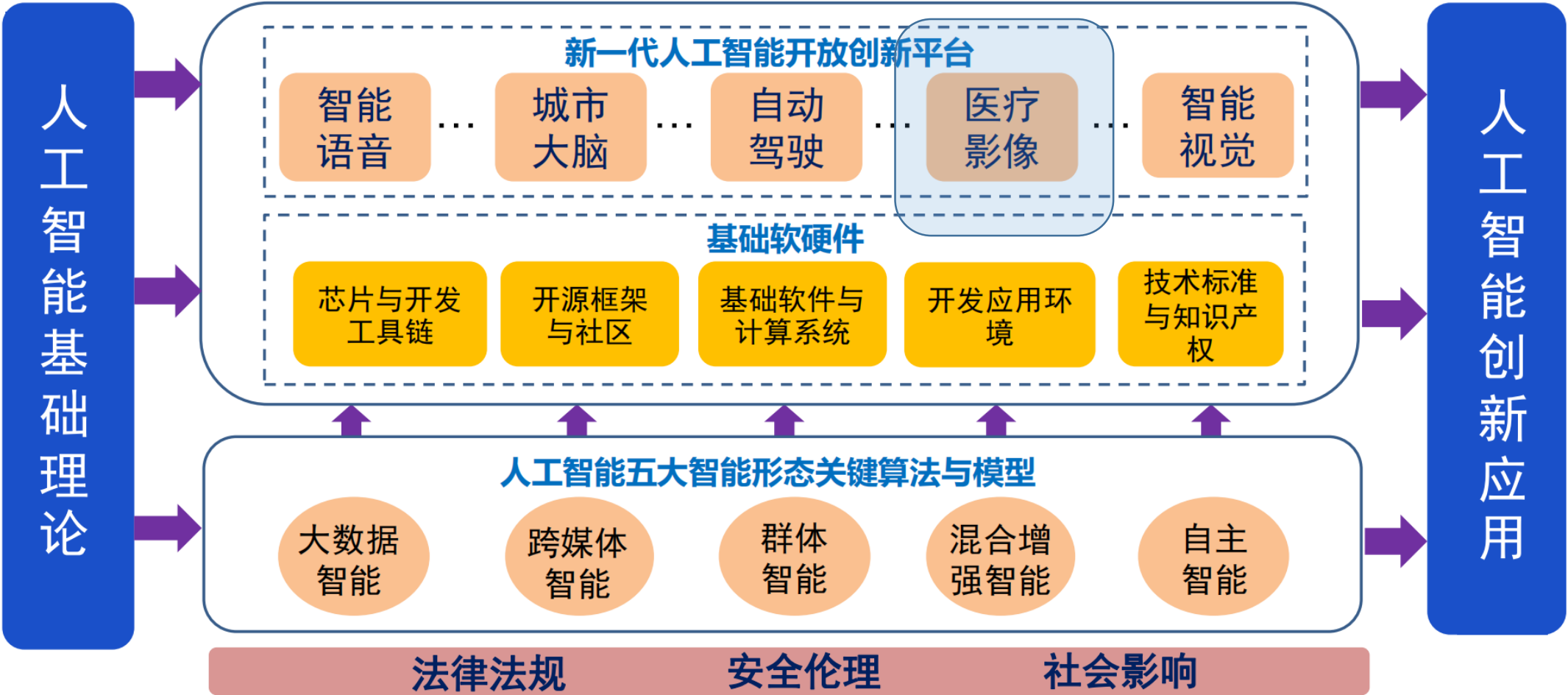
技术支撑+行业推动

艾森哲：到2026年人工智能每年能为全球医疗节约成本至少**1500亿美元**



《科技创新2030-新一代人工智能重大项目》

定位聚焦基础理论和关键技术，抢占人工智能发展战略制高点



Lecture 14

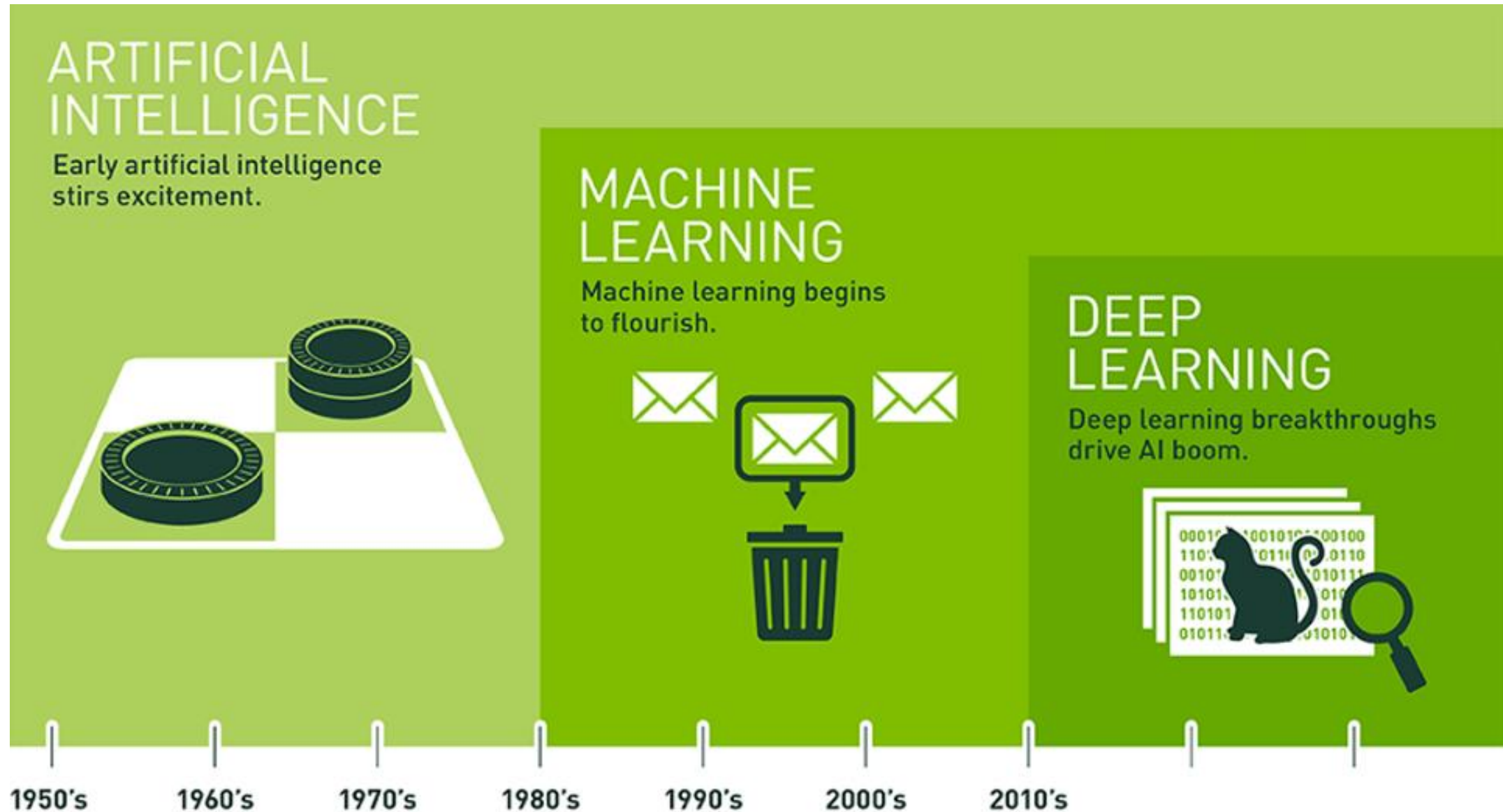
- Introduction to AI in Medicine
- Machine learning in BIP
 - Learnable features (data driven)
 - High dimensional features (deep learning)
- Neural Networks
 - Perceptron
 - Fully connected network

Lecture 15

- CNN
 - Basic structure of CNN
 - Introduction to classical networks
 - AlexNet, VGG, ResNet, UNet
- Attention
 - Soft Attention
 - Self Attention: Transformer
- Generative Network
 - GAN
 - Diffusion Model

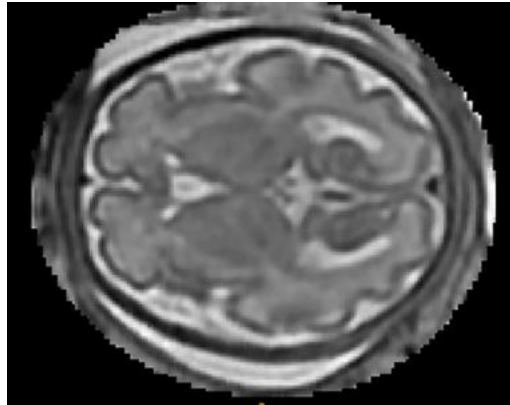
互动模式

	讲解人
1.线性可分，激活函数与多层感知机	林磊
2.全连接、MLP的异同	郑晓旭
3. CNN的卷积层细节，与前面图像卷积异同	史晓
4. CNN池化层细节	徐畅
5.Alexnet	徐梁策
6.VGG	吴依依
7.Resnet	周冰语
8.UNet	颜熙
9.Attention	王礼铭
10.Transformer	刘智瑞
11.GAN	周昭宇
12.Diffusion Model	丁思涵

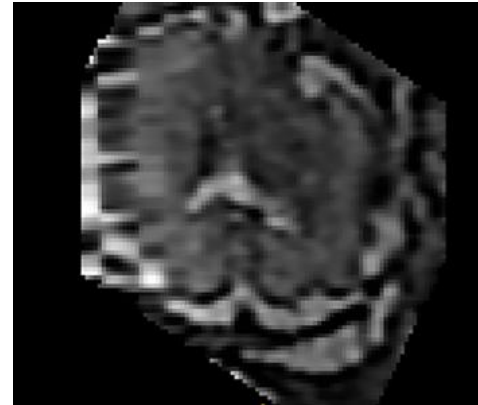


Machine Learning in BIP

- What is a high quality medical image?



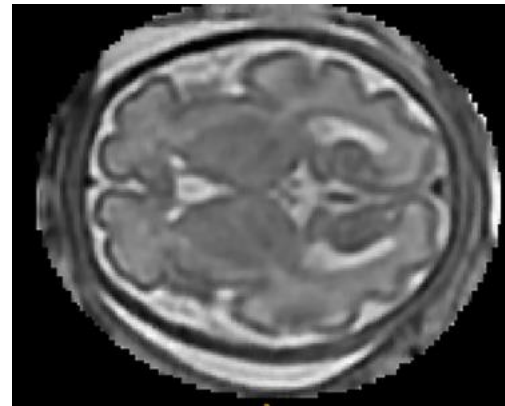
diagnoseable



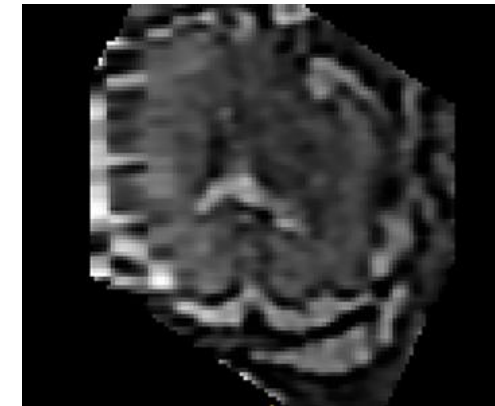
Not helpful

Machine Learning in BIP

- What is a high quality medical image?
- Human grade (**Golden standard**)
- Feature engineering
 - Clear edge
 - High contrast
 - High SNR
 - Motion artifact
 -



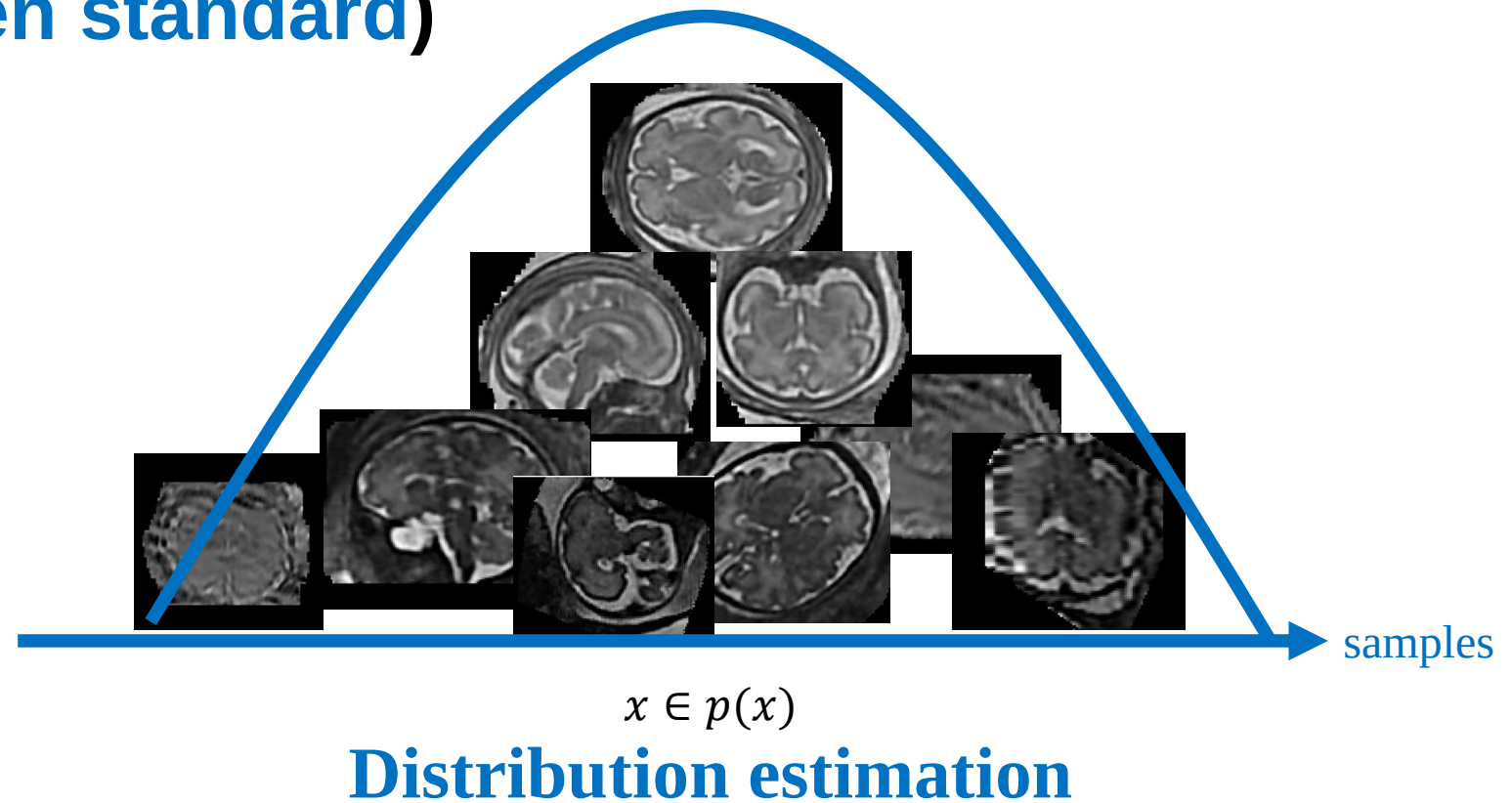
High quality



Low quality

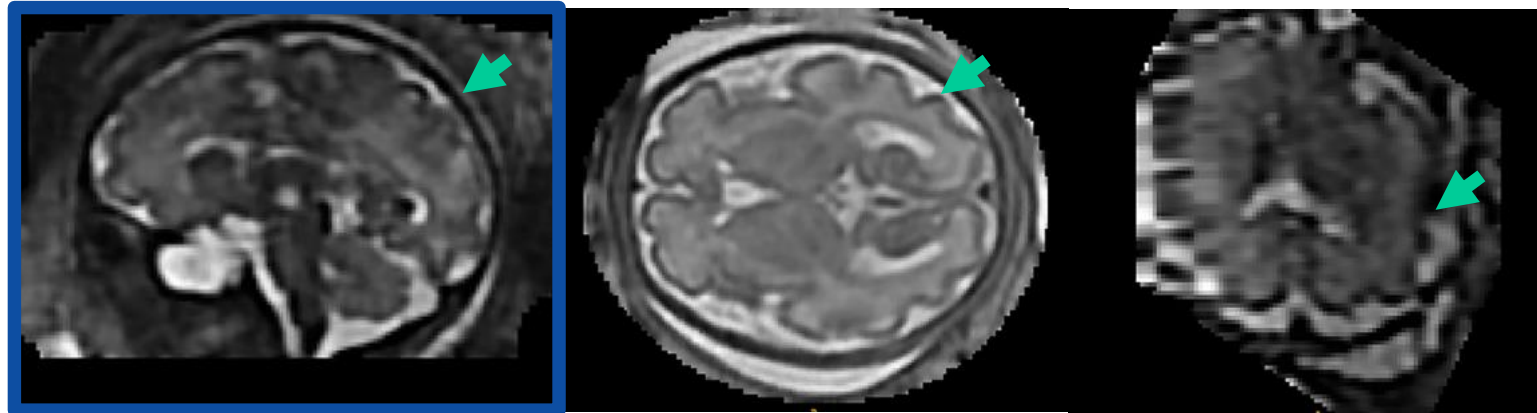
Machine Learning in BIP

- What is a high quality medical image?
- Human grade (**Golden standard**)
- Feature engineering
 - Clear edge
 - High contrast
 - High SNR
 - Motion artifact
 -



Machine Learning in BIP

- What is a high quality medical image?
- Feature engineering
 - Clear edge



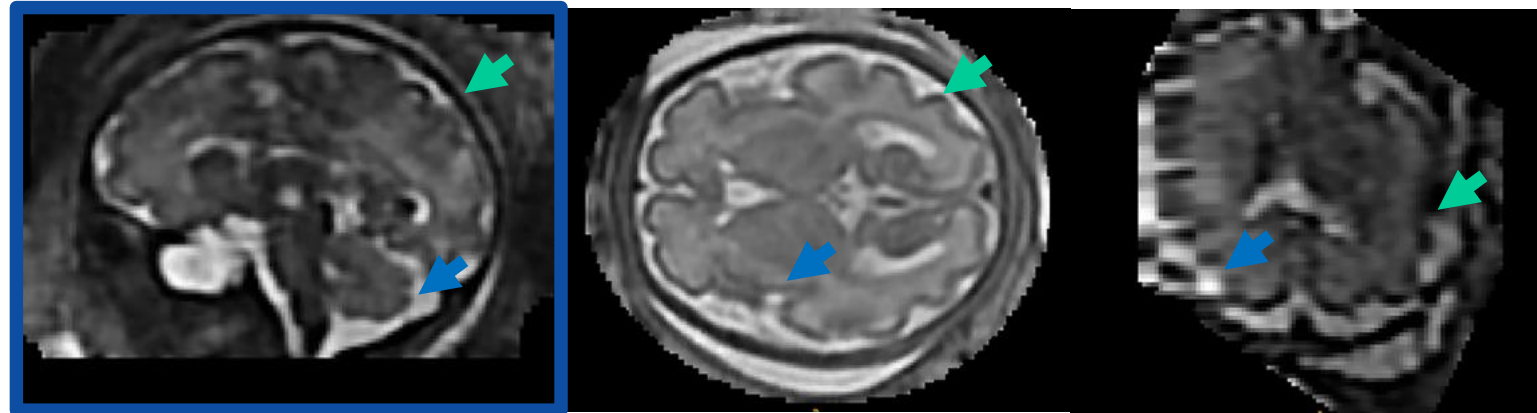
New data

High quality

Low quality

Machine Learning in BIP

- What is a high quality medical image?
- Feature engineering
 - Clear edge
 - High contrast



New data

High quality

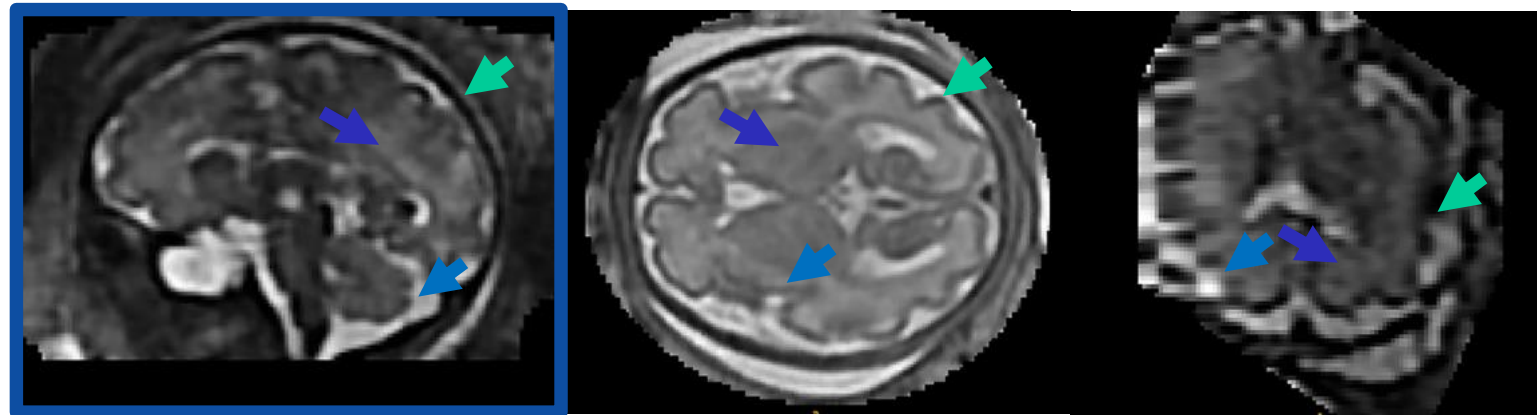
Low quality

Machine Learning in BIP

- What is a high quality medical image?

- Feature engineering

- Clear edge
- High contrast
- High SNR



New data

High quality

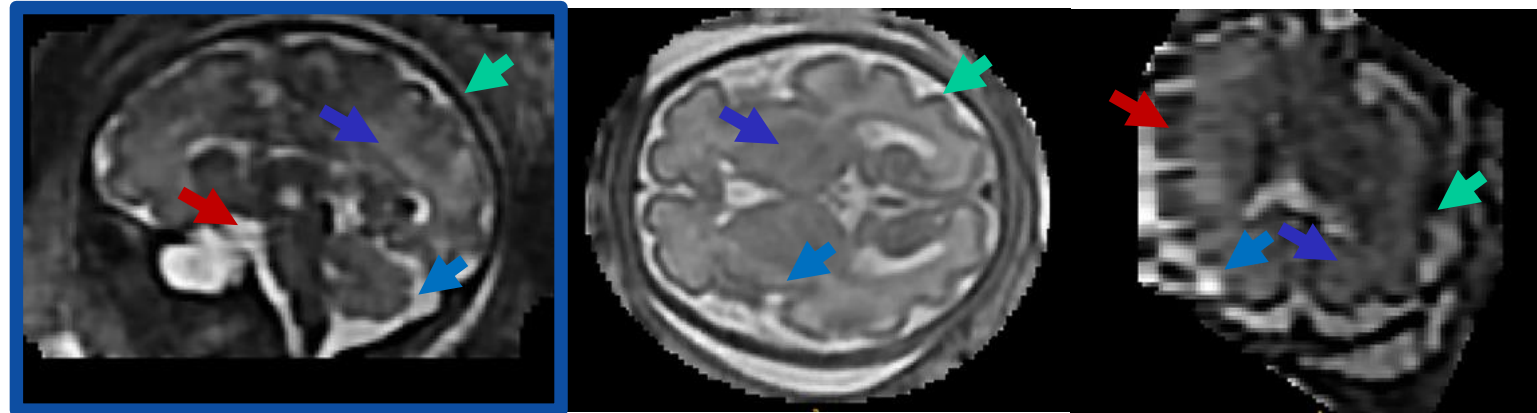
Low quality

Machine Learning in BIP

- What is a high quality medical image?

- Feature engineering

- Clear edge
- High contrast
- High SNR
- Motion artifacts



New data

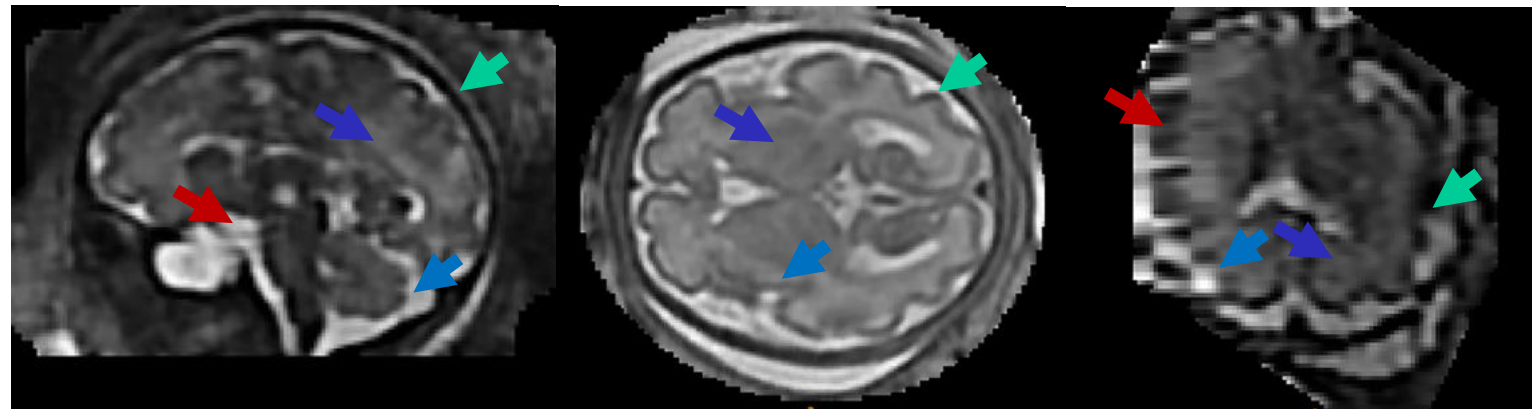
High quality

Low quality

- High or low quality?

Machine Learning in BIP

- Feature engineering
 - Explainable
 - Easy to fine
- Problems
 - Limited by human
 - Small sample size
 - Can not get high dimensional features
 - Subject bias



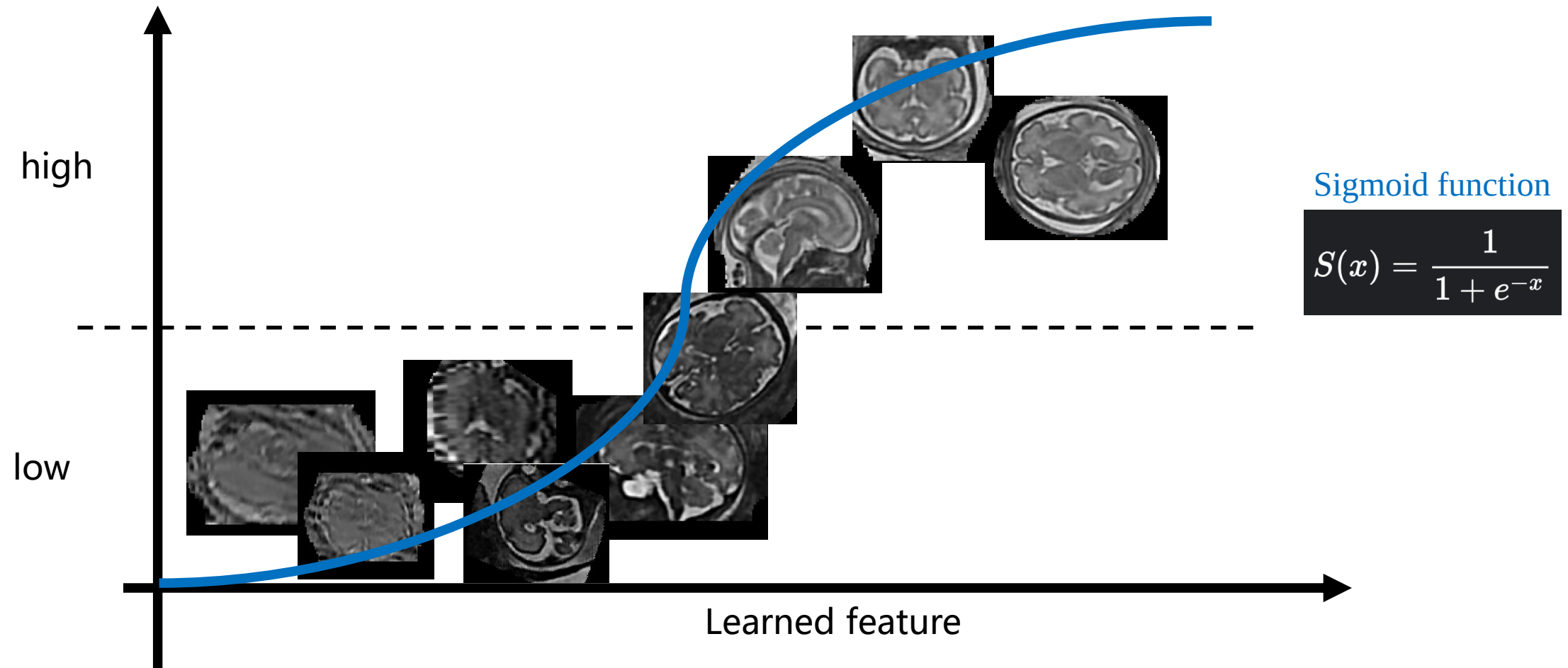
New data

High quality

Low quality

Machine Learning in BIP

- From feature engineering to machine learning



Machine Learning in BIP

Image quality High or Low

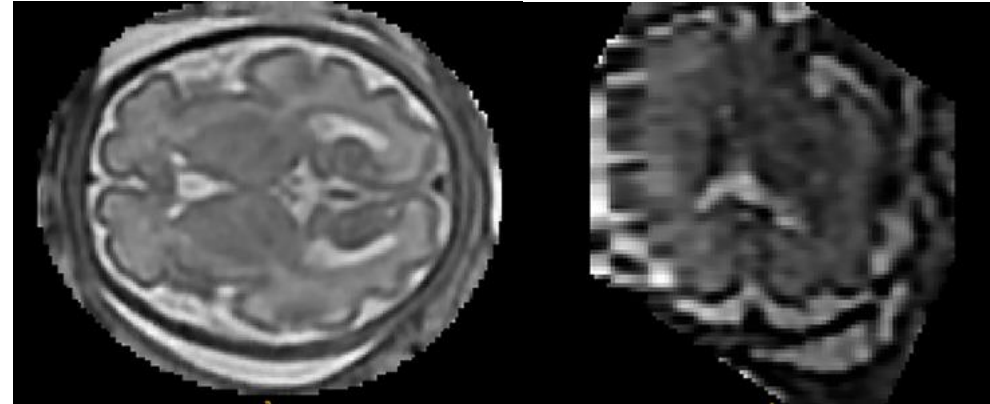
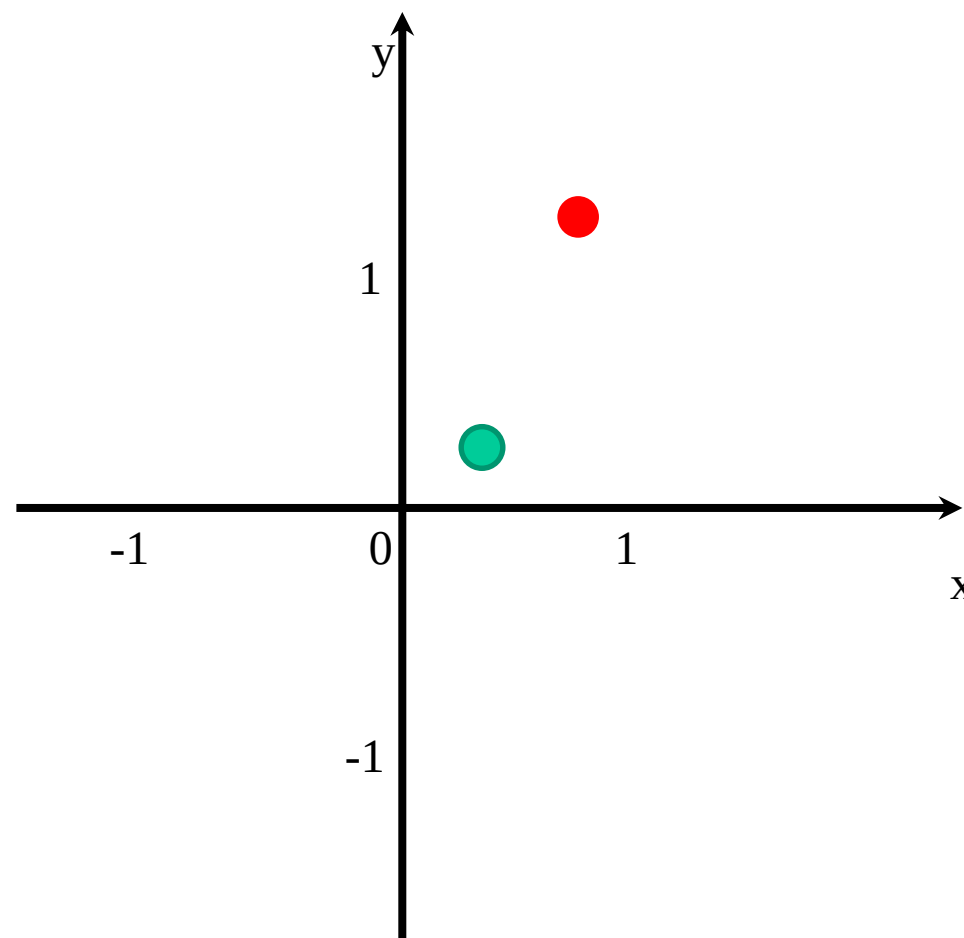


Image segmentation

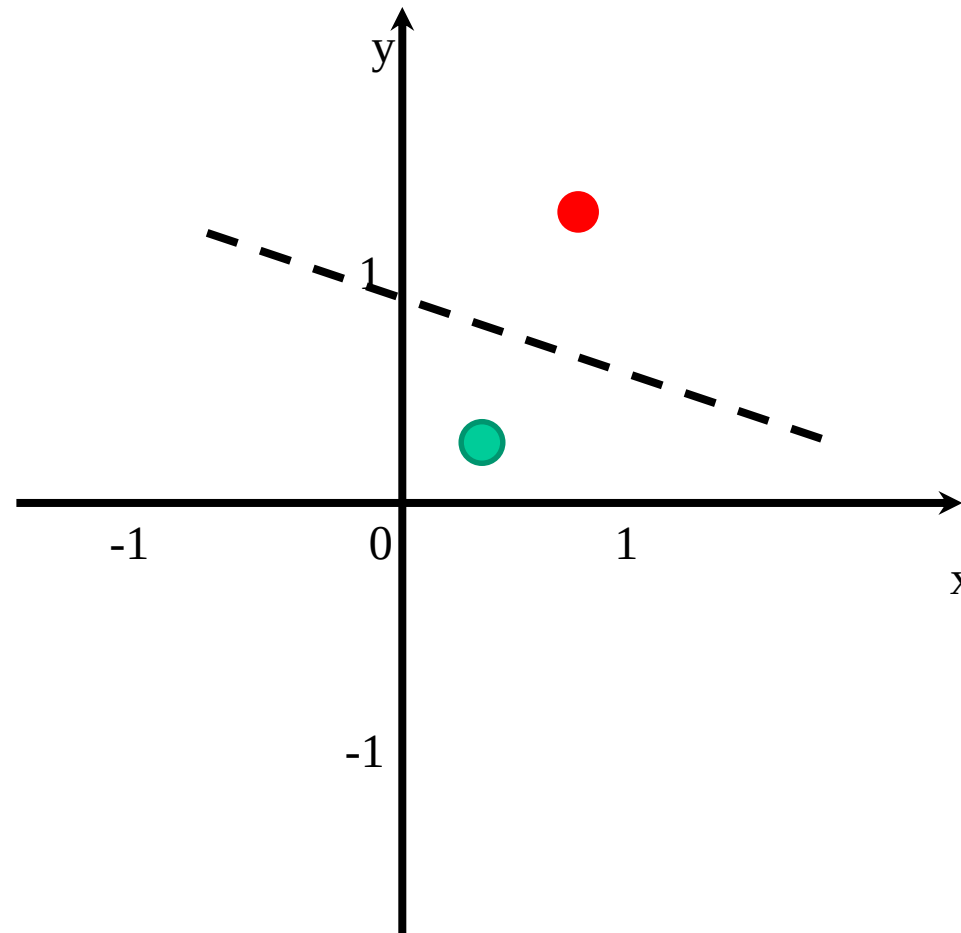
Label of pixels

[illegible]

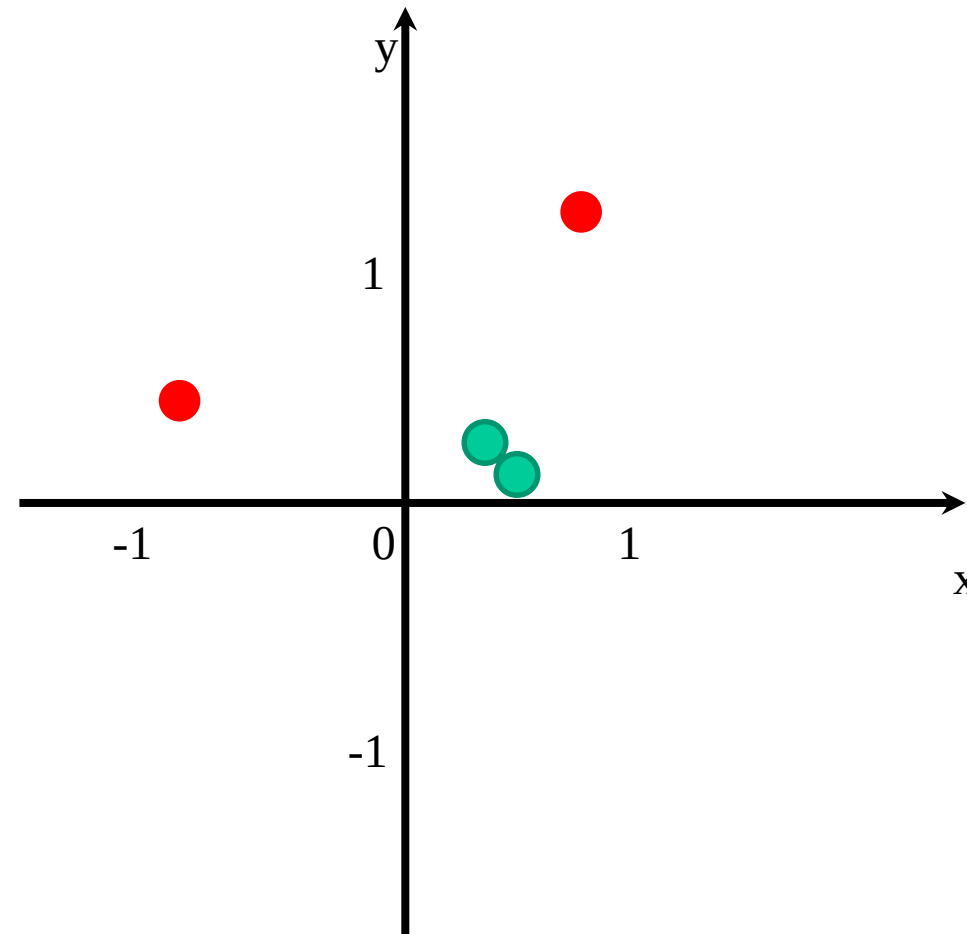
Machine Learning in BIP



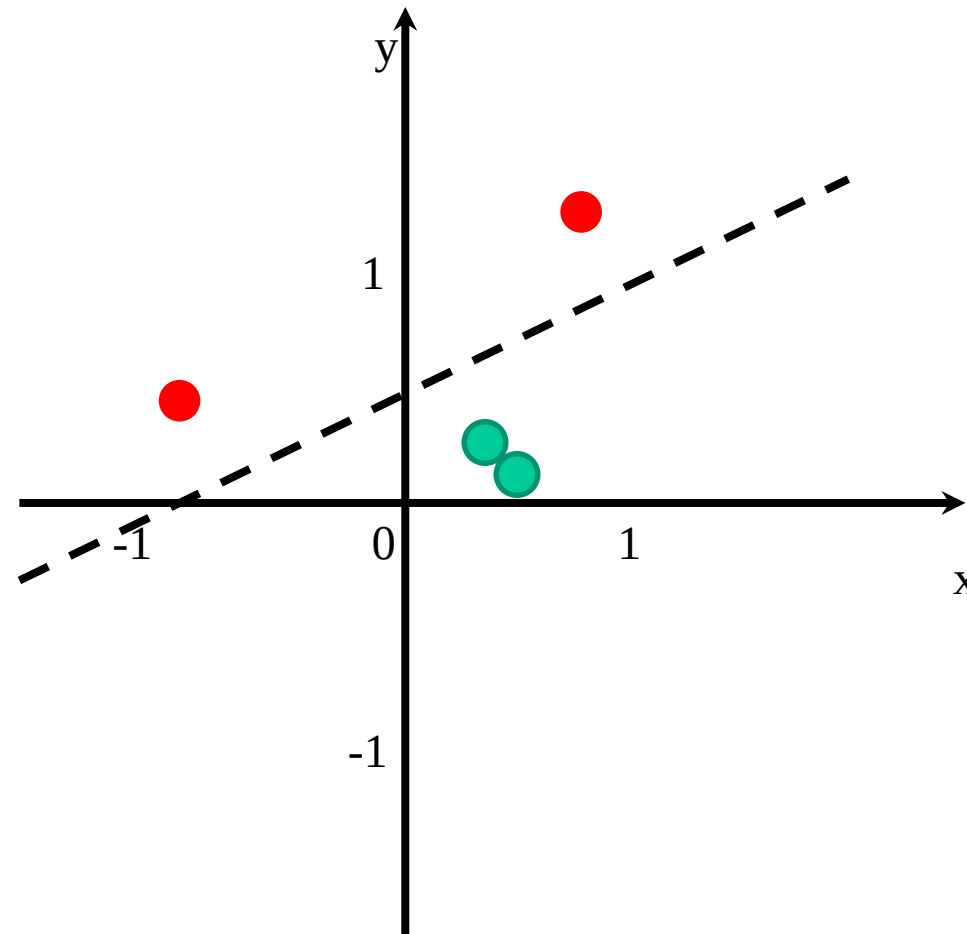
Machine Learning in BIP



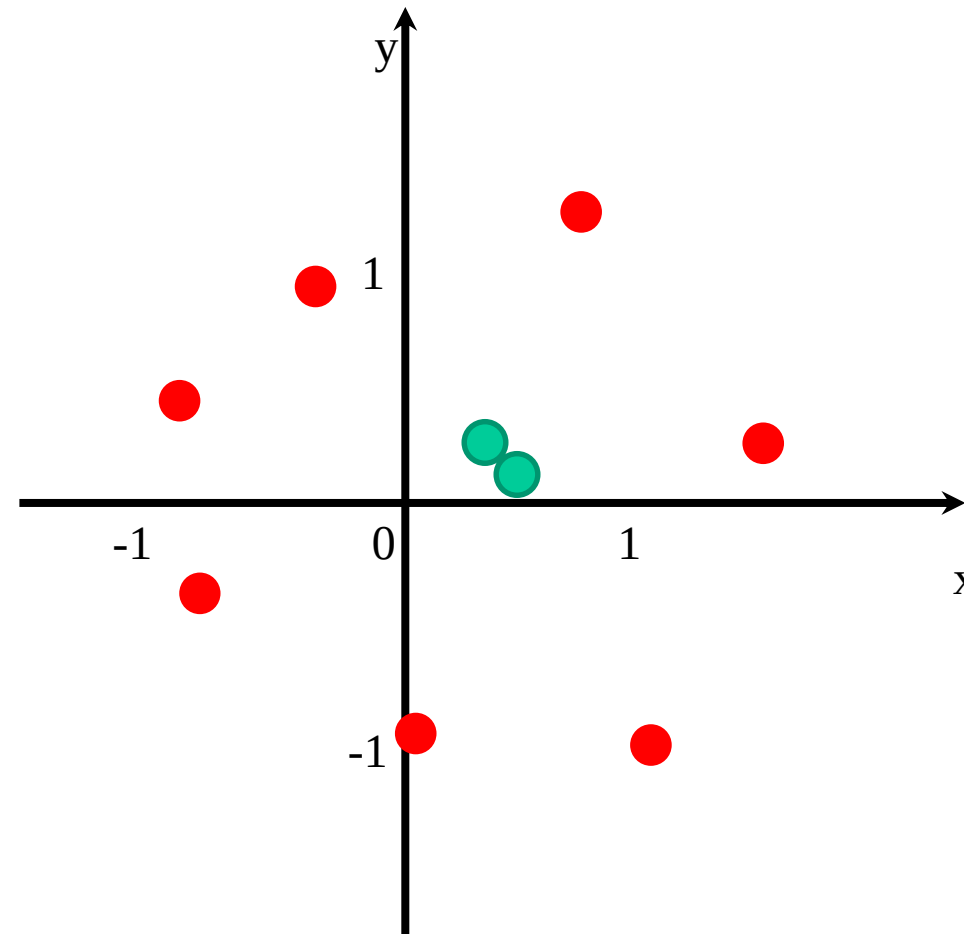
Machine Learning in BIP



Machine Learning in BIP

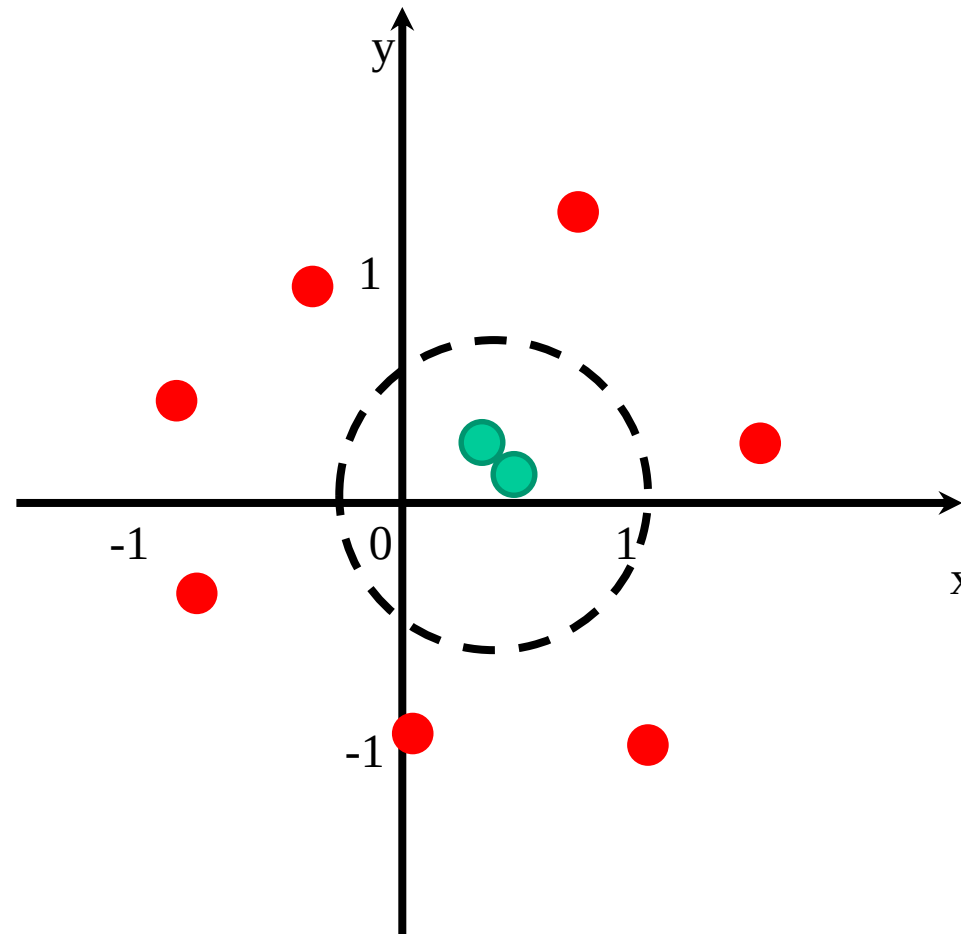


Machine Learning in BIP



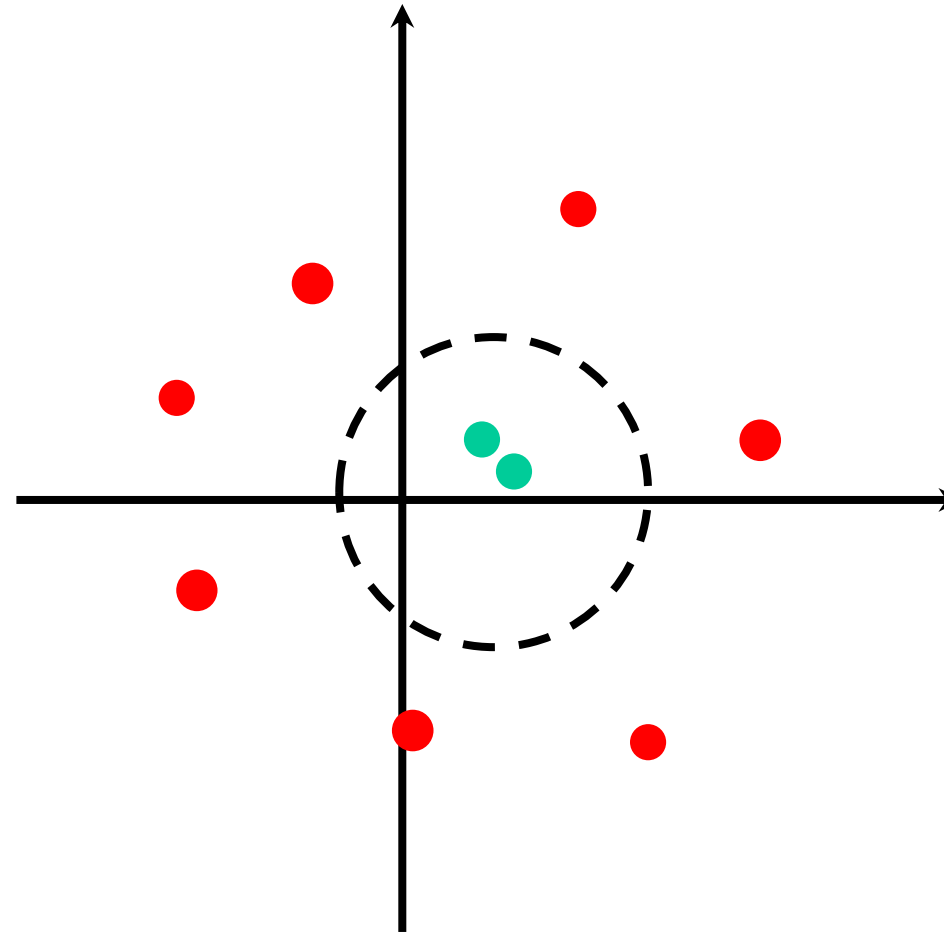
?

Machine Learning in BIP



!

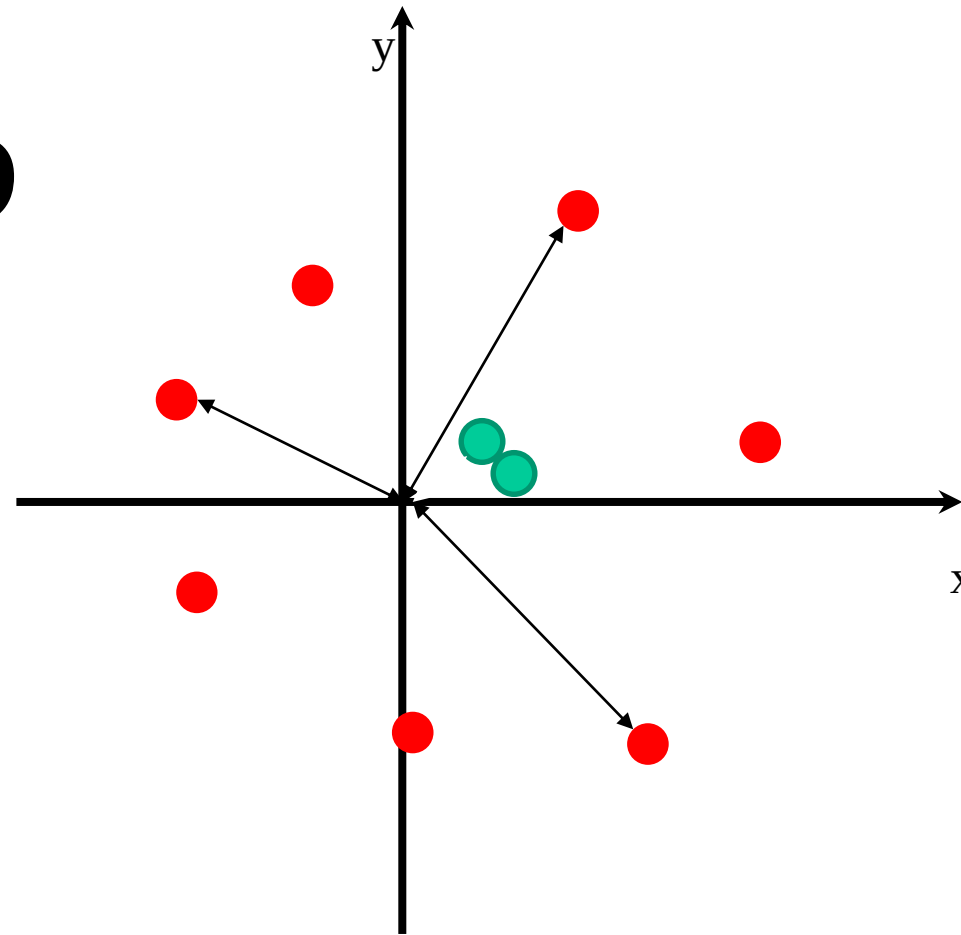
Machine Learning in BIP



Model=circle
None linear
Simpler mode?

Machine Learning in BIP

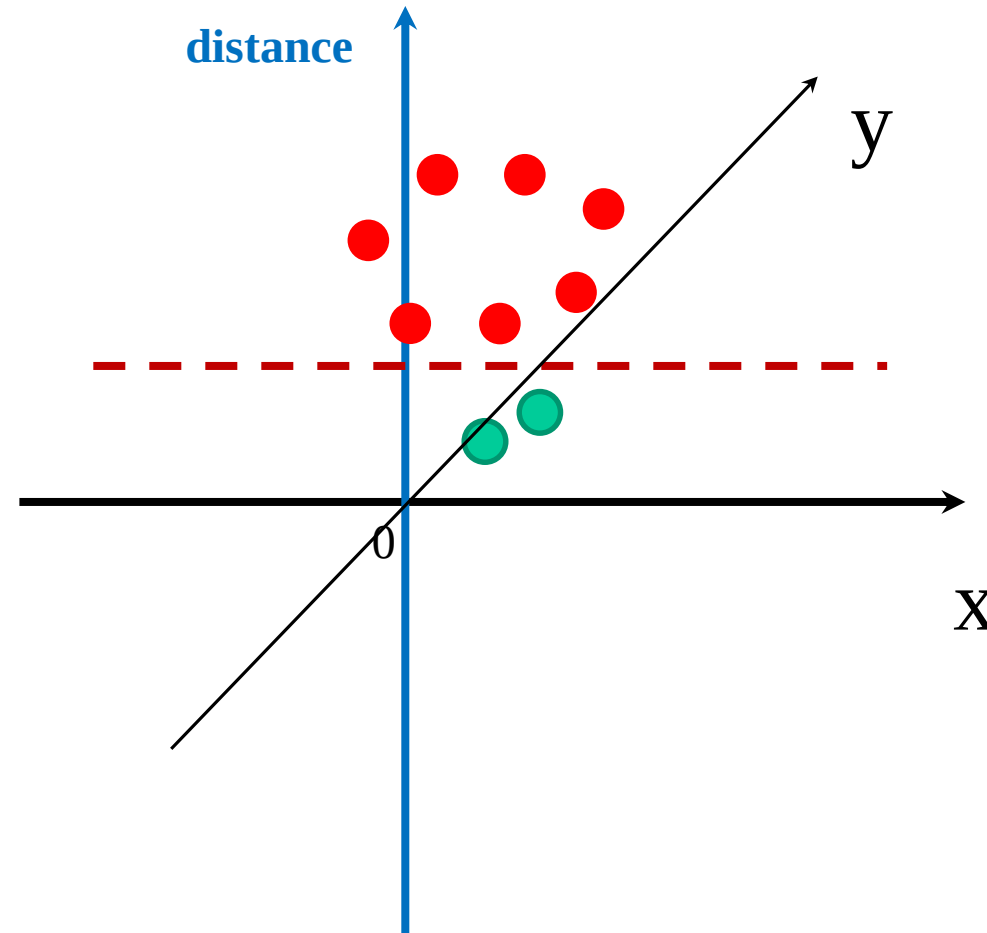
High dimension?



Machine Learning in BIP

New dimension
The Distance

!

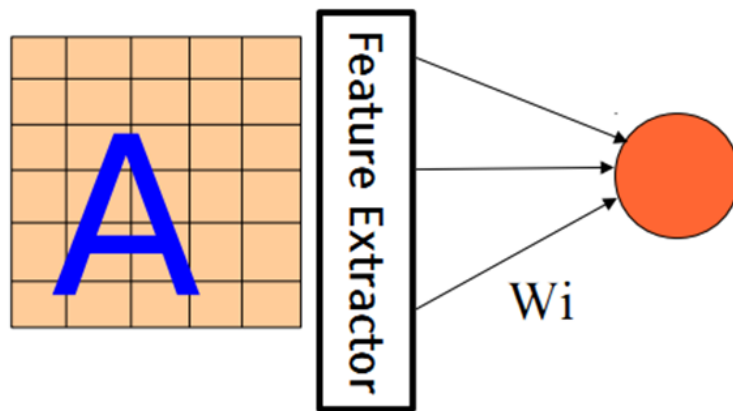


Contents

- Introduction to AI in Medicine
- Machine learning in BIP
 - Learnable features (data driven)
 - High dimensional features (deep learning)
- Neural Networks
 - Perceptron
 - Fully connected network

Neural Networks and Deep Learning – Background

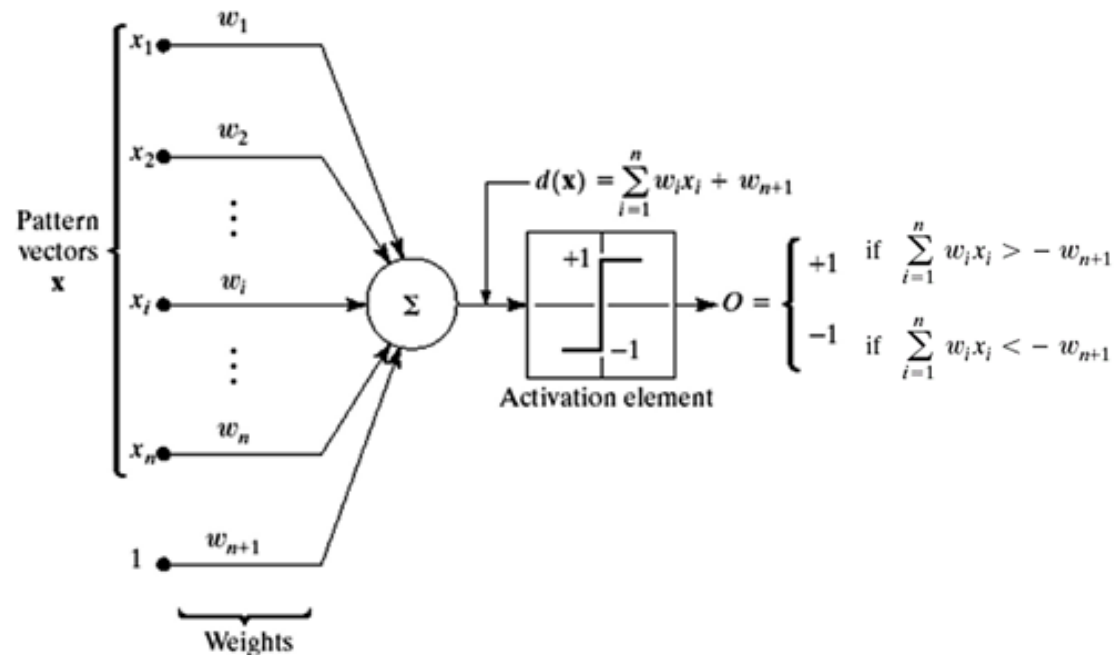
- The first learning machine: the **Perceptron**
 - Built at Cornell in 1960
- The Perceptron was a **linear classifier** on top of a simple **feature extractor**
- The vast majority of practical applications of ML today use **linear classifiers**.
- Designing a feature extractor requires considerable efforts by experts.



$$y = \text{sign} \left(\sum_{i=1}^N W_i F_i(X) + b \right)$$

Neural Networks

- Motivation: The statistical characteristics of pattern classes are often unknown but can be directly obtained through **training** and generating the required decision functions.
- Neural networks: They employ a large number of basic **nonlinear** computational units (neurons) organized in a network-like structure, similar to interconnected neurons in the brain.
- Perceptions of two pattern classes: Two equivalent forms.



$$d(x) = \sum_{i=1}^n w_i x_i + w_{n+1}$$

$$O = \begin{cases} +1, & \sum_{i=1}^n w_i x_i > -w_{n+1} \\ -1, & \sum_{i=1}^n w_i x_i < -w_{n+1} \end{cases}$$

Neural Networks

- **Vectorized representation:**

- $y = \{x_1, x_2, \dots, x_n, 1\}^T, w = \{w_1, w_2, \dots, w_n, w_{n+1}\}^T$, so the decision function is:

$$d(y) = \sum_{i=1}^{n+1} w_i x_i = w^T y$$

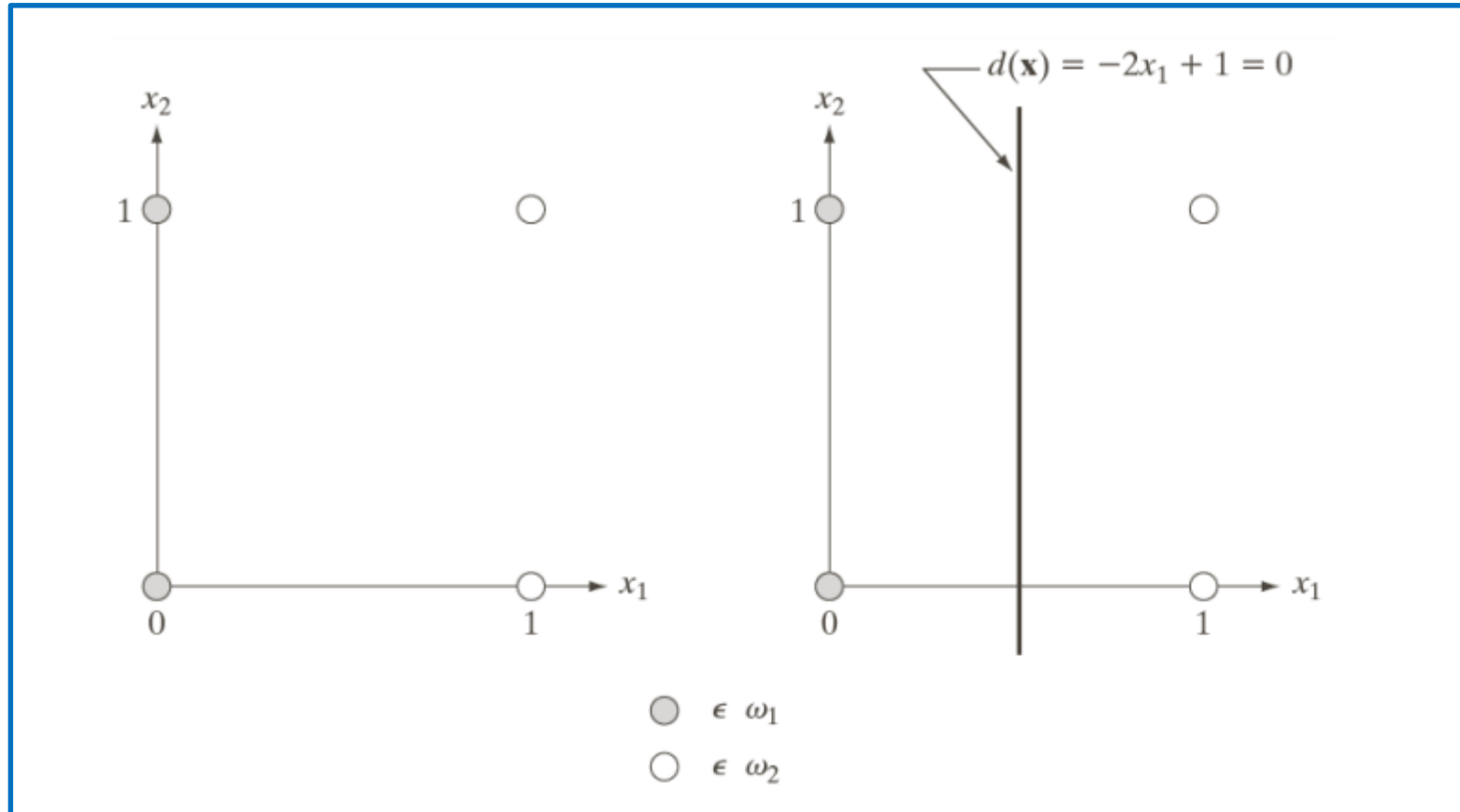
- **Linearly separable class:**

- **Ideal case:** if $y \in C_1$, $w^T y(k) > 0$; if $y \in C_2$, $w^T y(k) < 0$

- **Training algorithm:**

- if $y \in C_1$ and $w^T y \leq 0$, then $w(k+1) = w(k) + \alpha y$
- if $y \in C_2$ and $w^T y \geq 0$, then $w(k+1) = w(k) - \alpha y$
- Otherwise, $w(k)$ remain the same
- The algorithm converges when the entire training set of two classes loops through the machine without any errors.

Neural Networks



Is a single-layer Perception enough?

Example XOR

$$x_1, x_2, y \in \{0,1\}$$

Ground truth XOR function $y = f^*(\mathbf{x}) = \begin{cases} 1, & \text{if } x_1 \neq x_2 \\ 0, & \text{if } x_1 = x_2 \end{cases}$

Our model $y = f(\mathbf{x})$

Our training data

x	y
(0,0)	0
(0,1)	1
(1,0)	1
(1,1)	0

Example XOR

- Human design

- Loss function: L2 norm (not optimal)

$$\text{Loss function } J(\theta) = \sum (f^*(x) - f(x, \theta))^2$$

$$f(x, \theta) = f(x; \mathbf{w}, b) = \mathbf{x}^T \mathbf{w} + b$$

- Linear regression model

$$w, b = \arg \min J(w, b)$$

$$w, b = \arg \min ((0 - b)^2 + (1 - w_1 - b)^2 + (1 - w_2 - b)^2 + (w_1 + w_2 + b)^2)$$

- Solution

$$\frac{\partial J}{\partial w_1} = -2(1 - w_1 - b) + 2(w_1 + w_2 + b) = 0 \quad \longrightarrow \quad w_2 = 0 \quad w_1 = 0$$

$$\frac{\partial J}{\partial b} = 2b - 2(1 - b) - 2(1 - b) + 2b = 0 \quad \longrightarrow \quad b = 1/2$$

Example XOR

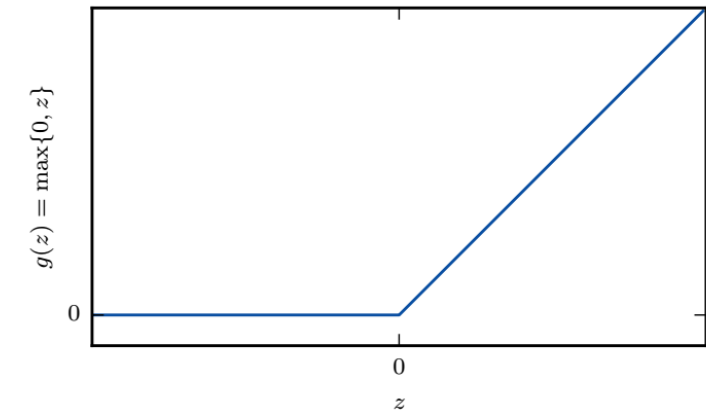
- Solution $\hat{y} = f(x; w, b) = 1/2$ **Not working!**
- Linear function **cannot** approximate XOR (nonlinear)
- Introduce
 - A hidden layer

$$h = f_1(x; w, b)$$

$$y = f_2(h, w)$$

- Introduce nonlinear in f_1
 - Rectified linear unit (ReLU) $f_1(x) = \max\{0, x\}$

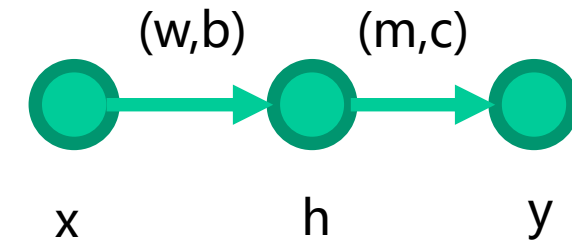
$$h = f_1(x^T w + b)$$



Example XOR

- New model

$$y = m^T \max\{0, w^T x + b\} + c$$



- Training is skipped!!! Solution only

$$y = \begin{bmatrix} 1 \\ -2 \end{bmatrix}^T \max\{0, \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} x + \begin{bmatrix} 0 \\ -1 \end{bmatrix}\}$$

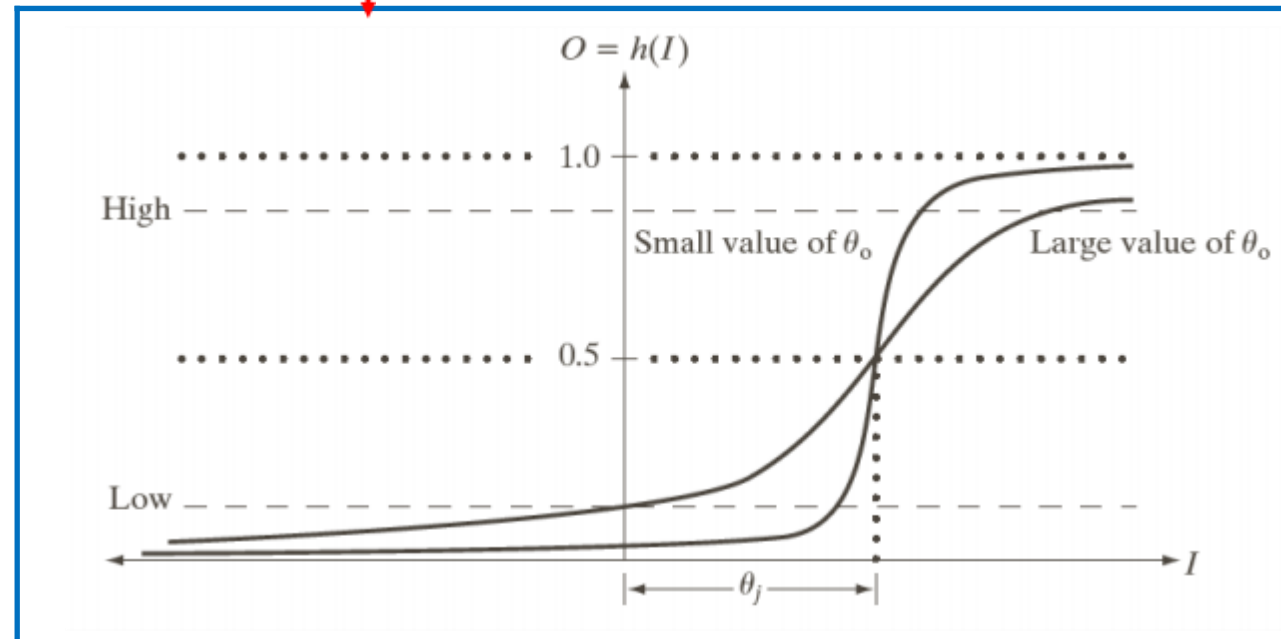
$$\begin{aligned}
 x &= \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} & x^T w &= \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix} & x^T w + b &= \begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} & \text{ReLU}(x^T w + b) &= \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} & m^T \text{ReLU}(x^T w + b) &= \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}
 \end{aligned}$$

Activation Function

- Multilayer feedforward neural network
 - Activation functions are differentiable.

Activation
functions

$$h_j(I_j) = \frac{1}{1 + e^{-(I_j + \theta_j)/\theta_o}}$$

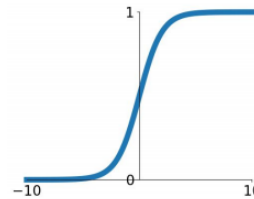


Activation Function

- Activation Functions

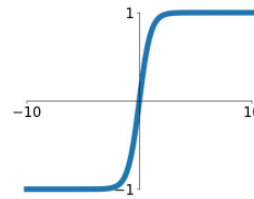
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



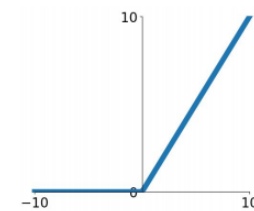
tanh

$$\tanh(x)$$



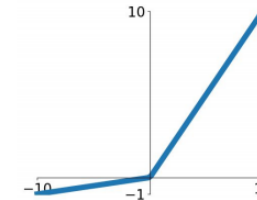
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

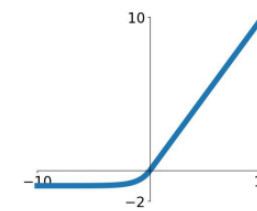


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

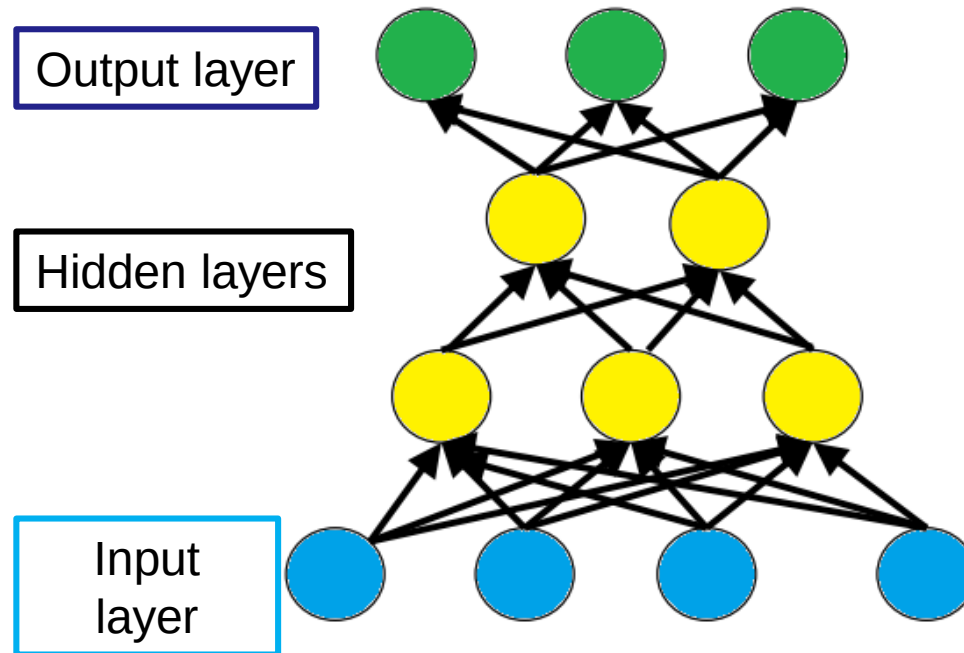
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



- Using elementary functions and their translations to form "basis functions" (normalized).

Multi-layer Perceptron



Width: number of neurons of each layer

Depth: number of layers

■ Properties of architecture

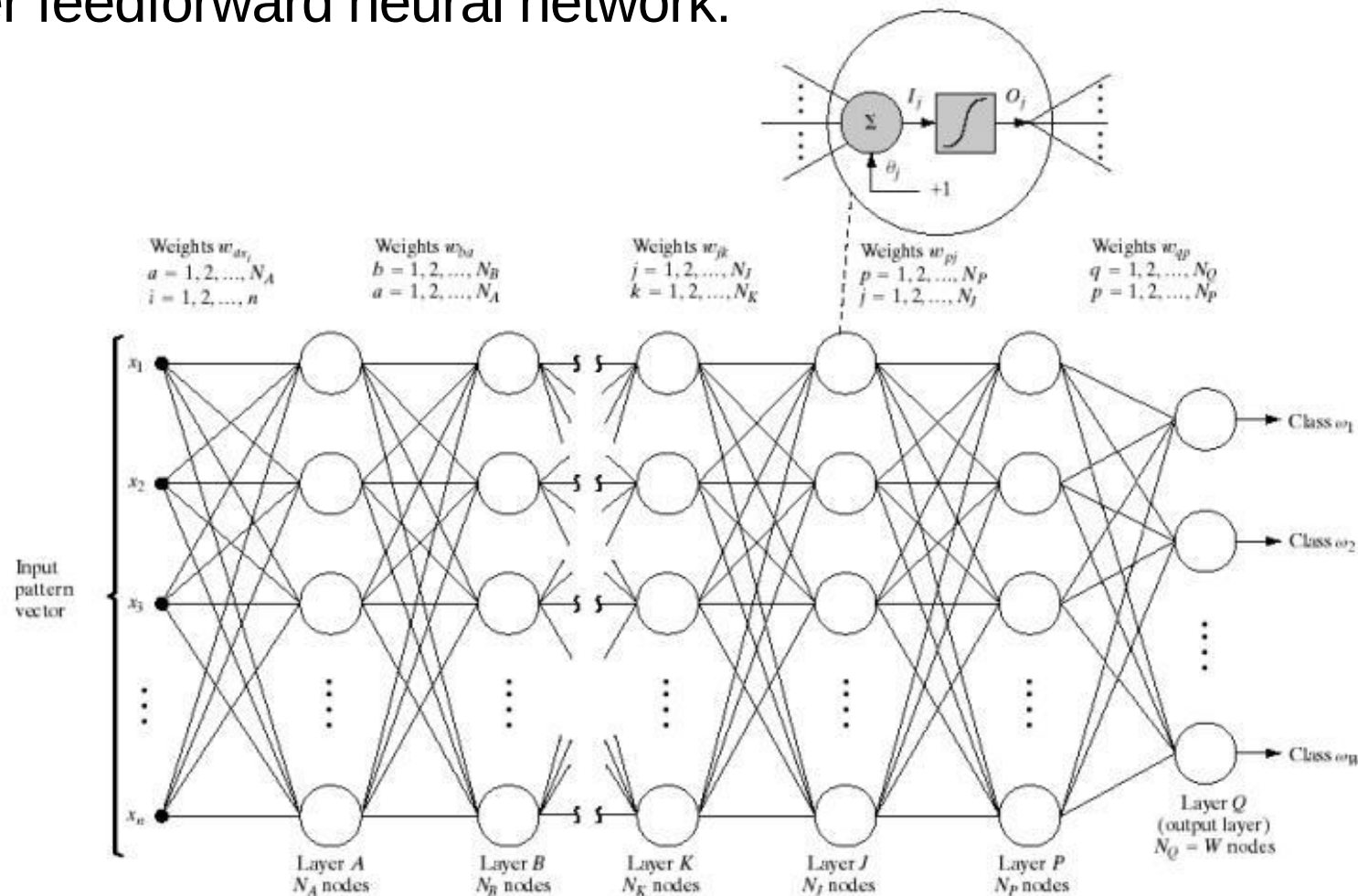
- No connections within a layer
- No direct connections between input and output layers
- Fully connected between layers

■ Universal approximation theorem:

A feed-forward network with a single hidden layer containing a finite number of neurons (a multi-layer perceptron, MLP), can approximate continuous functions on compact subset of $\mathbf{R}^n$, under mild assumptions on the activation function.

Neural Network Architecture

- Multilayer feedforward neural network:



Training a neural network

- **Error backward propagation algorithm (BP)**

- Training sample set

- Cost function

- The cost function for a single sample $\mathbf{x}$ and y :

$$J(\mathbf{W}, b; \mathbf{x}, y) = \frac{1}{2} \| h_{\mathbf{W}, b}(\mathbf{x}) - y \|^2$$

- The whole cost function for the training set:

$$\begin{aligned} J(\mathbf{W}, b) &= \left[\frac{1}{m} \sum_{i=1}^m J(\mathbf{W}, b; x^{(i)}, y^{(i)}) \right] + \frac{\lambda}{2} \sum_{l=1}^{n_l-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (W_{ji}^{(l)})^2 \\ &= \left[\frac{1}{m} \sum_{i=1}^m \left(\frac{1}{2} \| h_{\mathbf{W}, b}(\mathbf{x}^{(l)}) - y \|^2 \right) \right] + \frac{\lambda}{2} \sum_{l=1}^{n_l-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (W_{ji}^{(l)})^2 \end{aligned}$$

- The first term is a **mean square error term** and the second is a **regularization term**.
 - The regularization term is designed to reduce the amplitude of weight and prevent overfitting.

Training a neural network

- **Back Propagation algorithm**

- Parameters are initialized randomly
- Carry out feedforward conduction calculation and use forward conduction formula to get the output

value of $L_2, L_3, \dots, L_{n_l}$

- For each output cell i in layer n_l (output layer) :

$$h_{\mathbf{W},b}(\mathbf{x}) = a_i^{(n_l)} = f(z_i^{(n_l)})$$

- We calculate the residual error according to the following formula:

$$\delta_i^{(n_l)} = \frac{\partial}{\partial z_i^{(n_l)}} J(\mathbf{W}, b, \mathbf{x}, y) = \frac{\partial}{\partial z_i^{(n_l)}} \frac{1}{2} \| y - h_{\mathbf{W},b}(\mathbf{x}) \|^2 = - \left(y - a_i^{(n_l)} \right) \cdot f' \left(z_i^{(n_l)} \right)$$

Training a neural network

- Back Propagation algorithm**

- The residual error of each layer of $l = n_l - 1, n_l - 2, n_l - 3, \dots, 2$ and the i th node of layer l is calculated as follows:

$$\begin{aligned}
 \delta_i^{(n_l-1)} &= \frac{\partial}{\partial z_i^{(n_l-1)}} J(\mathbf{W}, b; \mathbf{x}, y) \\
 &= \sum_j \frac{\partial J(\mathbf{W}, b; \mathbf{x}, y)}{\partial z_j^{(n_l)}} \cdot \frac{\partial z_j^{(n_l)}}{\partial z_i^{(n_l-1)}} \\
 &= \sum_j \delta_j^{(n_l)} \cdot \frac{\partial}{\partial z_i^{(n_l-1)}} \left[\sum_k W_{jk}^{(n_l-1)} f(z_k^{(n_l-1)}) \right] \\
 &= \sum_j \delta_j^{(n_l)} \cdot \frac{\partial}{\partial z_i^{(n_l-1)}} \left[W_{ji}^{(n_l-1)} f(z_i^{(n_l-1)}) \right] \\
 &= \sum_j \delta_j^{(n_l)} \cdot W_{ji}^{(n_l-1)} \cdot f'(z_i^{(n_l-1)}) \\
 &= f'(z_i^{(n_l-1)}) \cdot \sum_j (W_{ji}^{(n_l-1)} \cdot \delta_j^{(n_l)})
 \end{aligned}$$

- The above equation can be equivalently expressed as:

$$\delta_i^{(l)} = \left(\sum_{j=1}^{s_{l+1}} W_{ji}^{(l)} \delta_j^{(l+1)} \right) f'(z_i^{(l)})$$

Training a neural network

- **Back Propagation algorithm**

- Calculate the required partial derivative as follows:

$$\frac{\partial}{\partial W_{ij}^{(l)}} J(\mathbf{W}, b; \mathbf{x}, y) = \frac{\partial J(\mathbf{W}, b; \mathbf{x}, y)}{\partial z_i^{(l+1)}} \cdot \frac{\partial z_i^{(l+1)}}{\partial W_{ij}^{(l)}} = \delta_i^{(l+1)} a_j^{(l)}$$

$$\frac{\partial}{\partial b_i^{(l)}} J(\mathbf{W}, b; \mathbf{x}, y) = \frac{\partial J(\mathbf{W}, b; \mathbf{x}, y)}{\partial z_i^{(l+1)}} \cdot \frac{\partial z_i^{(l+1)}}{\partial b_i^{(l)}} = \delta_i^{(l+1)} \cdot 1 = \delta_i^{(l+1)}$$

- Update parameters:

$$\begin{aligned} W_{ij}^{(l)} &= W_{ij}^{(l)} - \alpha \left[\frac{1}{m} a_j^{(l)} \delta_i^{(l+1)} + \lambda W_{ij}^{(l)} \right] \\ b_i^{(l)} &= b_i^{(l)} - \frac{\alpha}{m} \delta_i^{(l+1)} \end{aligned}$$

Training a neural network

- Gradient Descent algorithm
 - Batch gradient descent
 - Extremely high memory and computation cost
 - Mini-batch gradient descent
 - Batch-size = 40, 256 ... etc
 - Quickly converges
 - **Stochastic gradient descent**
 - Batch-size = 1
 - In DL, SGD~Mini-batch GD

- Step decay
 $\eta^{t+k} = \lambda \eta^t, \quad k=20, \quad \lambda=0.1$ **Learning rate**
- Exponential decay
 $\eta^t = \eta_0 e^{-kt}$ Parameter update
- 1/t decay $\eta^t = \frac{\eta_0}{1+kt}$ $\theta \leftarrow \theta - \boxed{\eta} \frac{\partial L}{\partial \theta}$
- SGD + Momentum
$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta)$$
$$\theta = \theta - v_t$$
- Per-parameter Adaptive Learning Rate

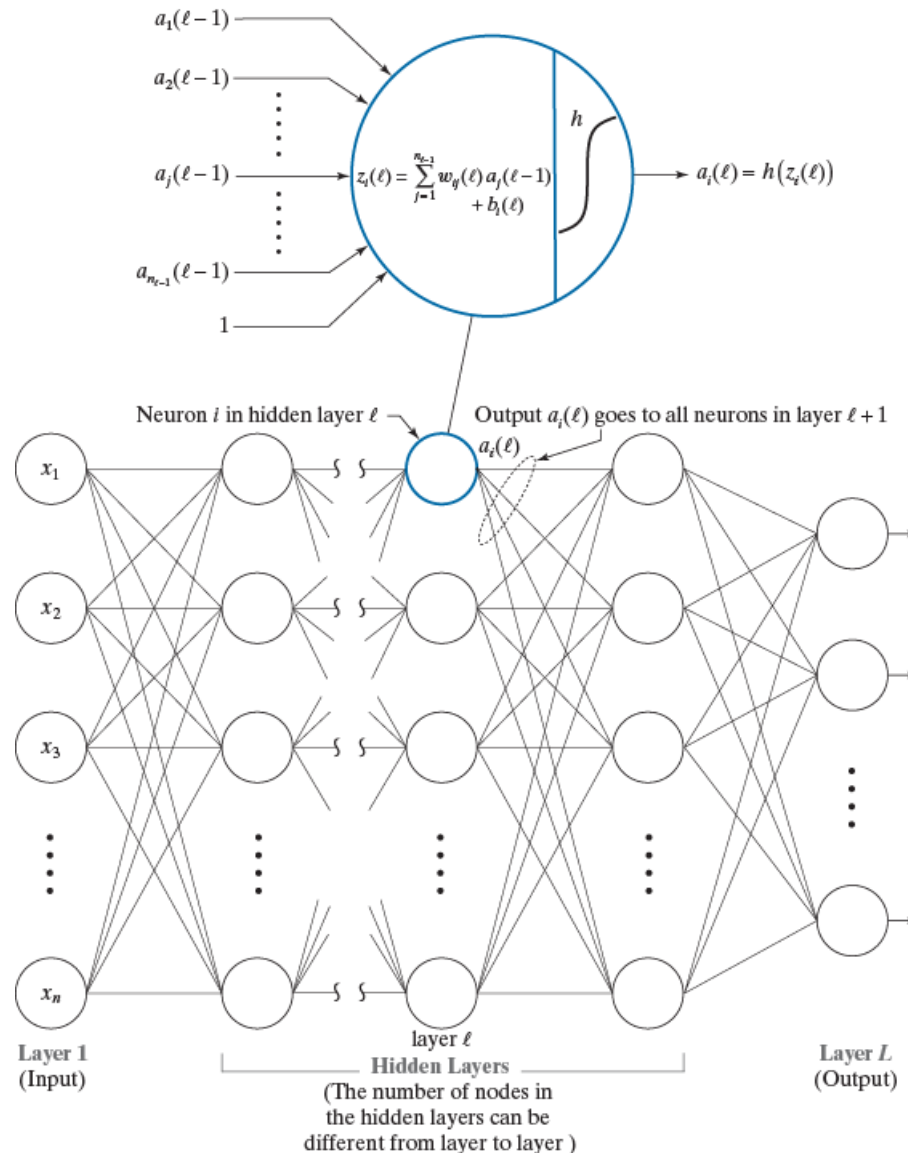


SGD without momentum



SGD with momentum

Deep Neural Networks: Should they all be Fully Connected?



Local correlation
(local visual field)?

Outline

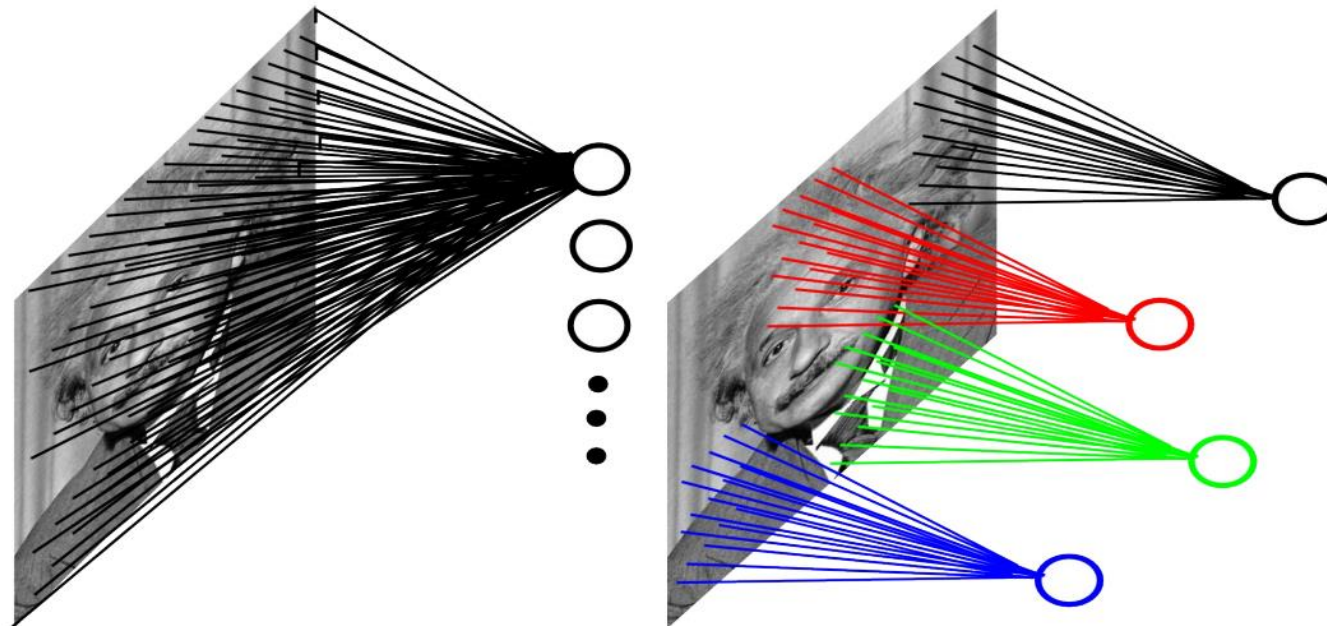
- CNN
 - Basic structure of CNN
 - Introduction to classical networks
 - AlexNet, VGG, ResNet, UNet
- Attention
 - Soft Attention
 - Self Attention: Transformer
- Generative Network
 - GAN
 - Diffusion Model



CNN

Fully-connected neural net in high dimension

- Example: 200x200 image
 - Fully-connected, 400,000 hidden units = 16 billion parameters
 - Locally-connected, 400,000 hidden units 10x10 fields = 40 million paras
 - Local connections capture local dependencies



Convolutional Neural Network

Property 1

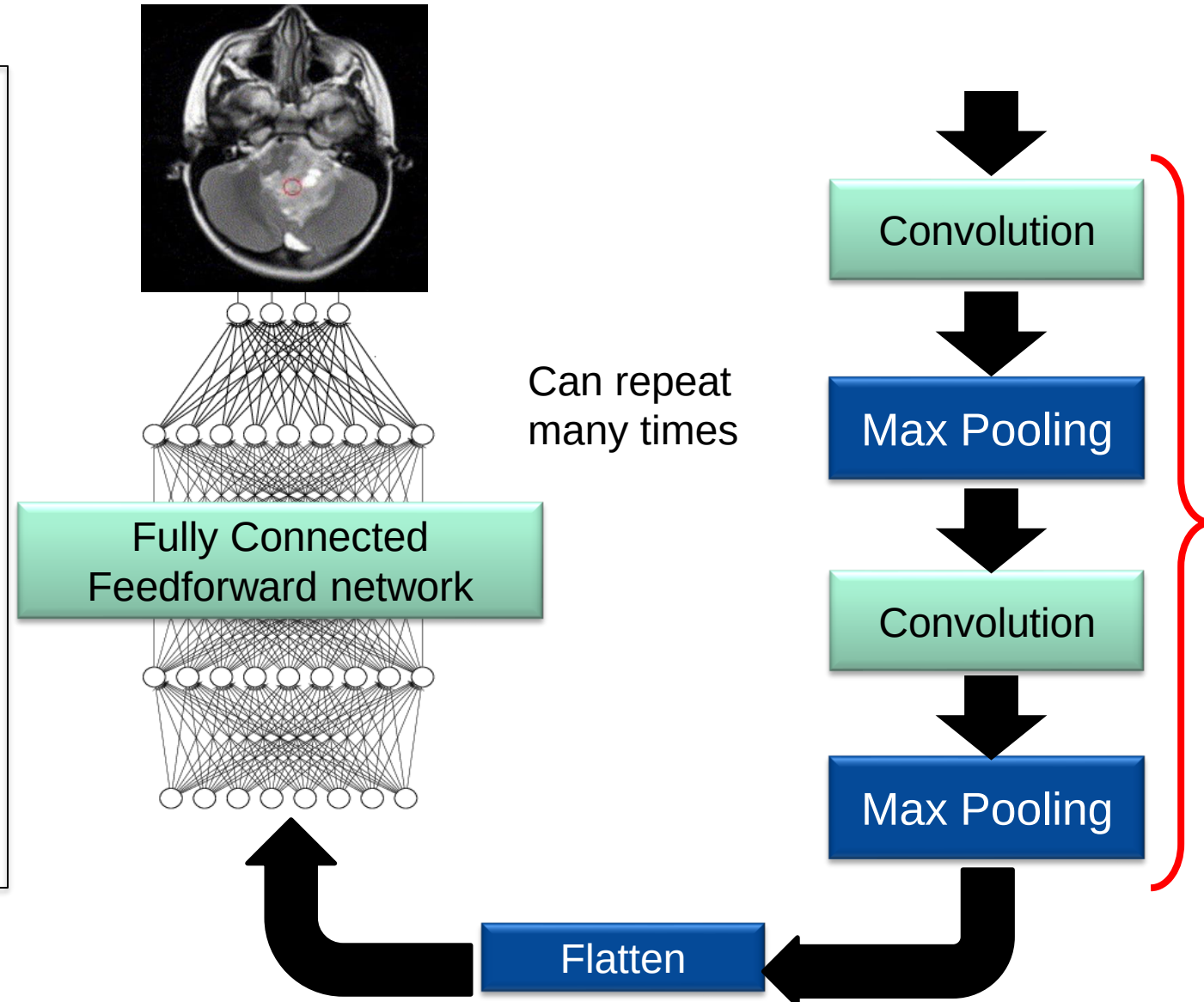
- Some patterns are much **smaller** than the whole image

Property 2

- The **same** patterns appear in different regions.

Property 3

- **Subsampling** the pixels will not change the object



CNN – Convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

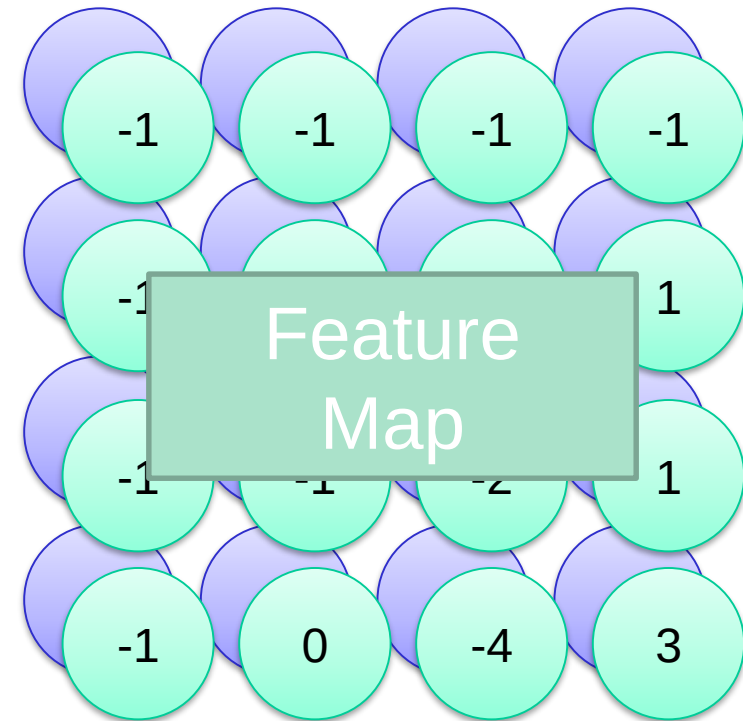


Filter 2

-1	1	-1
-1	1	-1
-1	1	-1

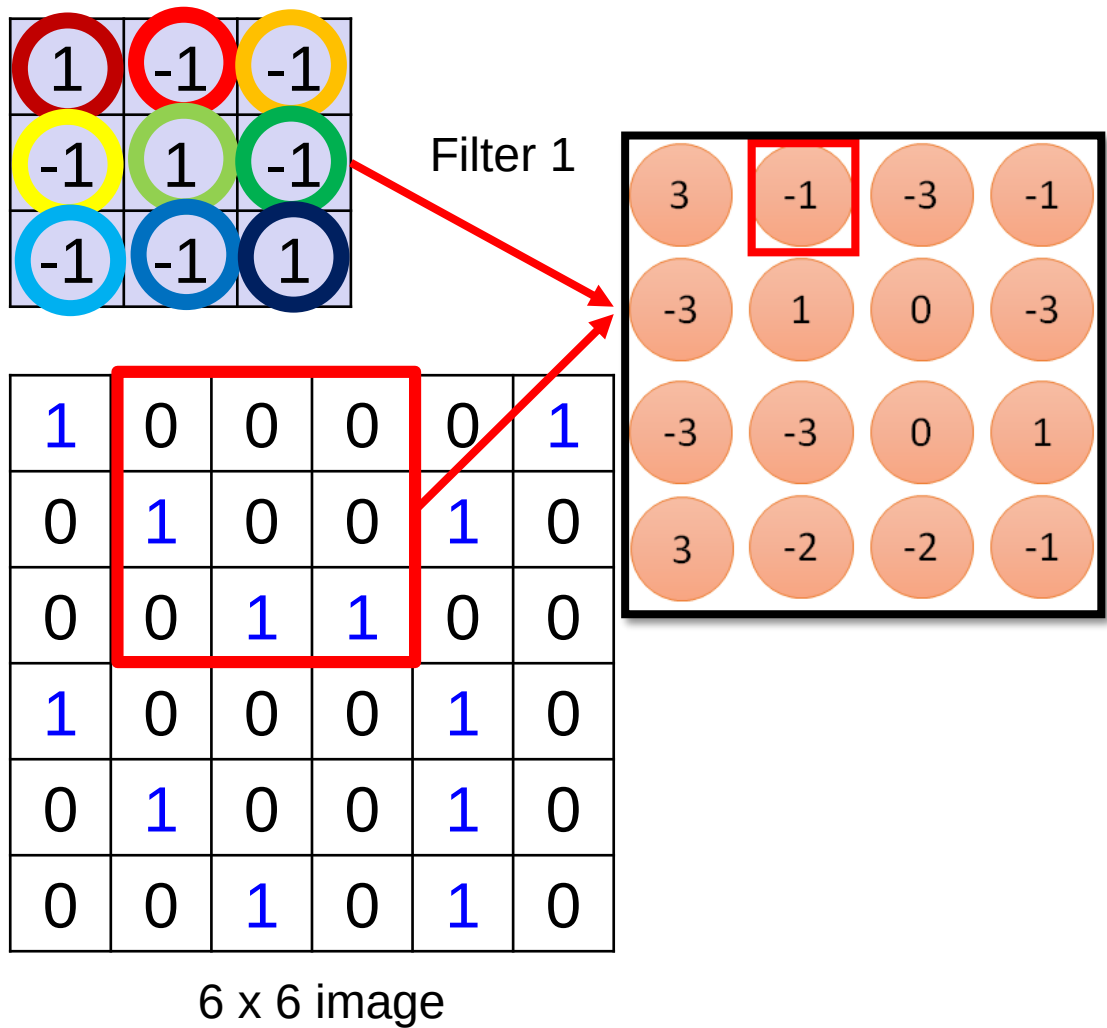


Do the same process
for every filter

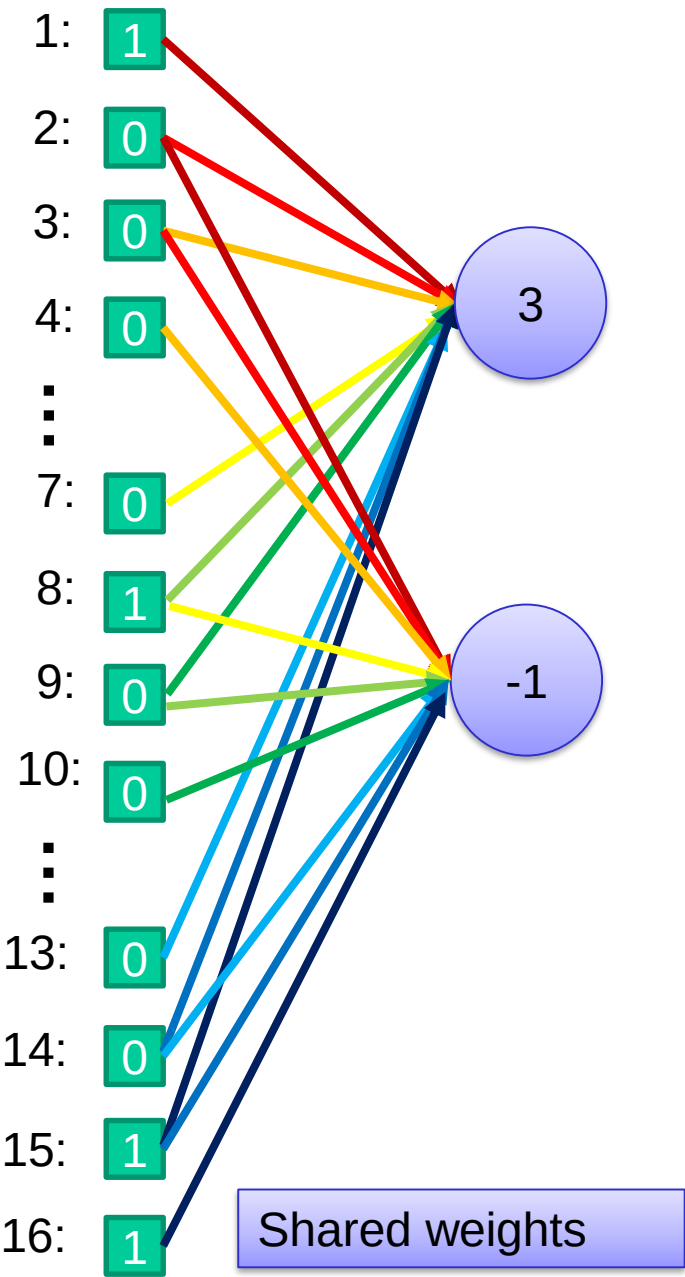


4 x 4 image

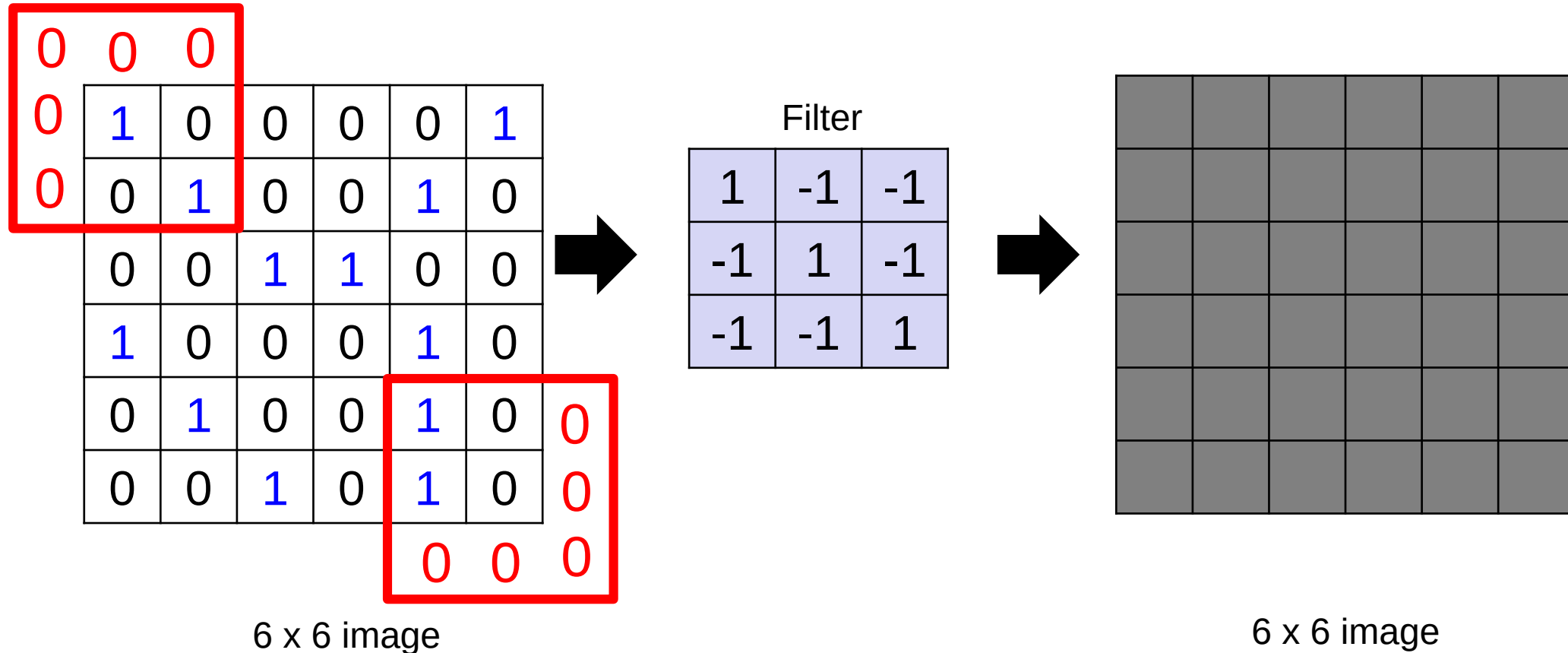
Convolution v.s. Fully Connected



Less parameters!



CNN – Zero Padding



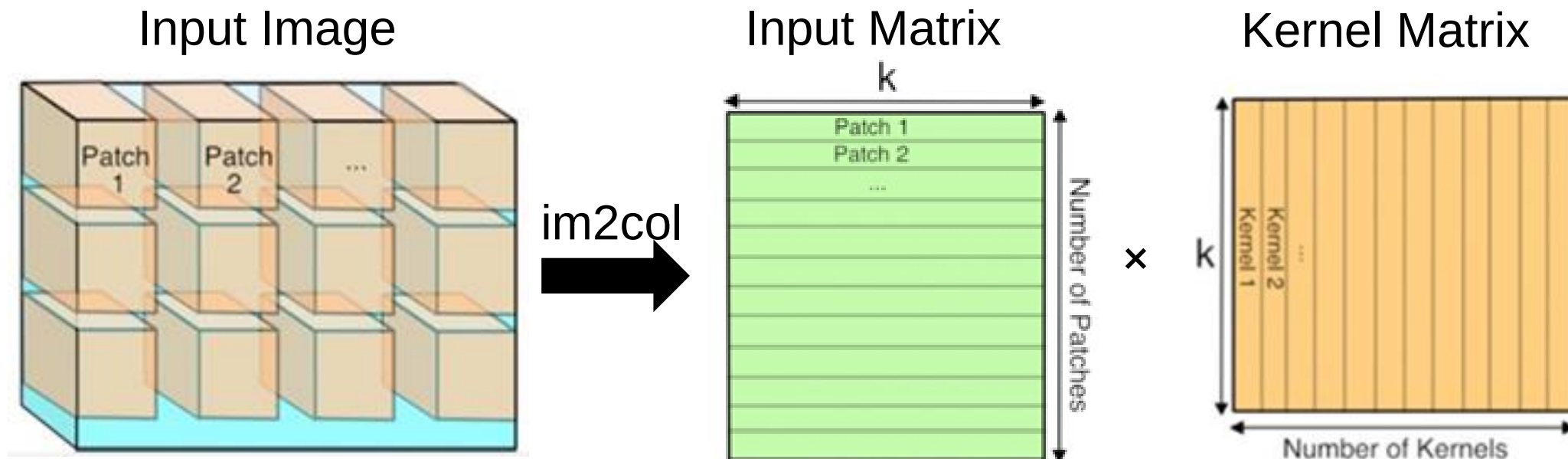
You will get another 6 x 6 images in this way



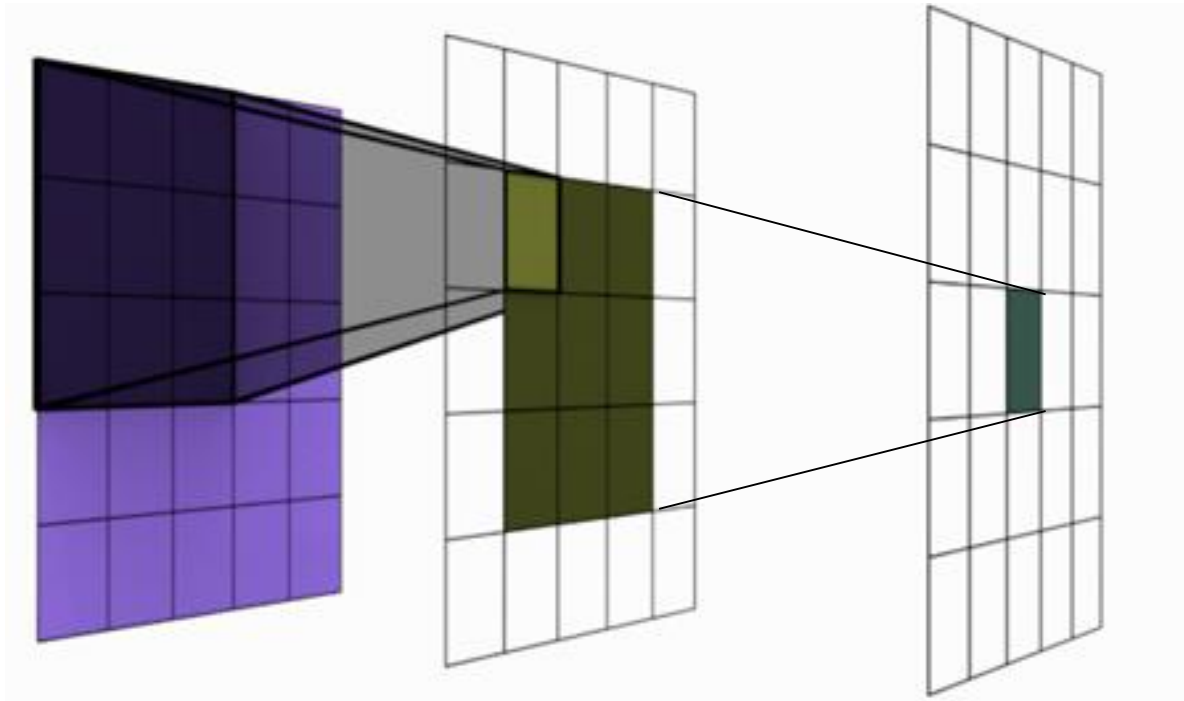
Zero padding

CNN – Matrix operation

- Time-consuming for each convolution kernel to work separately on the entire image.
- The convolution kernel covers each area of the image as a **patch**, then convert each patch into a vector.
- The convolution kernel can also be represented as a matrix.



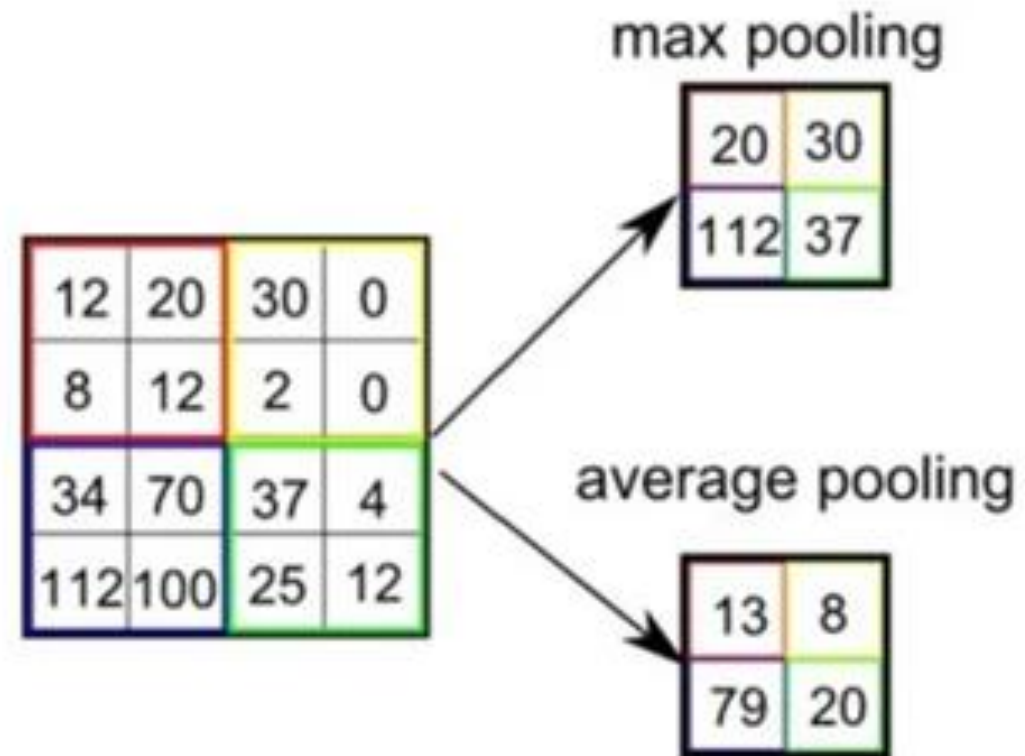
CNN – Receptive Fields



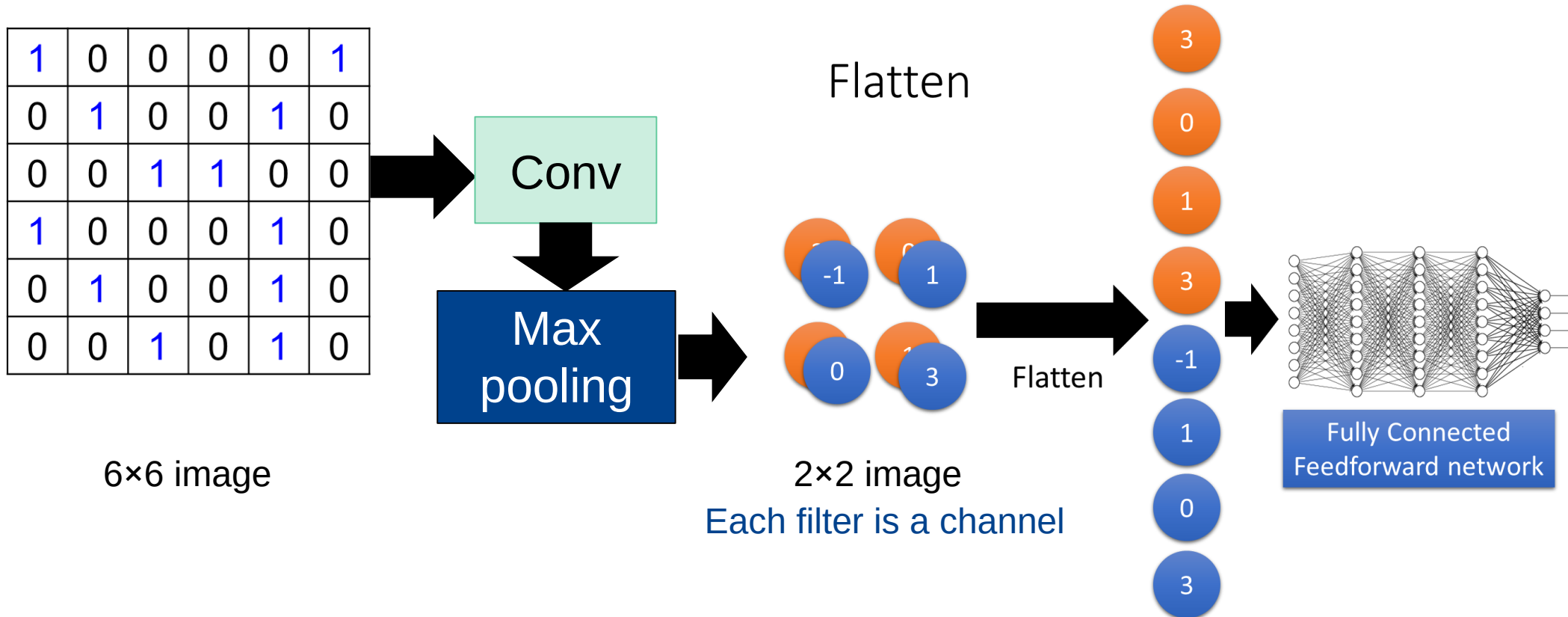
- **Receptive field:** the part of the image that is visible to one filter at a time.
- Two types:
 - Local Receptive Field
 - Global Receptive Field
- **For Layer-1:** Local/Global receptive field is 3×3 .
- **For layer-2:** Local receptive field is 3×3 while Global receptive field is 5×5 .
- In practice we use **global receptive fields**.

CNN – Pooling

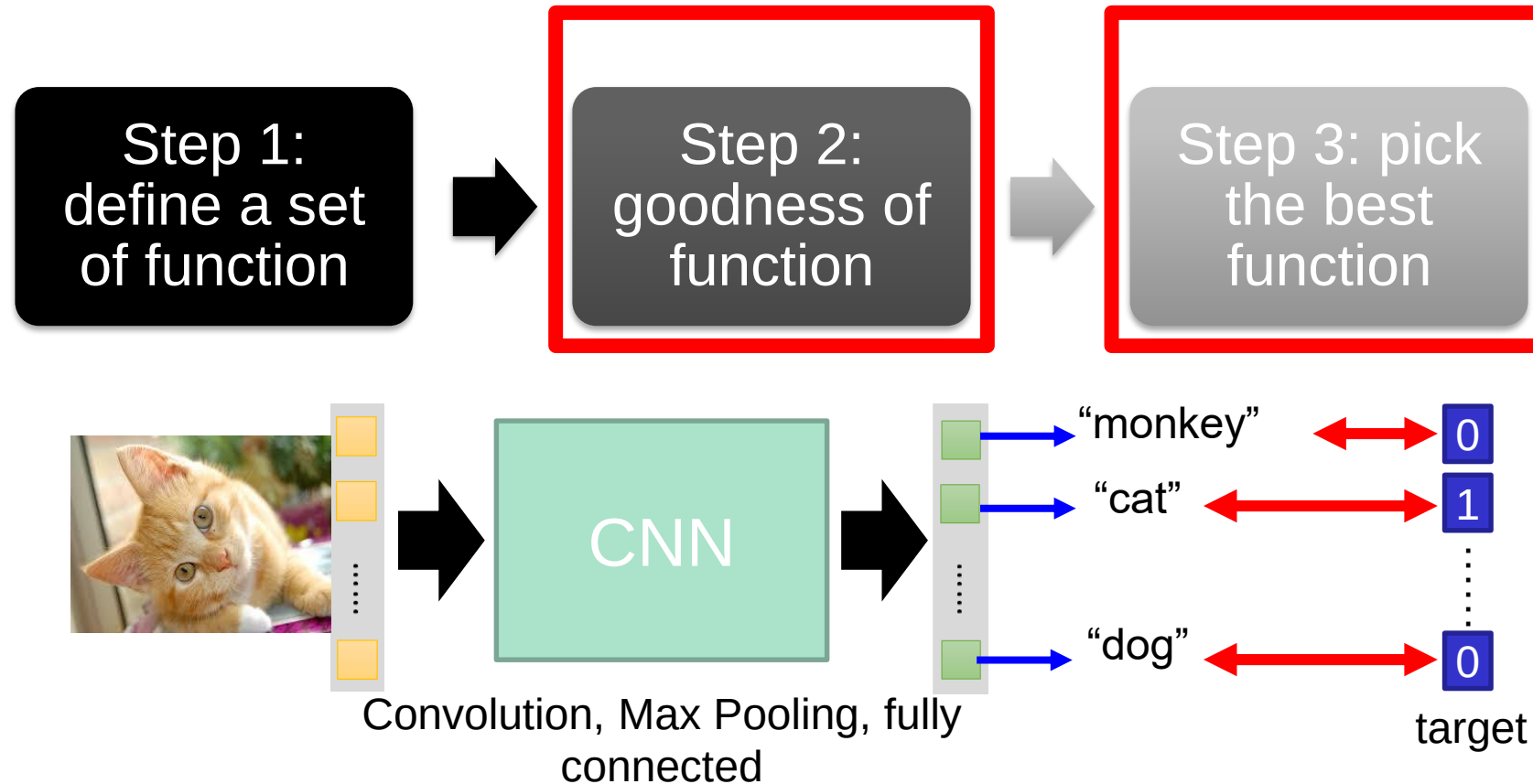
- An approach to down sampling. (Added after the convolutional layer)
- Types:
 - Max Pooling
 - Average Pooling
 - Median Pooling
 - Random Pooling
 - Global Pooling
 - ...
- Advantages:
 - Feature invariance
 - Feature dimension reduction
 - Prevent overfitting to a certain extent



CNN – Max pooling and Flatten

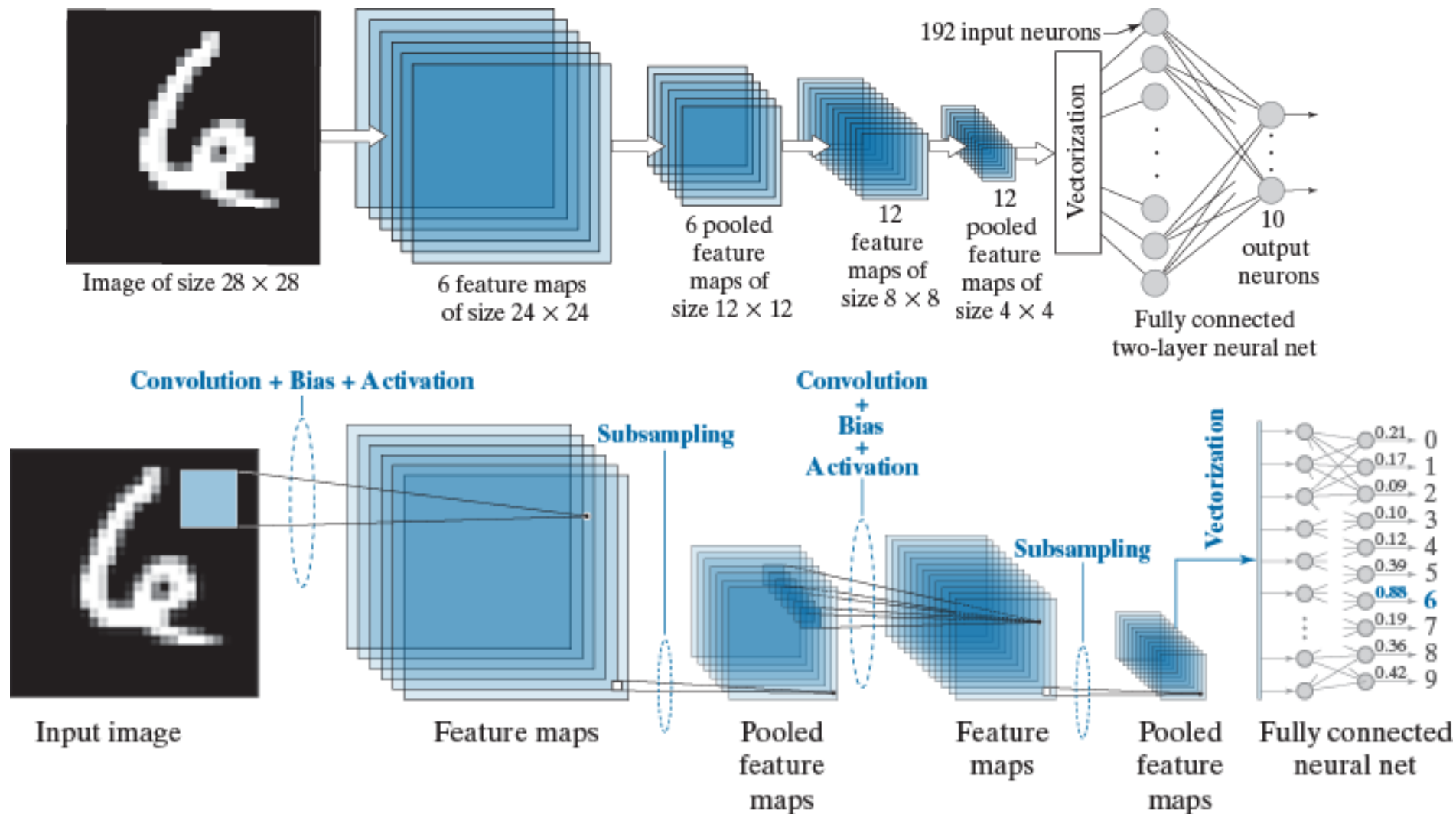


Convolutional Neural Network

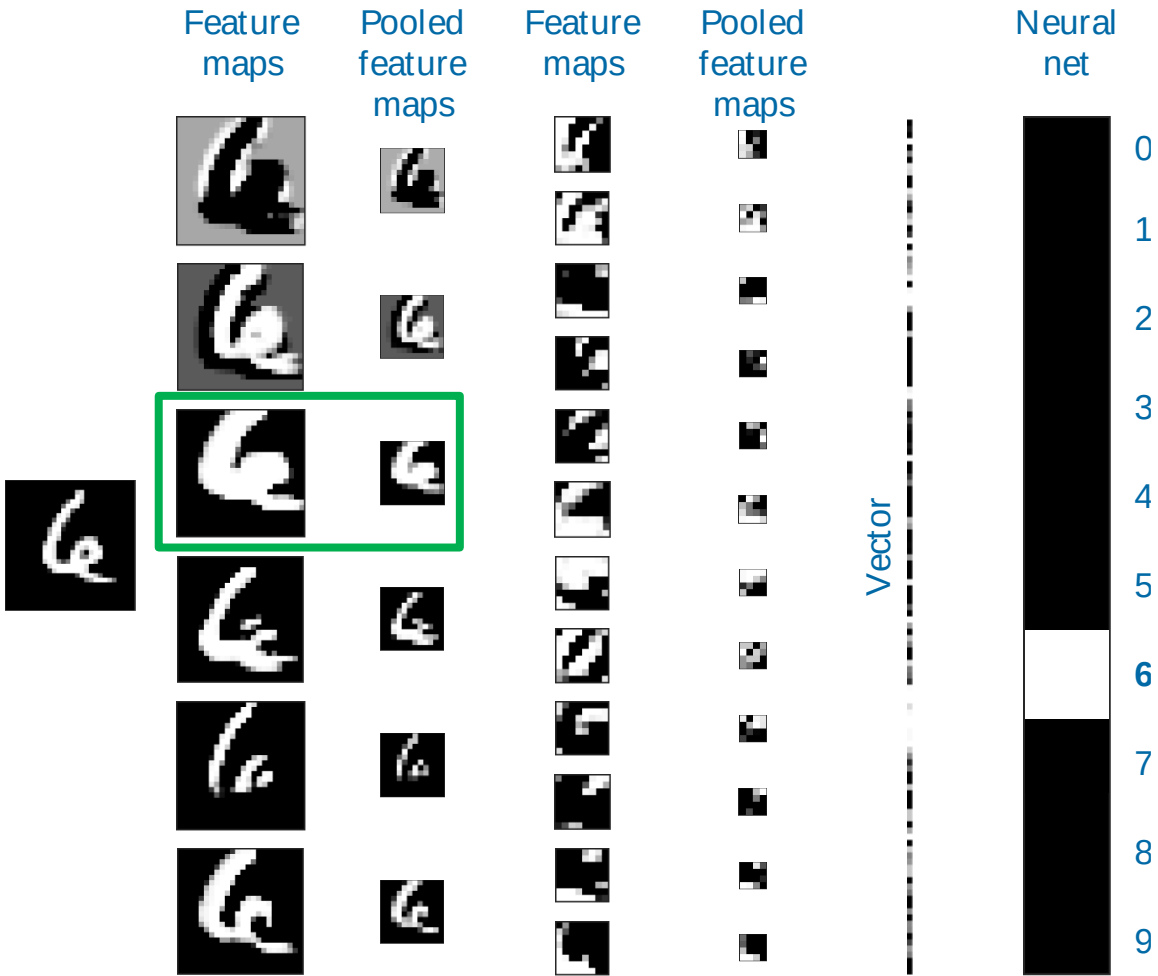


Learning: Nothing special, just gradient descent

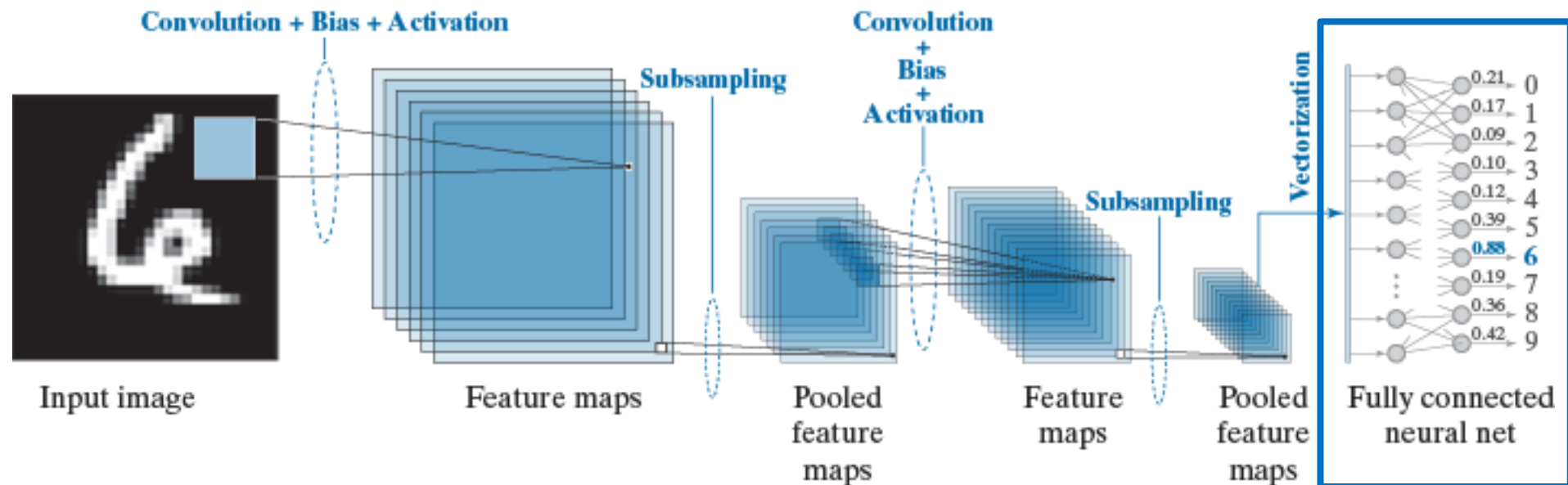
Mnist – CNN: patch



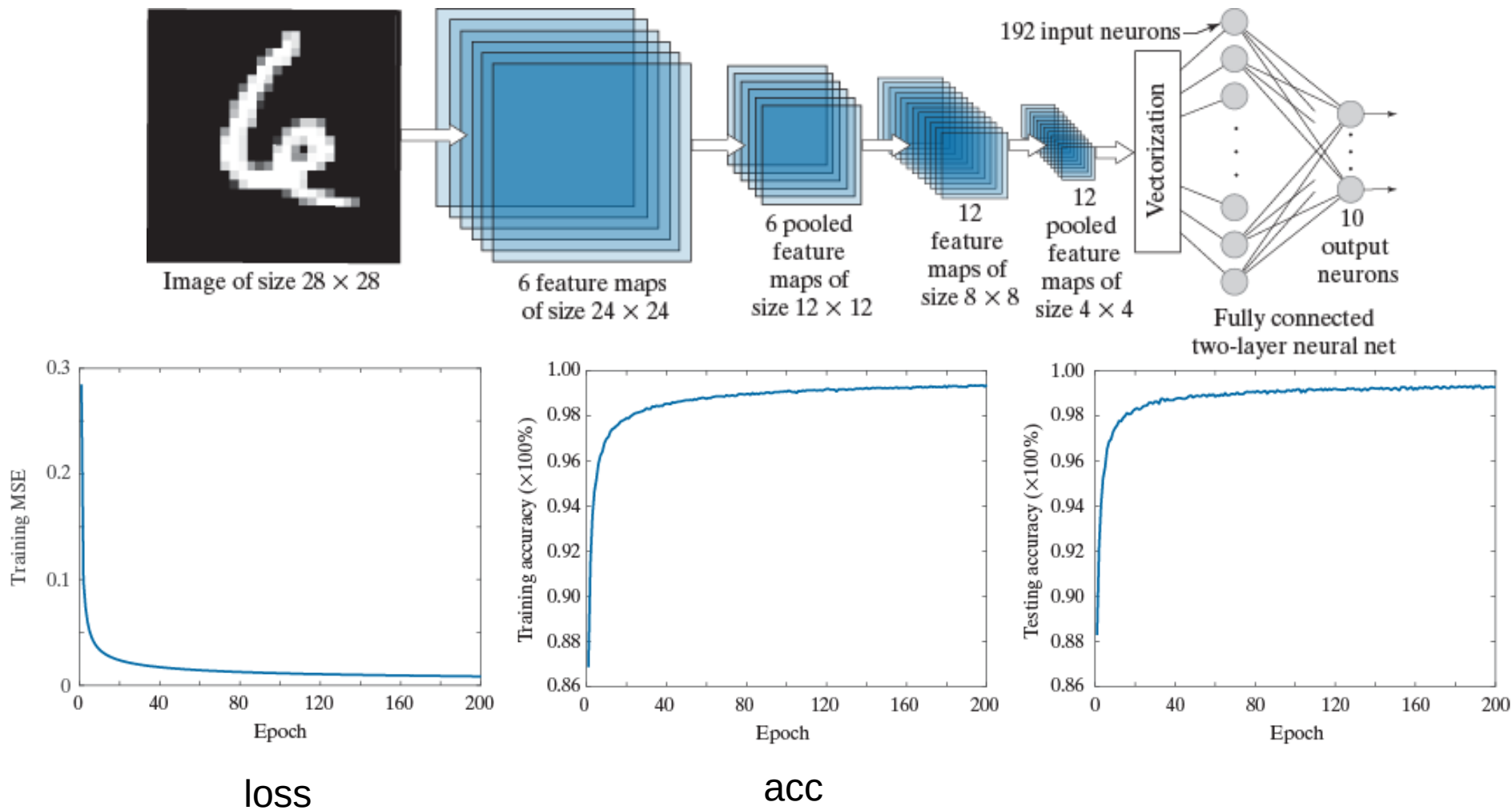
Mnist – CNN: Pooling



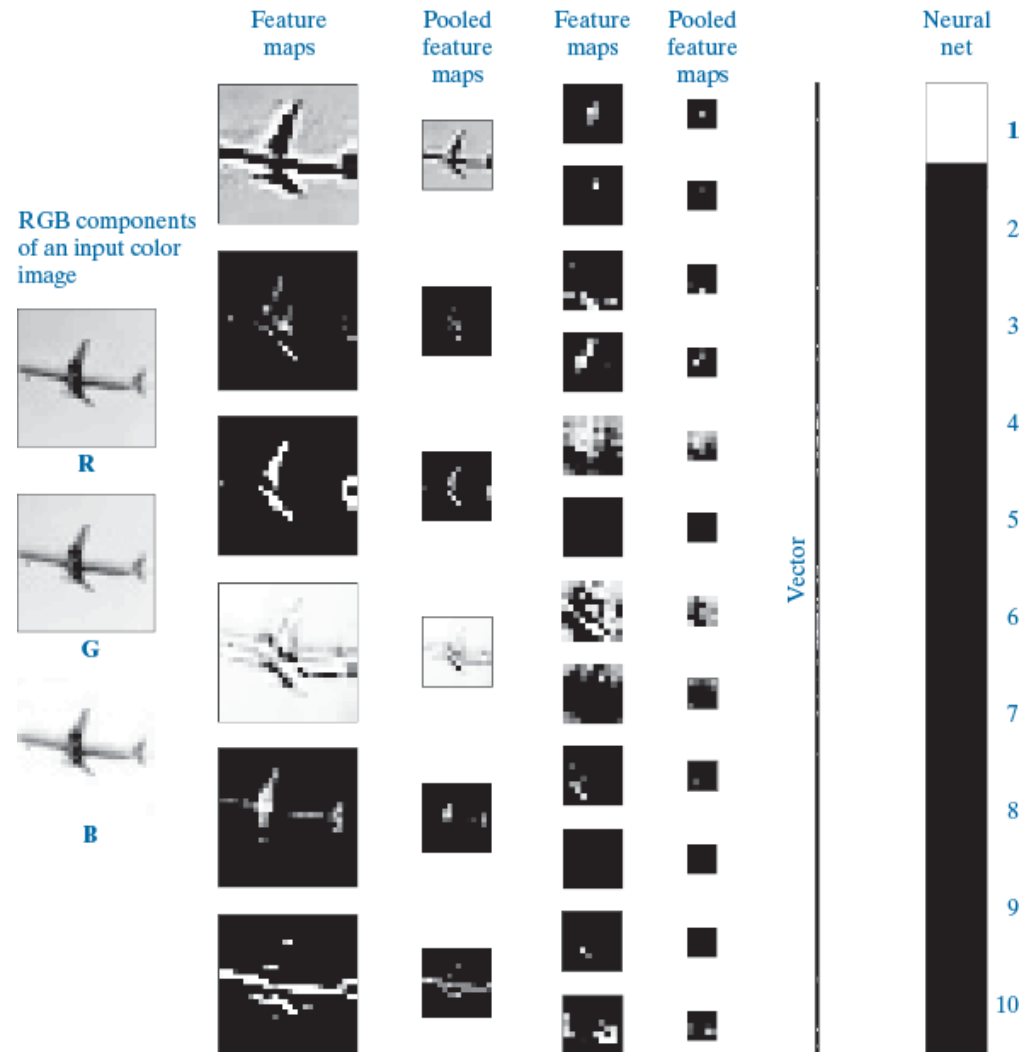
Mnist – CNN: Dense



Example: Mnist



Considerations

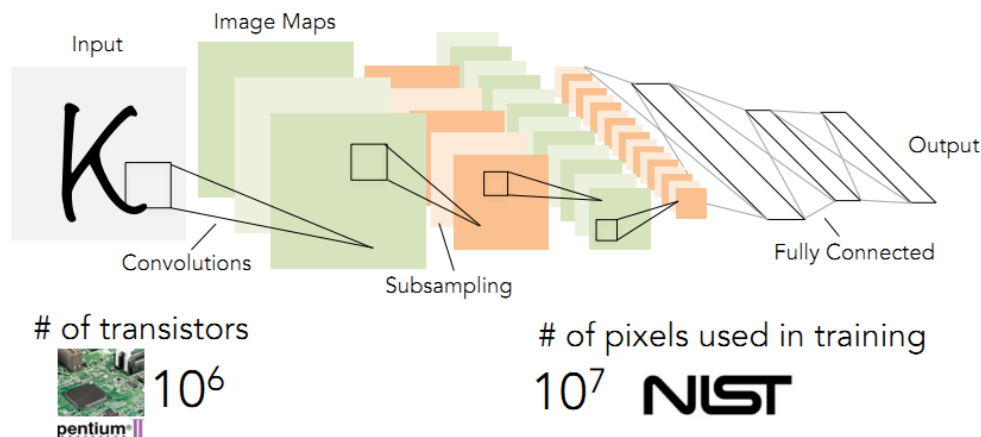


- Compacity
 - Number of layer
 - Number of neuron
- Activation function
- Training
 - Over fitting
 - Speed
- Augmentation

Neural Networks and Deep Learning – Background

1998
LeCun et al.

LeNet



2012

Krizhevsky et al.

AlexNet

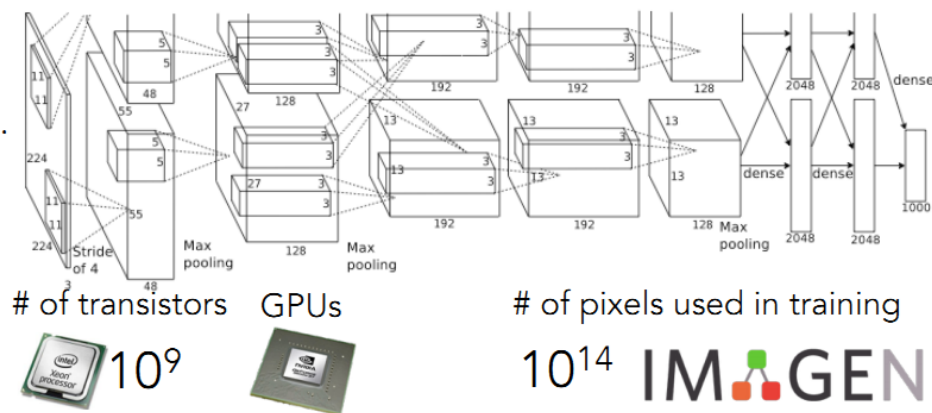


Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.



IMAGENET

www.image-net.org

22K categories and 14M images

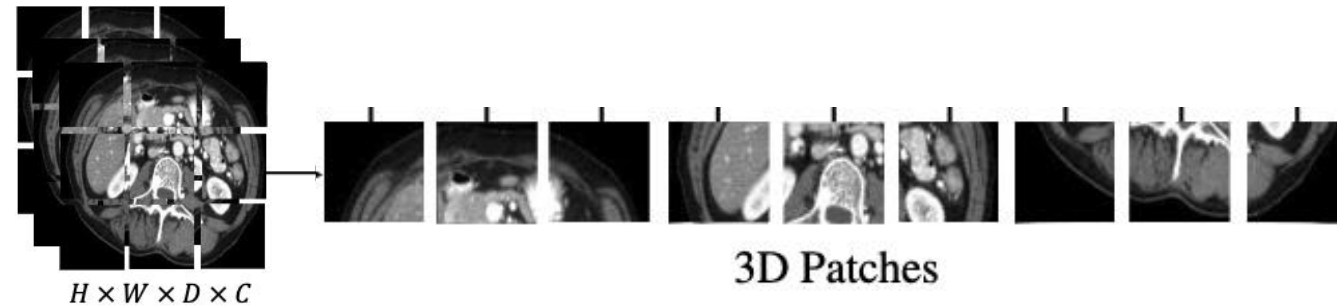
- Animals
 - Bird
 - Fish
 - Mammal
 - Invertebrate
- Plants
 - Tree
 - Flower
- Food
- Materials
- Structures
 - Artifact
 - Tools
 - Appliances
 - Structures
- Person
 - Scenes
 - Indoor
 - Geological Formations
 - Sport Activities



Deng, Dong, Socher, Li, Li, & Fei-Fei, 2009

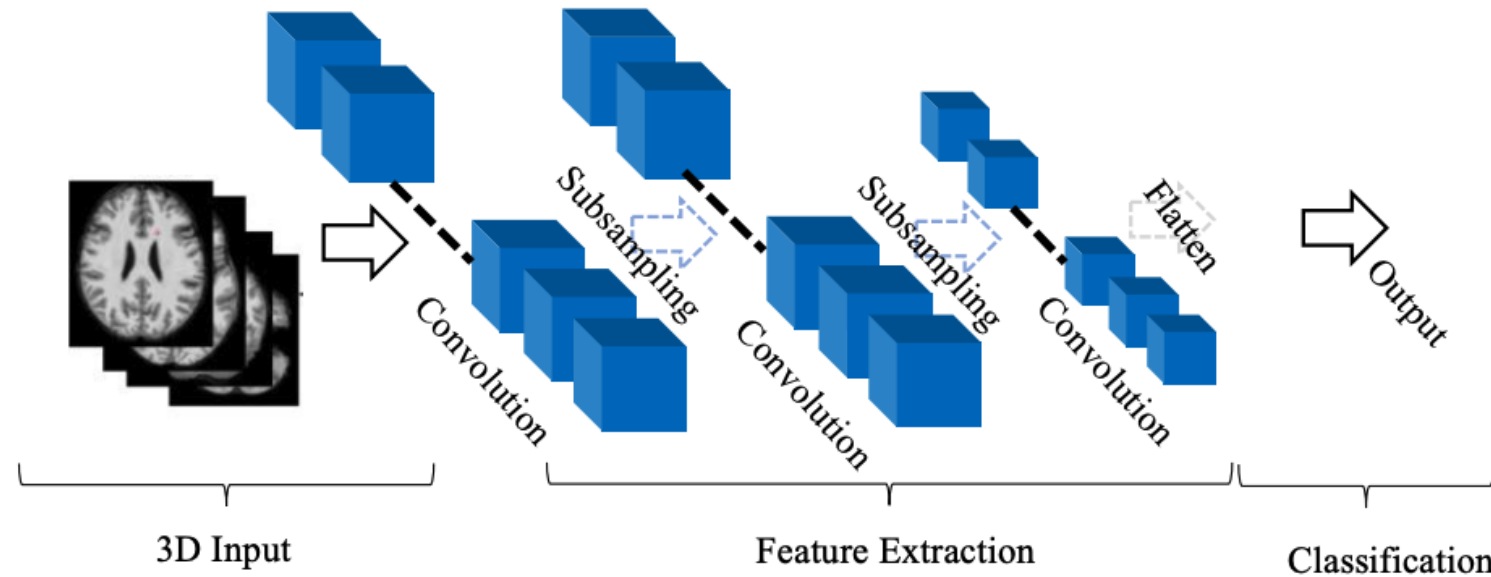
CNN in 3D Medical

- 3D Patch



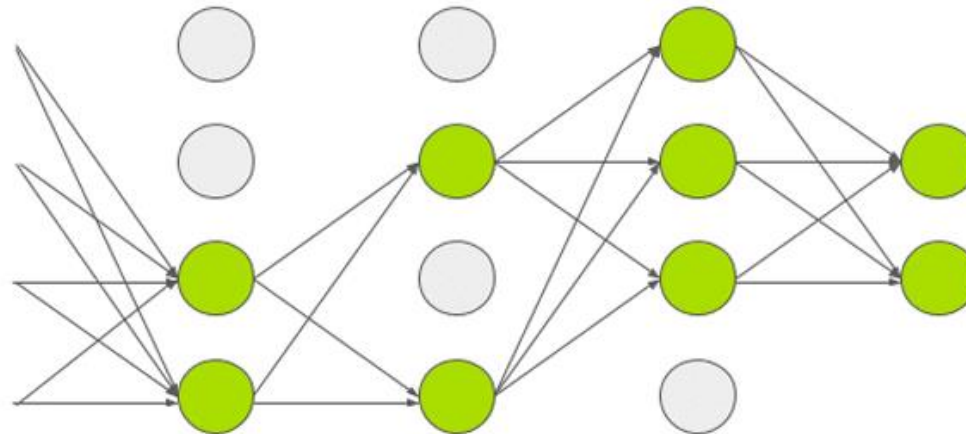
- Norm: z-score(MRI), Normalization of HU value (CT)

- 3D Model:



AlexNet

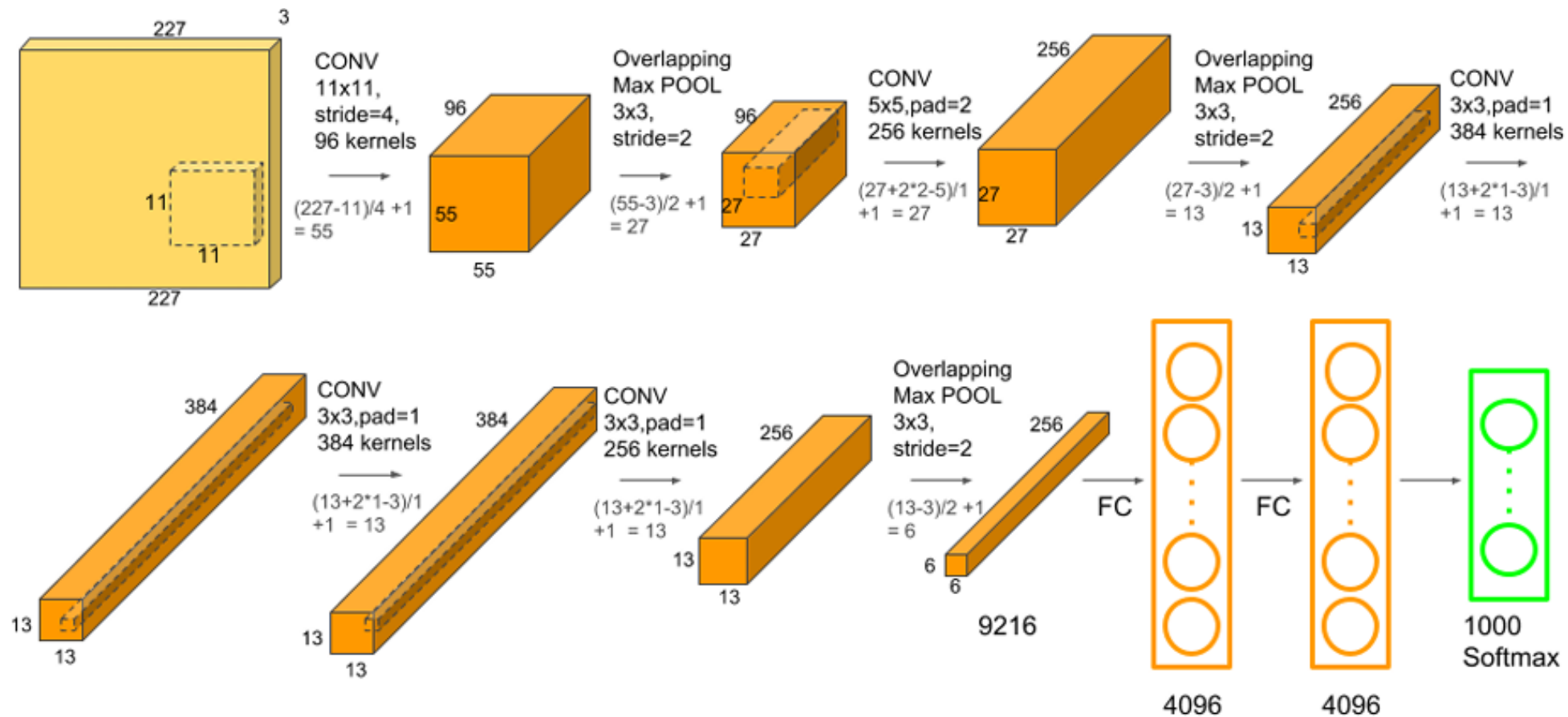
- Background: Geoffrey E. Hinton and his student Alex broke the Image Classification record using Alex-Net in the ILSVRC competition in 2012.
- Activation Functions: ReLU
- Reduce overfitting: Dropout & data enhancement
- Dual GPU → faster training speed.
- Overlapping maximum pooling → avoid the blurring effect of the average pooling.
- Local Response Normalization (LRN) layer → enhance the generalization ability of the model.



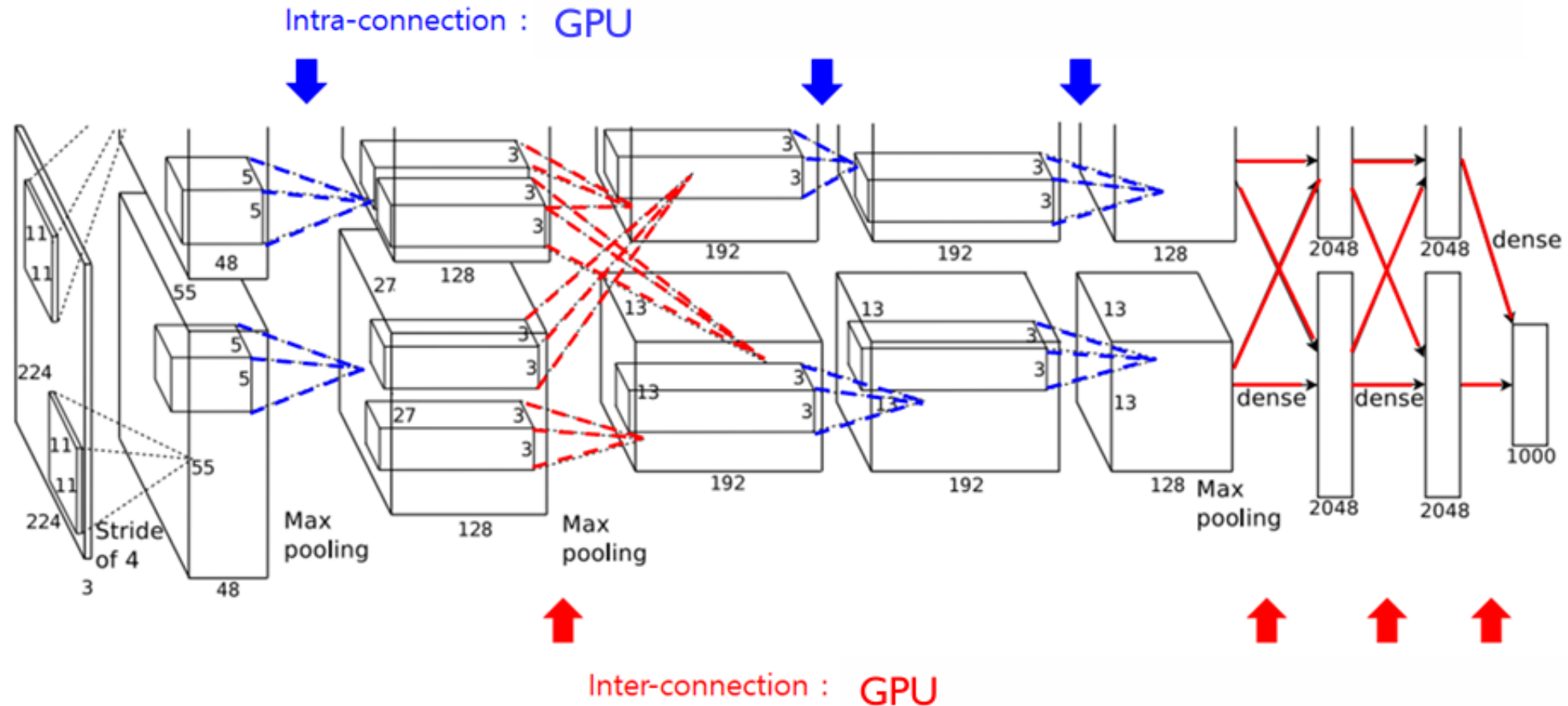
Dropout

- It consists of setting to zero the output of each hidden neuron with a probability of 0.5.
- The neurons which are “dropped out” in this way do not contribute to the forward pass and do not participate in **backpropagation**.

AlexNet



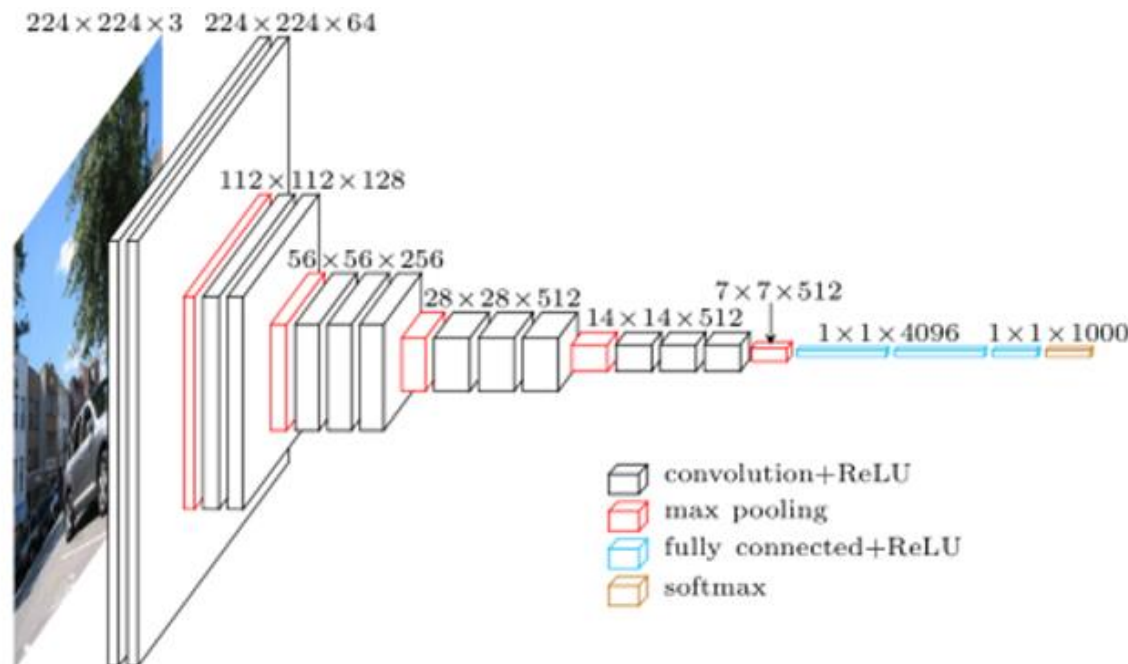
AlexNet



This training approach reduced top-1 and top-5 error rates by 1.7% and 1.2%, respectively, compared to the approach whose kernels were trained half on a single GPU.

VGG

- Background: VGG was proposed by Visual Geometry Group (VGG) , and VGG16 and VGG19 were divided according to the network depth.
- The advantage over AlexNet:
 - Deeper, with convolution layers of 16 (VGG16) or 19 (VGG19).
 - Convolution kernels are unified to 3×3 .

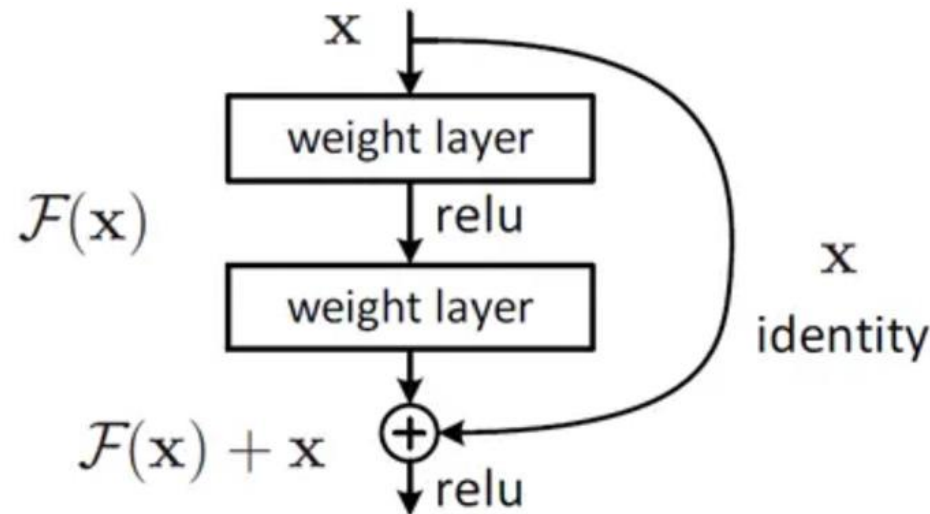


Stacked small convolution kernel:

- Increase the network depth to ensure more complex patterns learnt at a lower cost.

ResNet

- The deeper network, the better?



$$y_l = h(x_l) + F(x_l, W_l)$$

$$x_{l+1} = f(y_l)$$

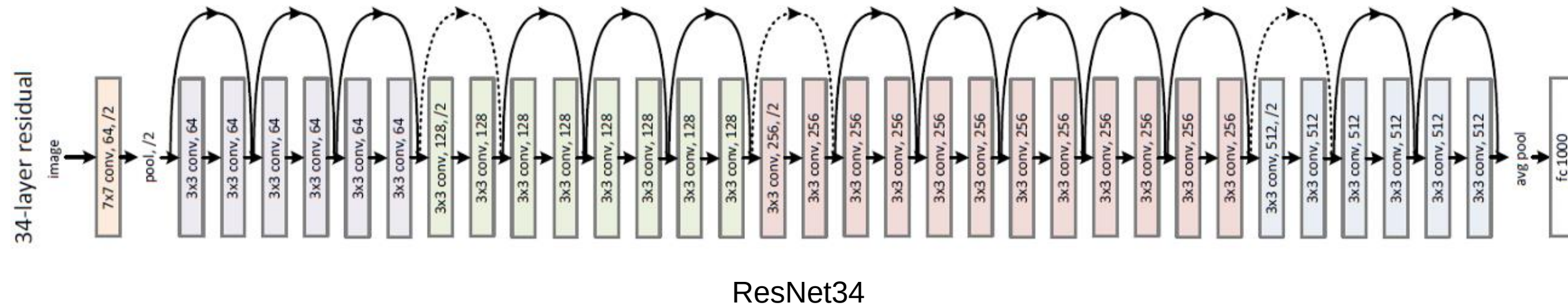
F : The residual function

h : Identity mapping

f : Activation function

- The learnt feature with x input is denoted as $H(x)$, but it is obtained by learning the residual $F(x) = H(x) - x$.
- When the residual is 0, the stack is only identically mapped, the network performance will not degrade.
- In fact, the residual will not be 0, which will also cause the accumulation layer to learn new features based on the input features.

ResNet

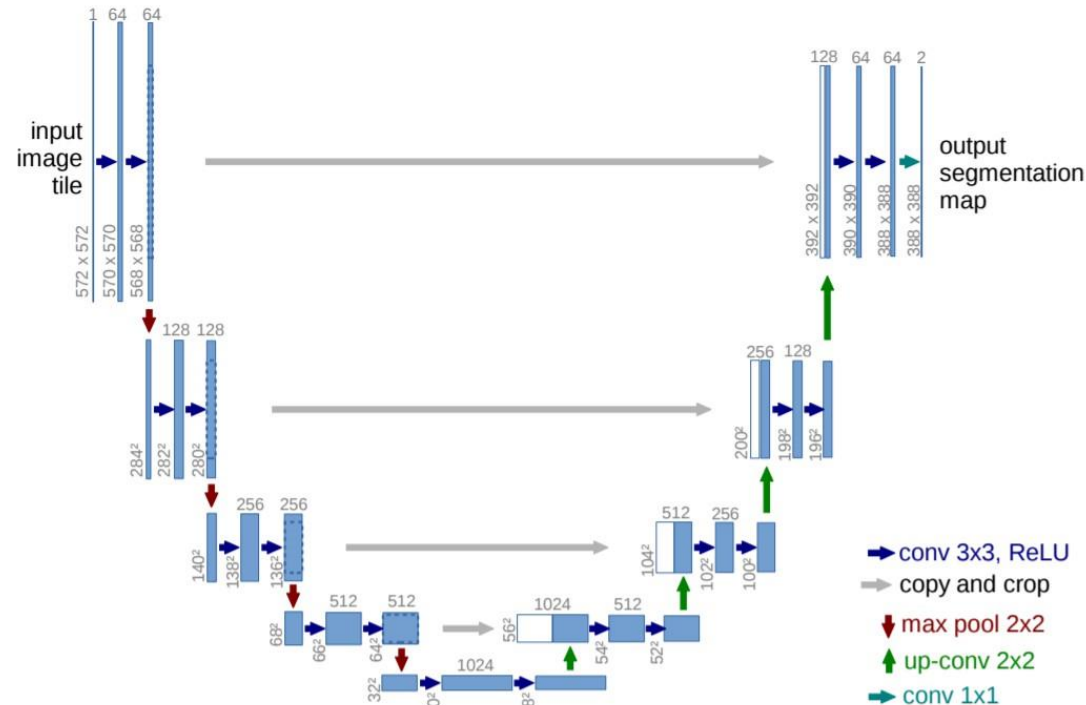


Zhang Y, et al. 2022

- Deepen the number of network layers
- Resolve gradient disappearance
- Improve model accuracy

UNet

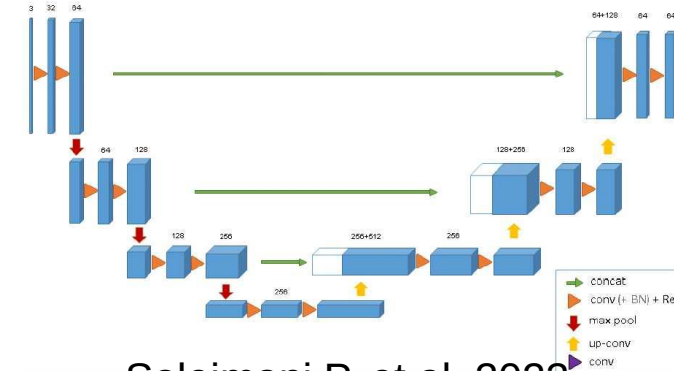
- Improved structure of FCN: U-Net



UNet-2D

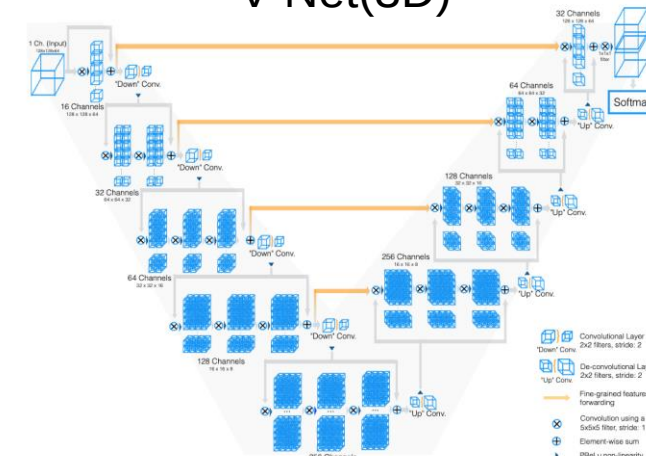
Key: skip Connection

3D-UNet



Soleimani P, et al. 2023

V-Net(3D)

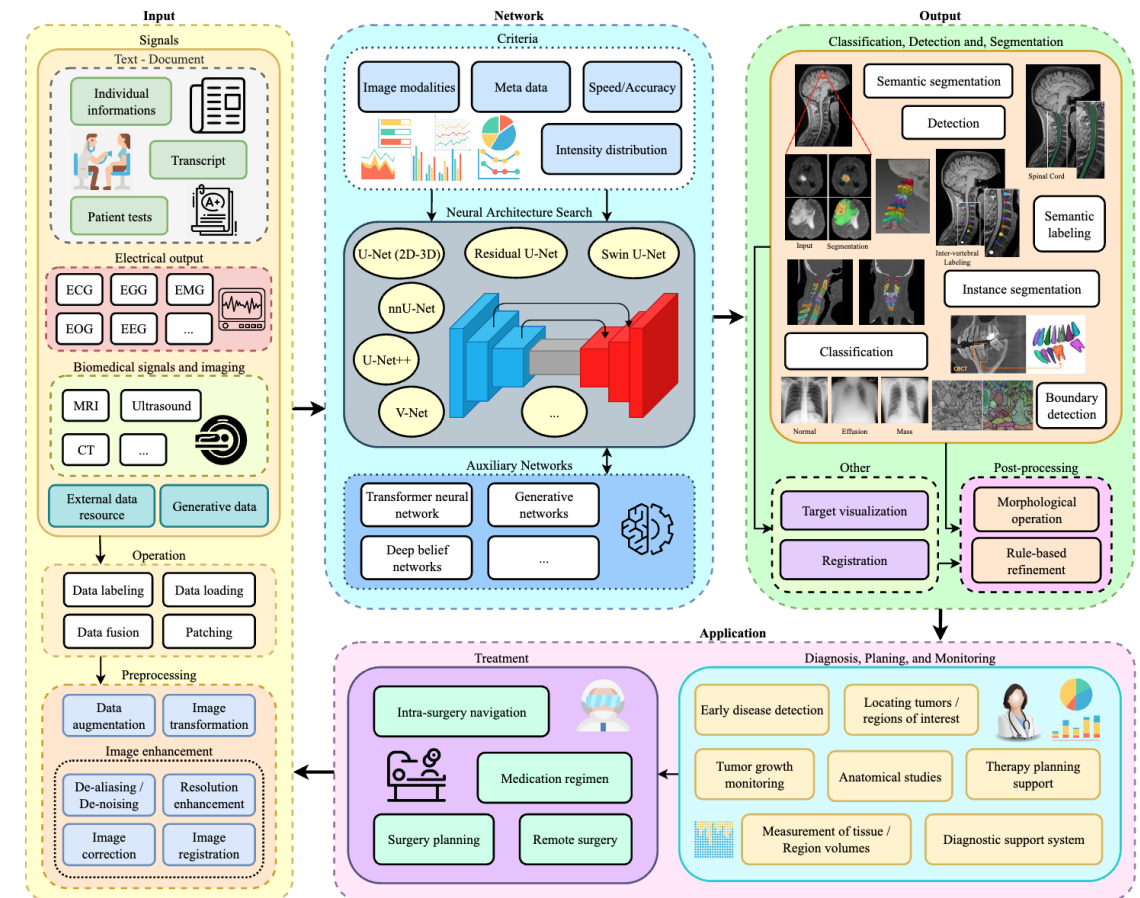


Milletari F, et al. 2016

- 3D convolution
- Transposed convolution instead of upsampling
- Dice coefficient as loss function

The Success of UNet

- The most widely used model in the field of medical image



Improved variants of UNet

All pipeline In UNet

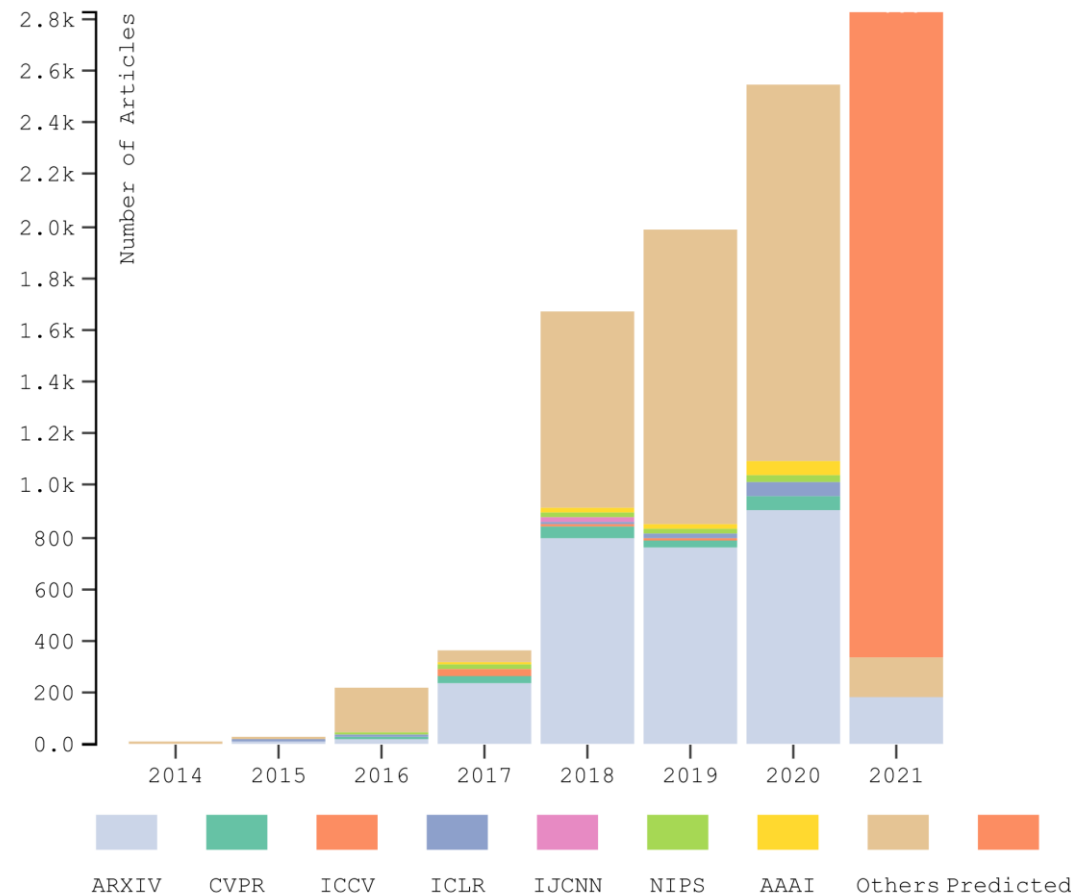
Azad R, et al. 2022



Attention

Attention

- Before deep learning since 1990s...Skipped
- With deep learning



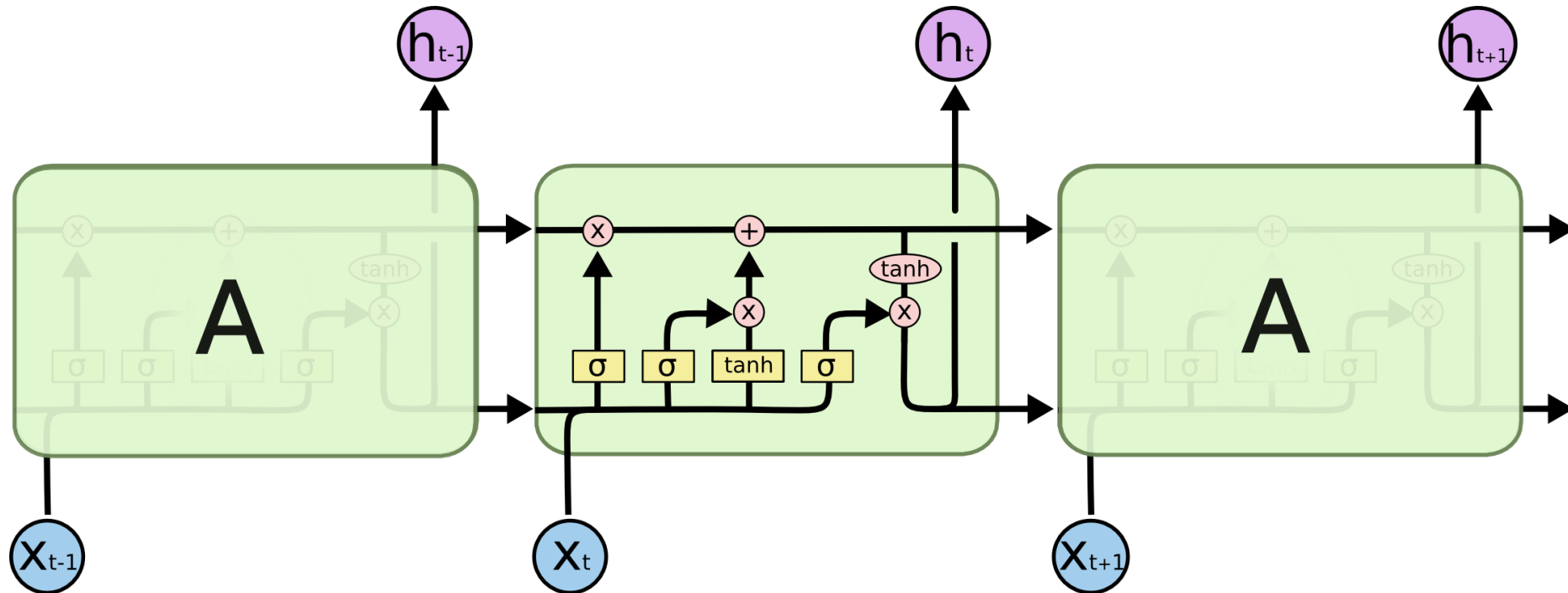
Attention

- Human visual system
 - Focus on essential feature
 - <https://youtu.be/cWwHI0CLalw>
- Utilize the **most relevant** parts of the input in a flexible manner
- Relevant → highest **weights (attention)**



Attention: hard attention

- LSTM (Long Short-Term Memory)



Soft Attention: Channel Attention

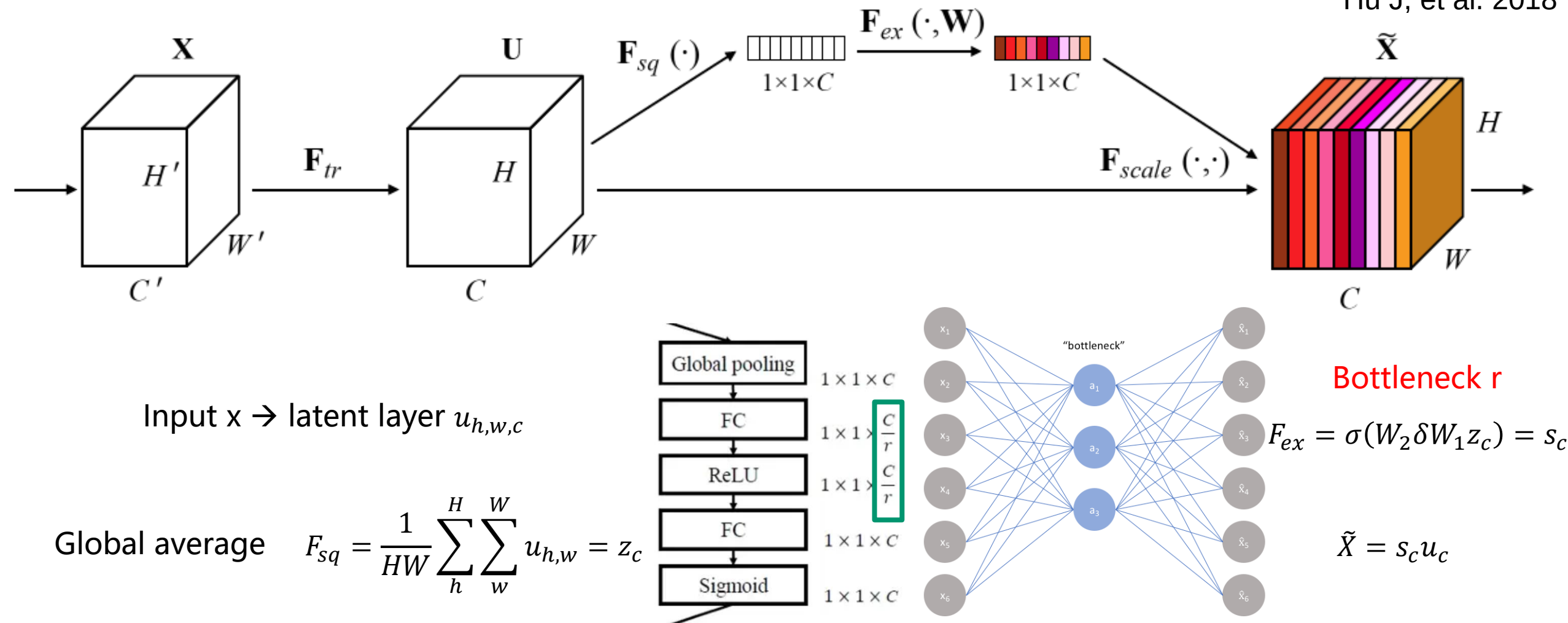
- Squeeze-Excitation (SE) Block
- Squeeze
 - global average pooling $\rightarrow$ each channel to a single value
- Excitation
 - convolutional block feature maps are weighted

$$f = \sigma(FC(ReLU(FC(f_g)))) \text{ average global pooling}$$

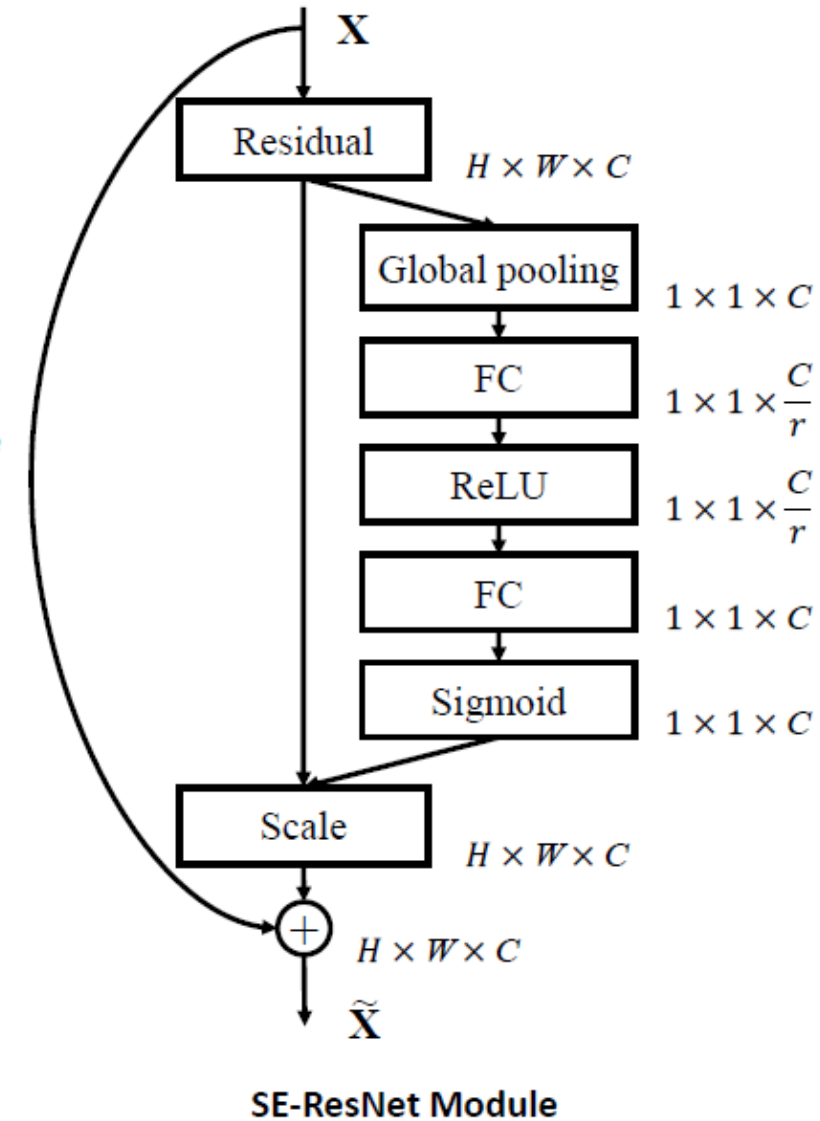
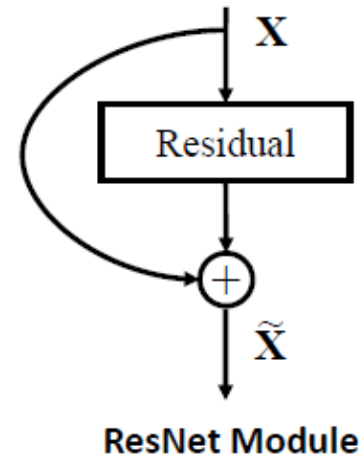
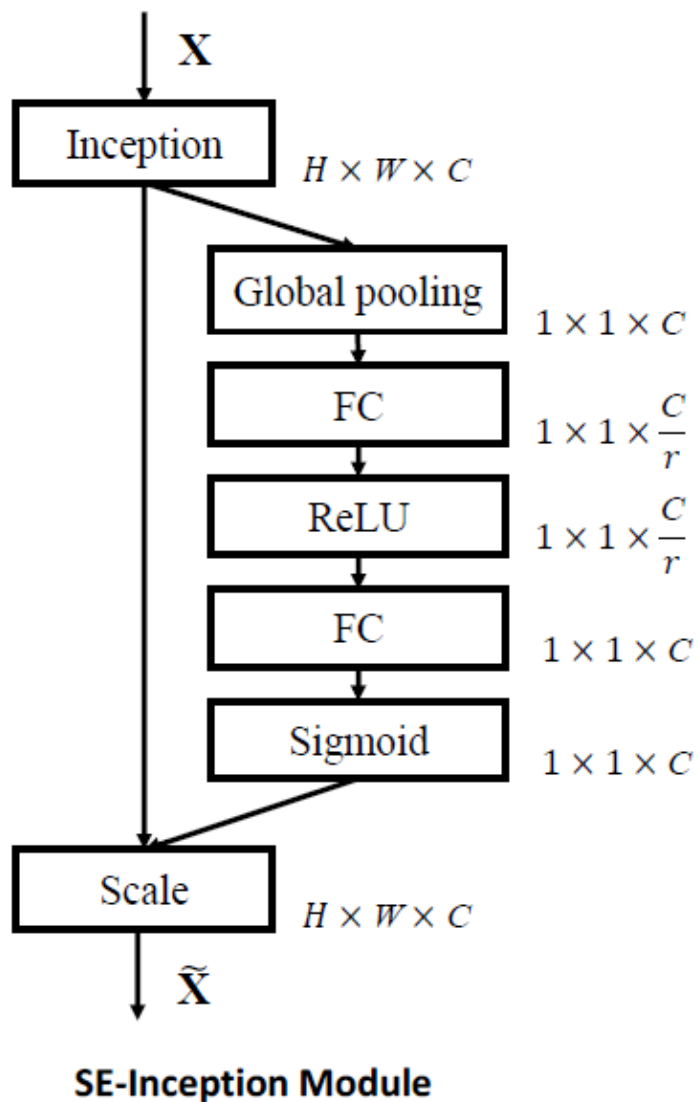
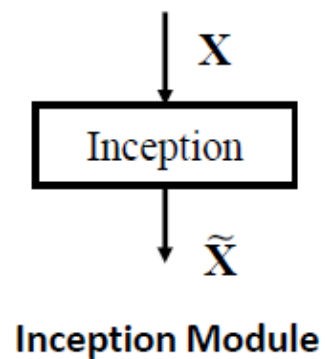


Soft Attention: Channel Attention

Hu J, et al. 2018

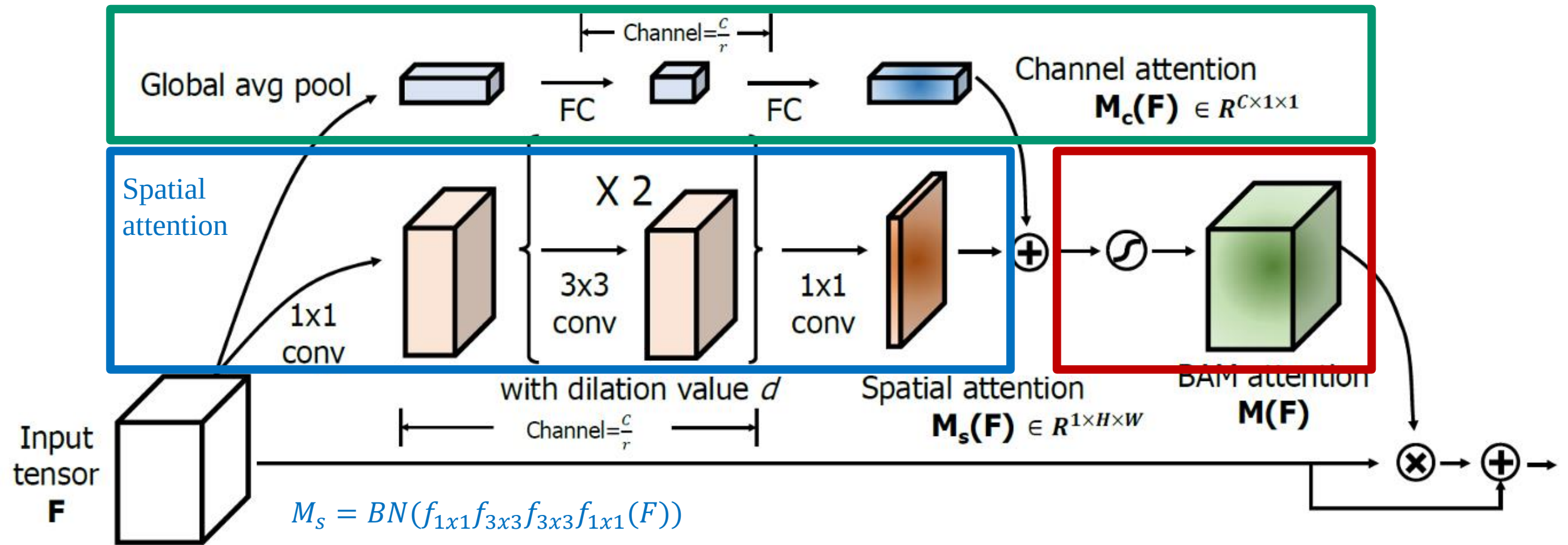


Soft Attention: Channel Attention



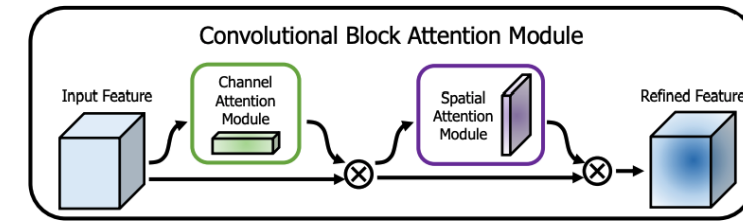
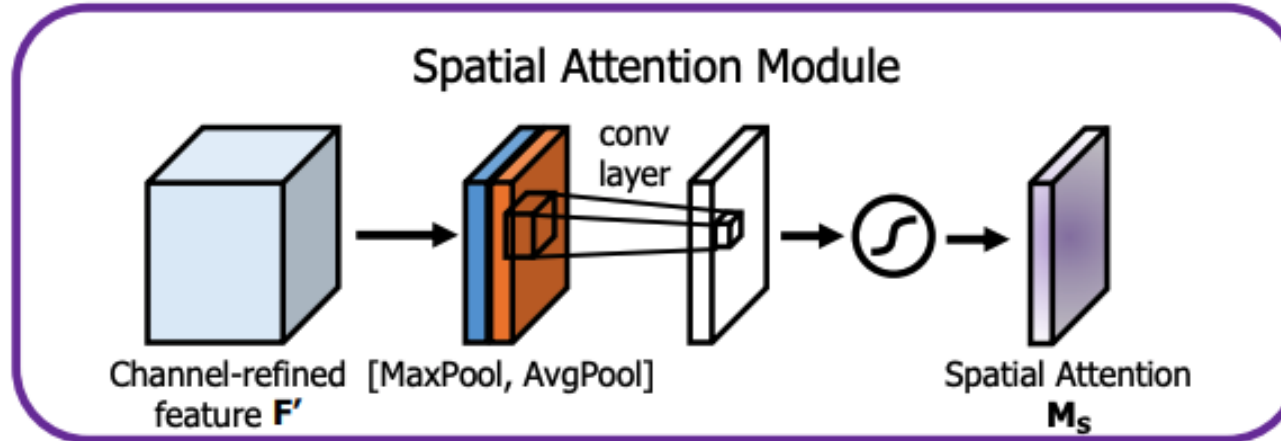
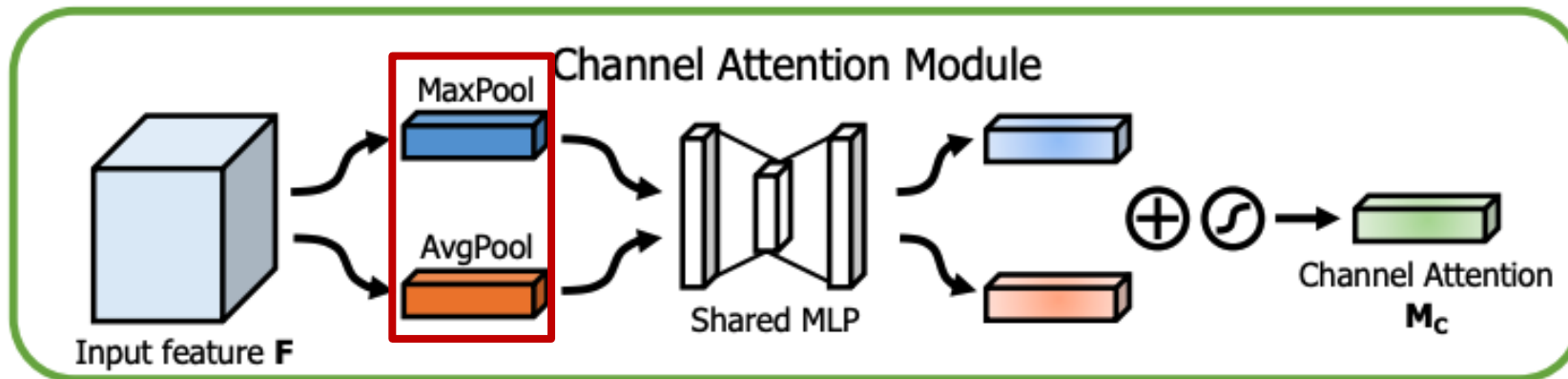
Soft Attention: Spatial Attention

- BAM: Bottleneck Attention Module



Soft Attention: C&S Attention

- Convolutional Block Attention Module (CBAM)

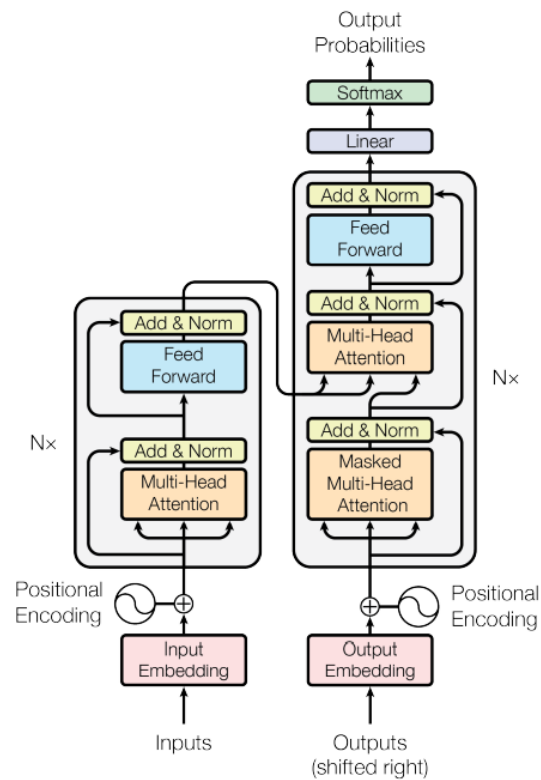


$$M_c = \sigma(MLP(Ave(F)) + MLP(Max(F)))$$

$$M_s = \sigma(f_{7 \times 7} \left(\begin{matrix} Ave(F) \\ +Max(F) \end{matrix} \right))$$

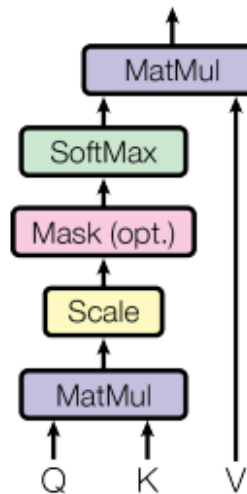
Transformer

- Attention is all you need (2017)

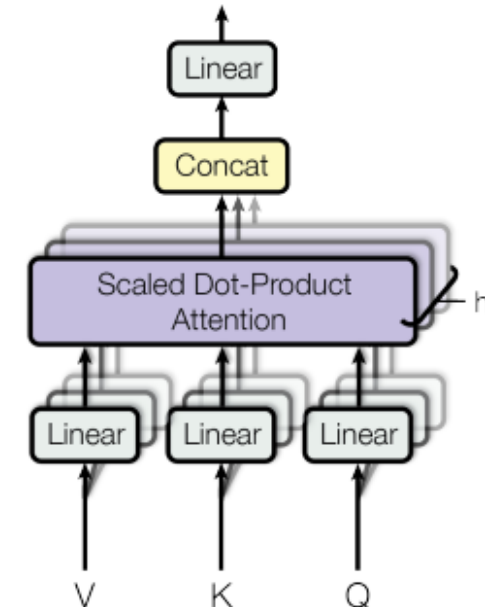


Transformer – Model architecture

Scaled Dot-Product Attention



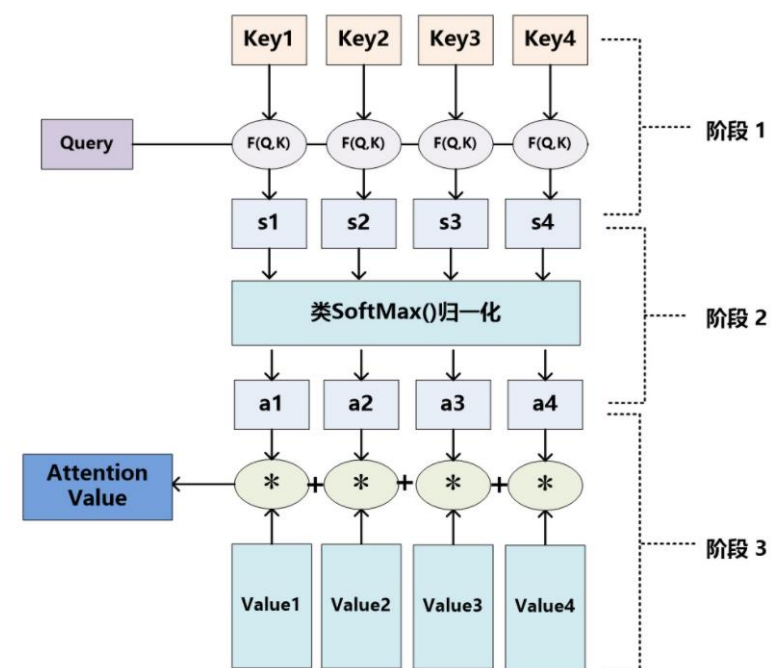
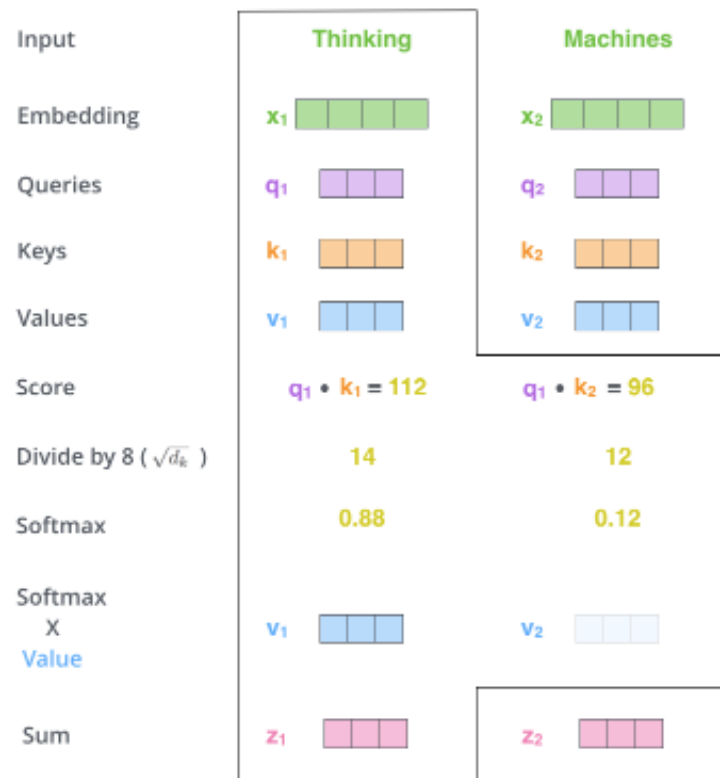
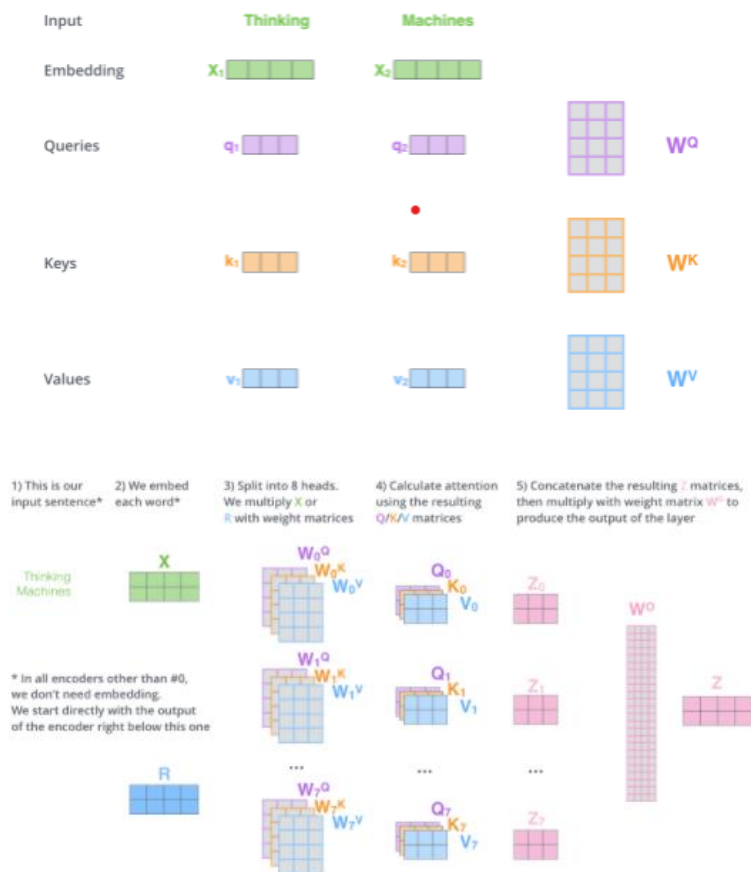
Multi-Head Attention



$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

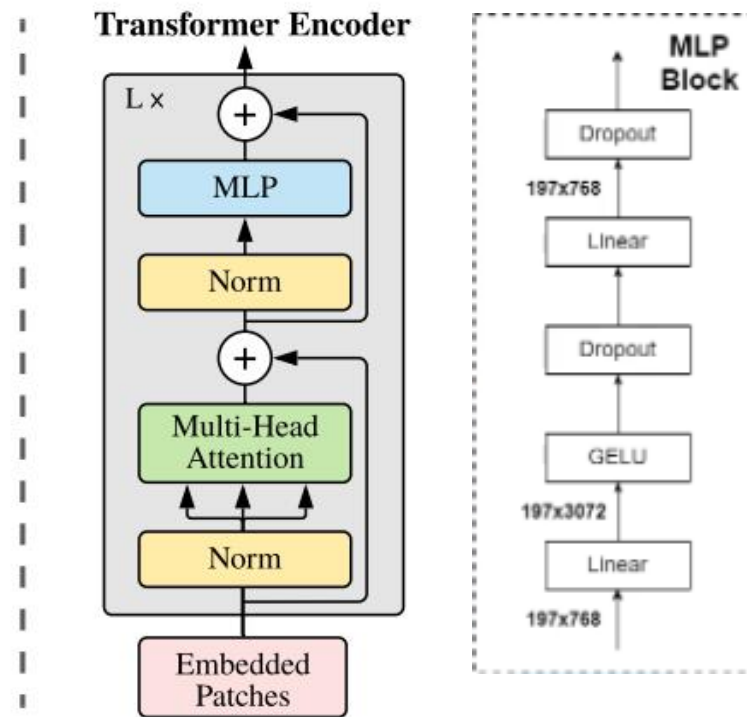
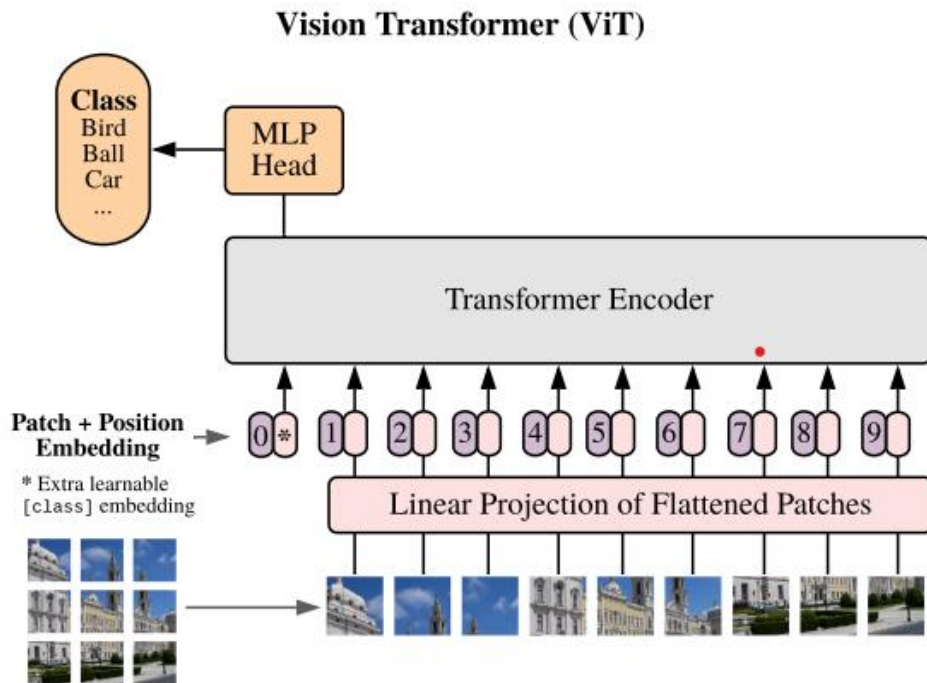
Transformer

• Self-Attention: Multi-Head



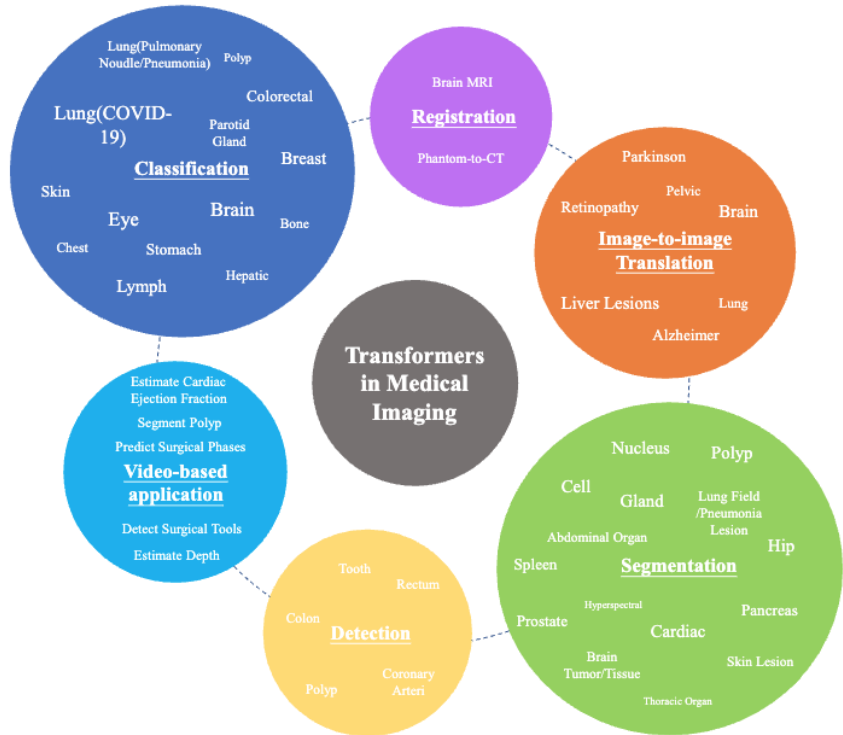
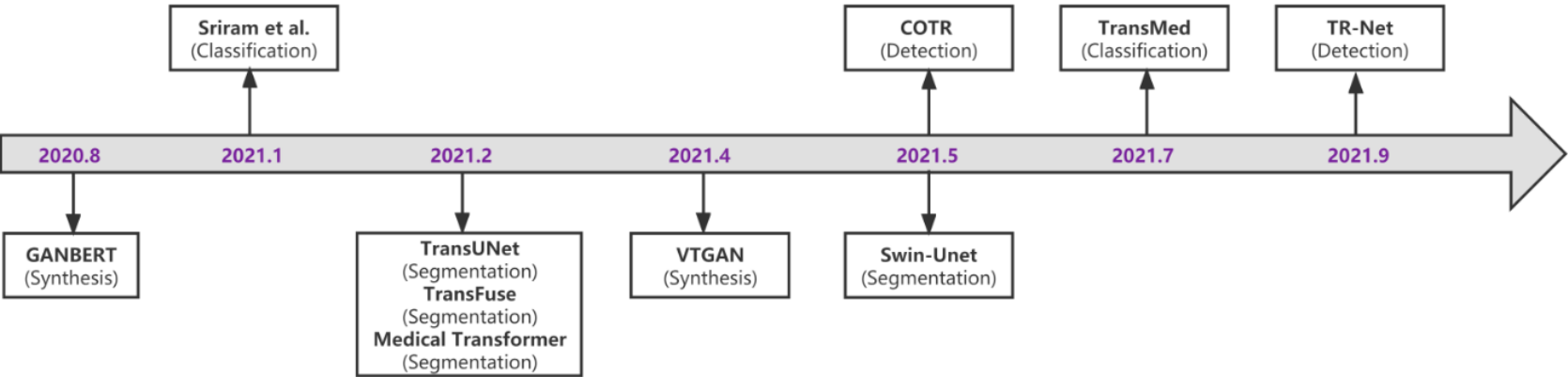
Transformer

- ViT: Vision transformer



- Embedding layer:**
 $[224, 224, 3] \rightarrow [14, 14, 768] \rightarrow [196, 768]$, plus an analogy token to get $[197, 768]$, plus position encoding: $[197, 768] \rightarrow [768]$
- Transformer layer:**
 The basic structure is similar to the original transformer, but the multi-head attention first follows the Norm. There's a Dropout layer.

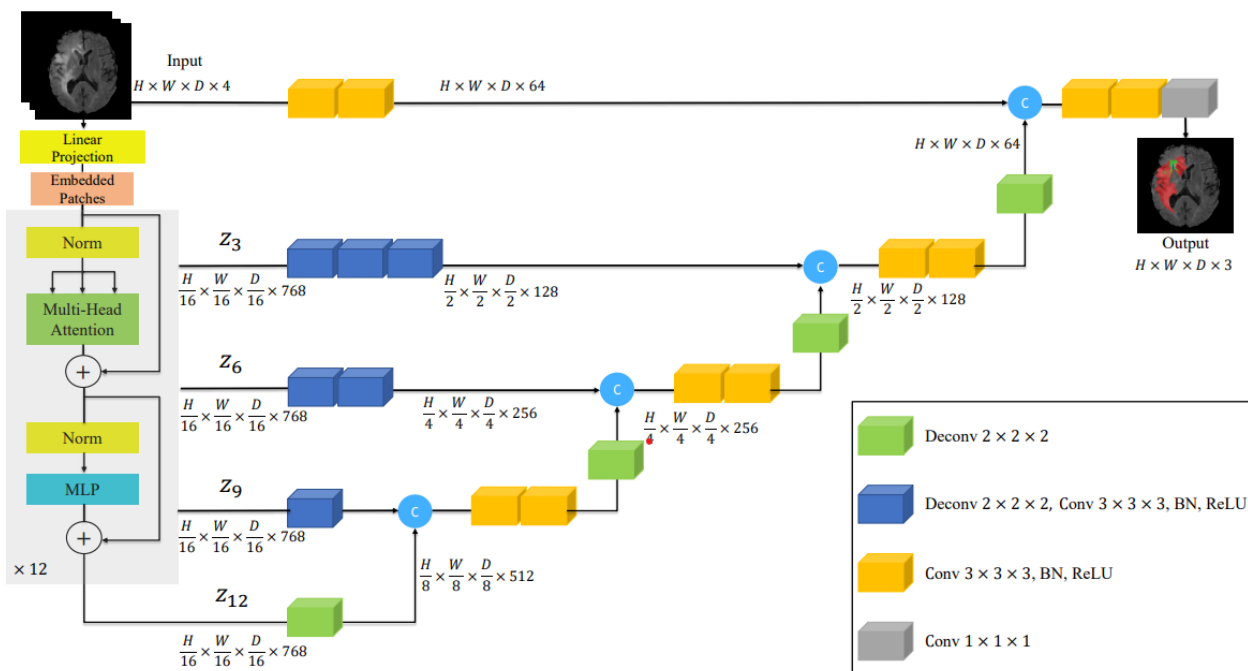
Transformer In Medical Image



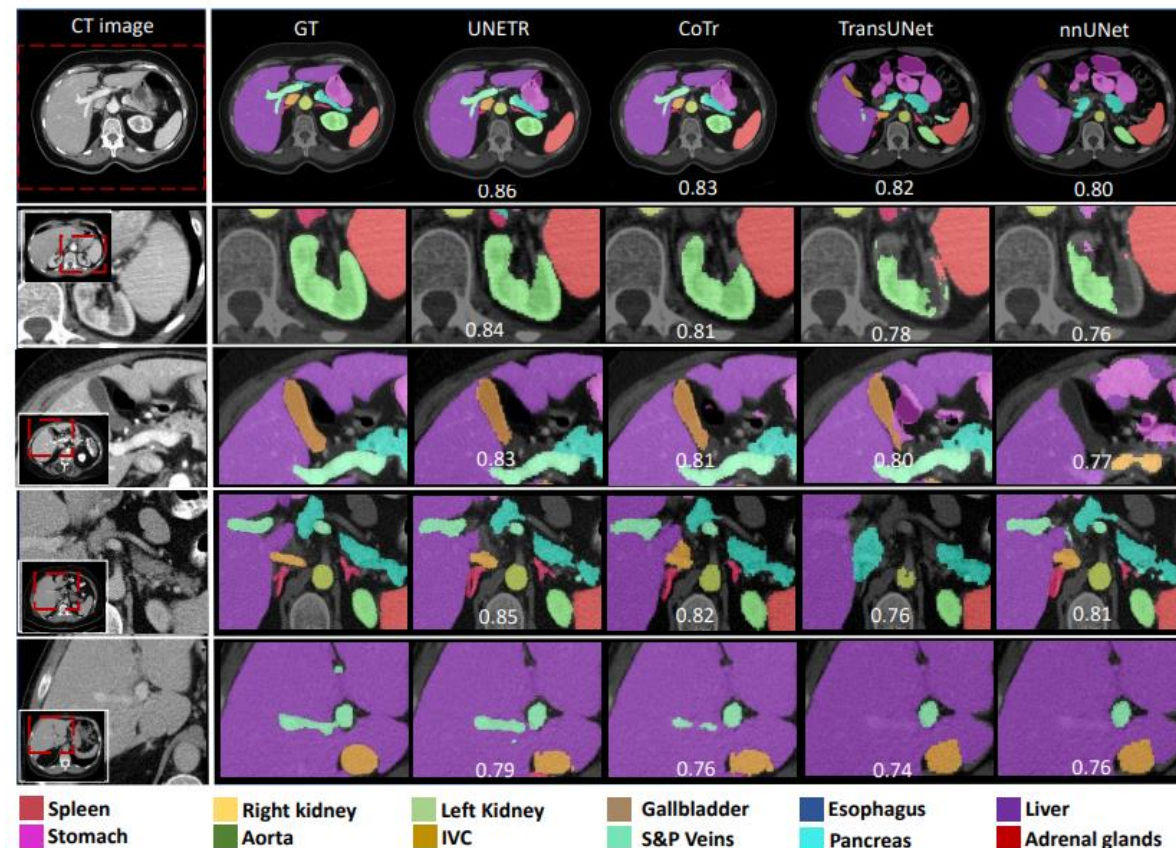
He K, et al. 2023

Transformer In Medical Image

- Medical image segmentation

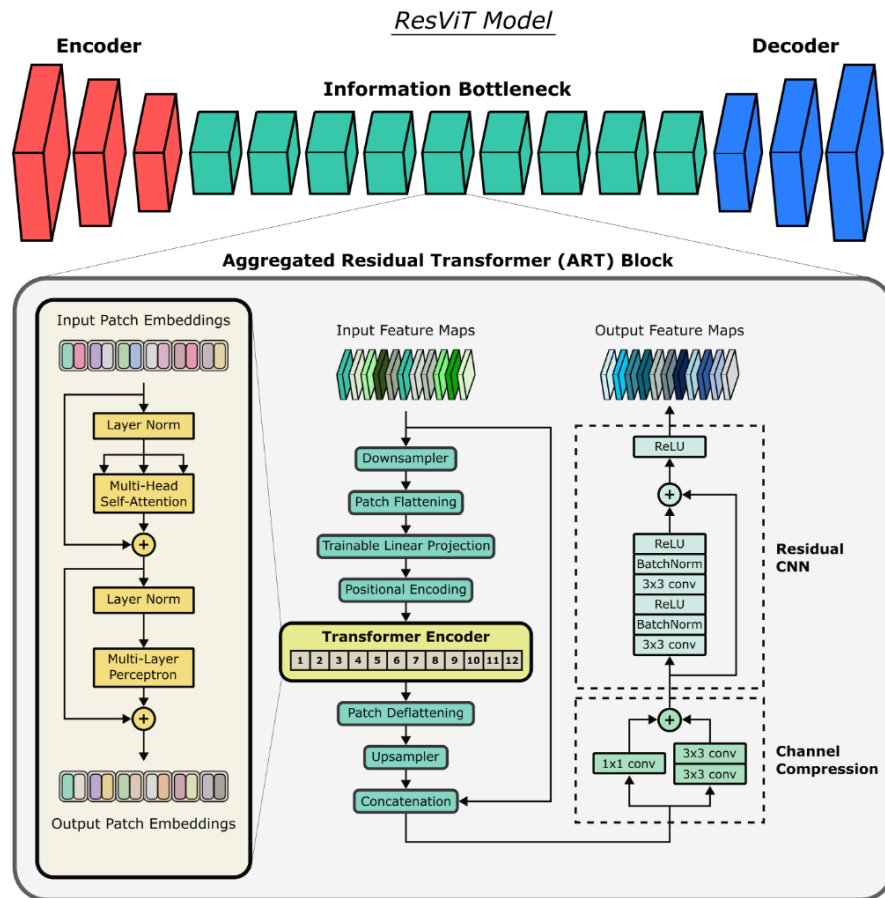


UNETR

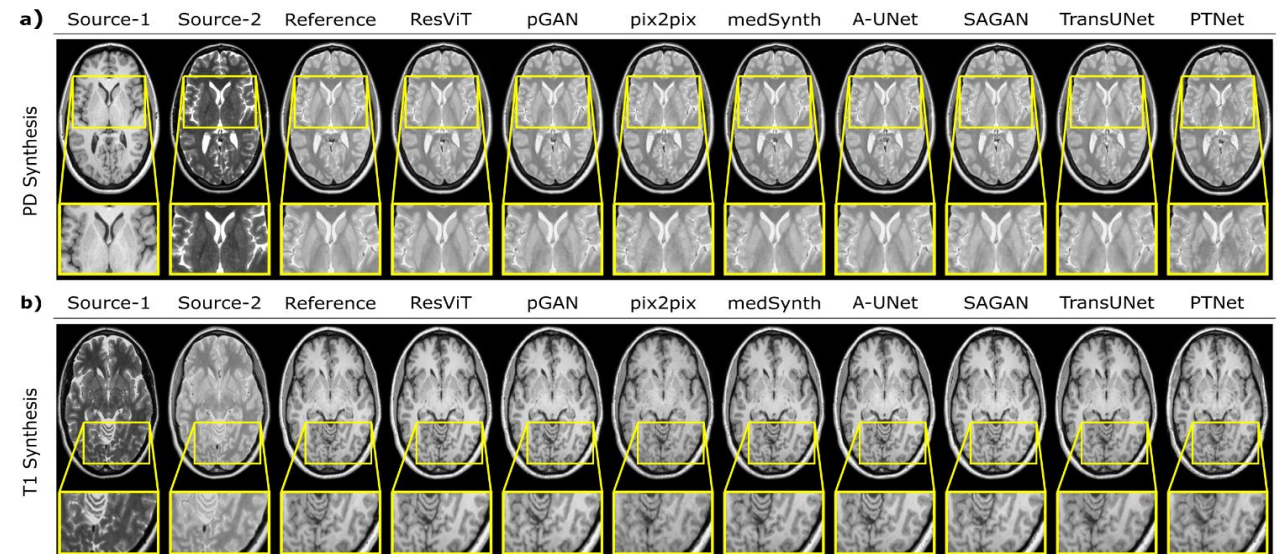
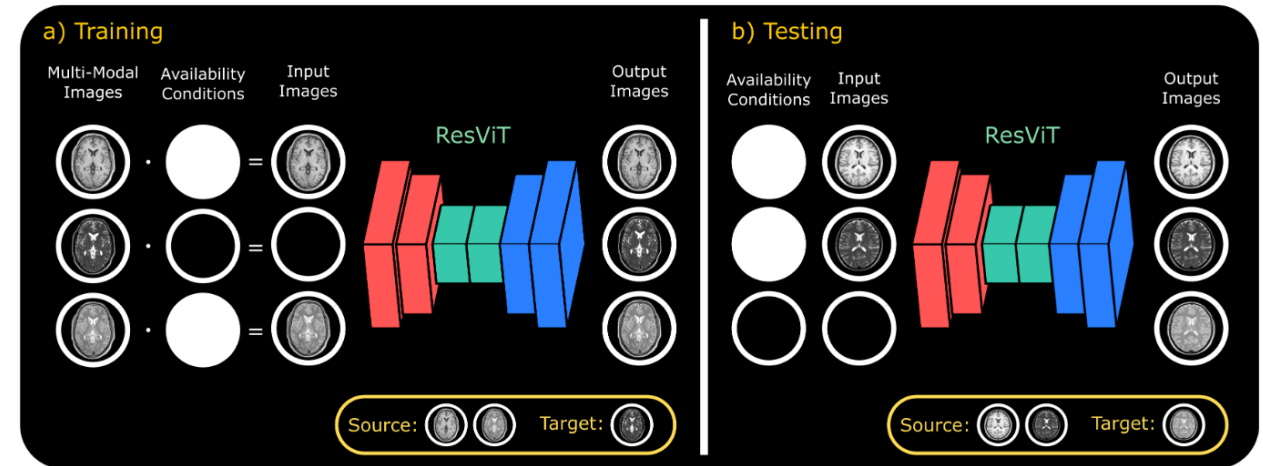


Transformer In Medical Image

- Medical image synthesis



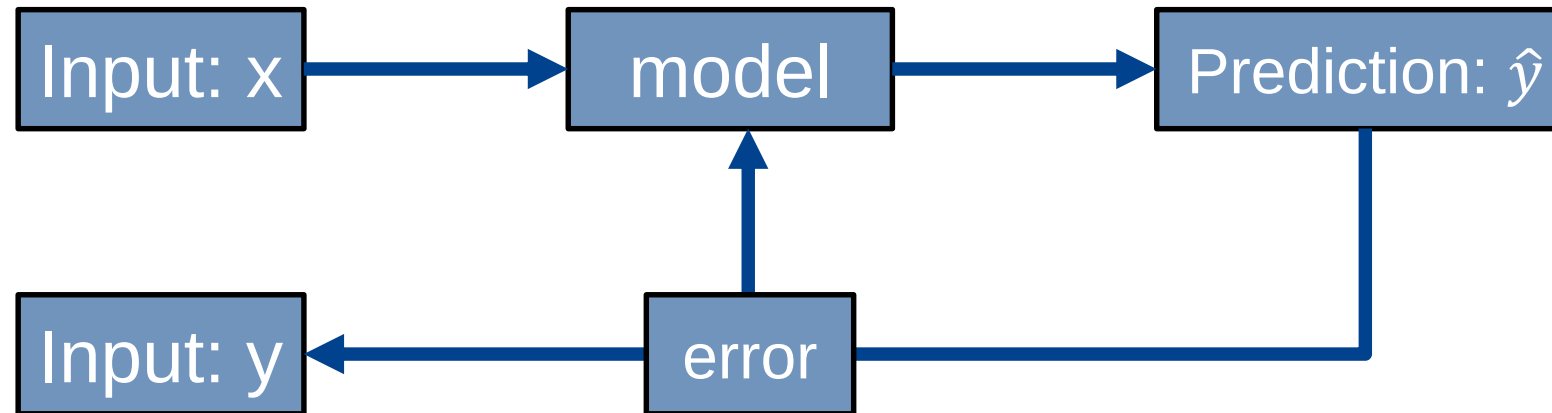
ResViT





Generative Models

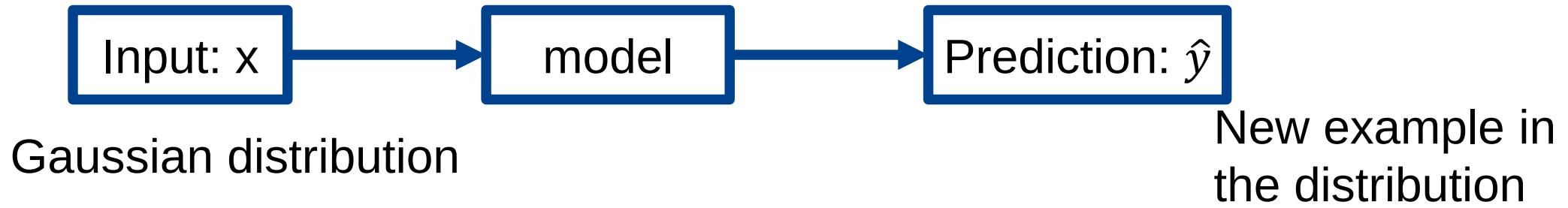
General model



General model

unsupervised

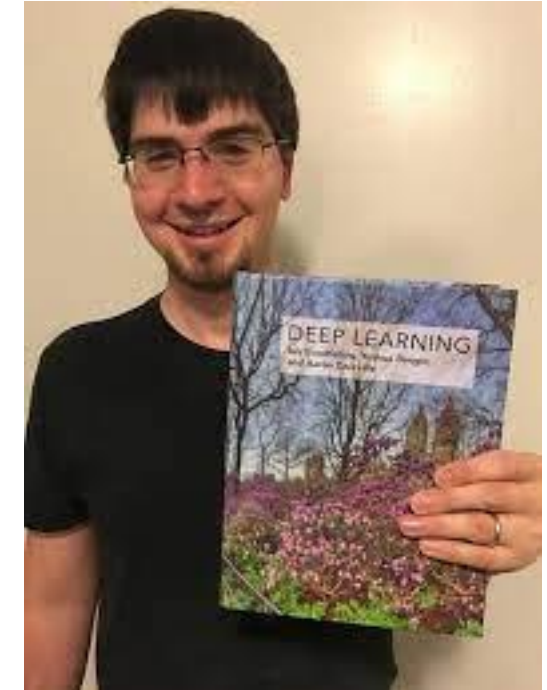
generative



A good model generates indistinguishable example

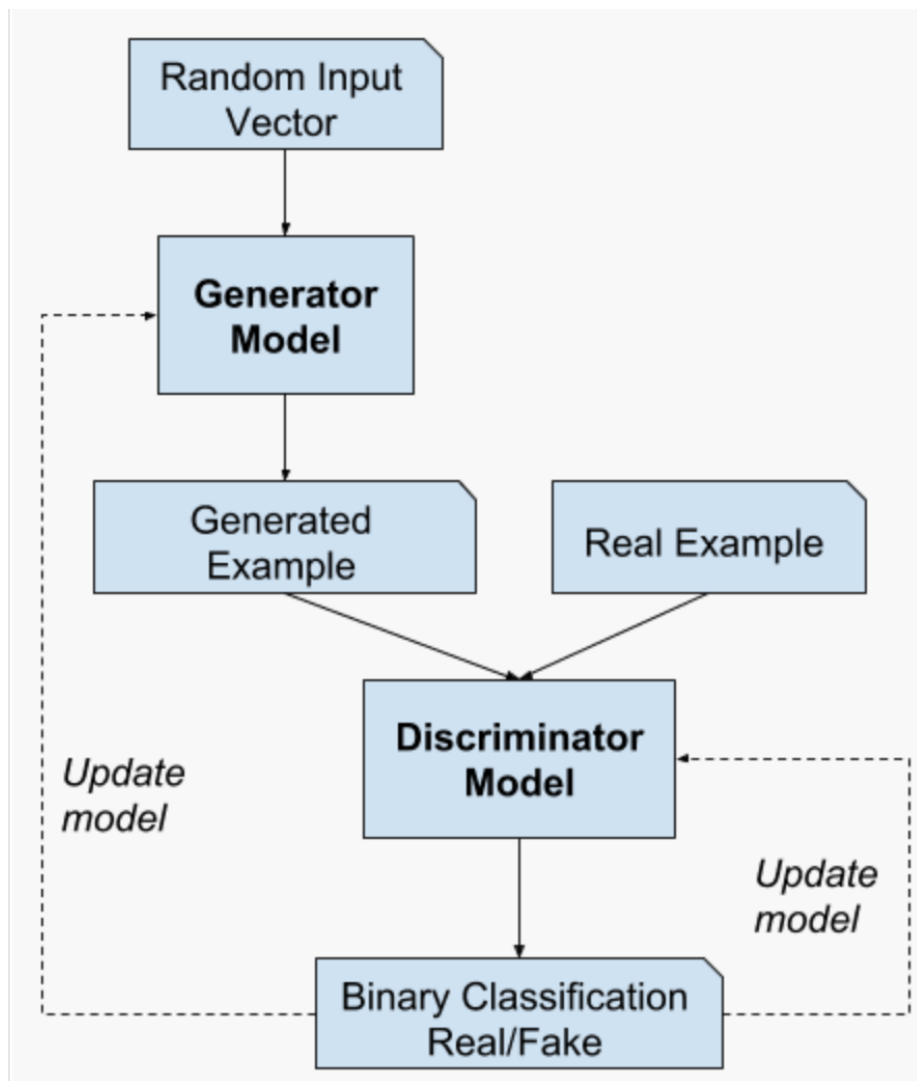
Generative Adversarial Networks

- 2014, citation: 50K
- Two submodels
 - Generator: generate new plausible examples from the problem domain.
 - Discriminator: classify examples as real (from the domain) or fake (generated)

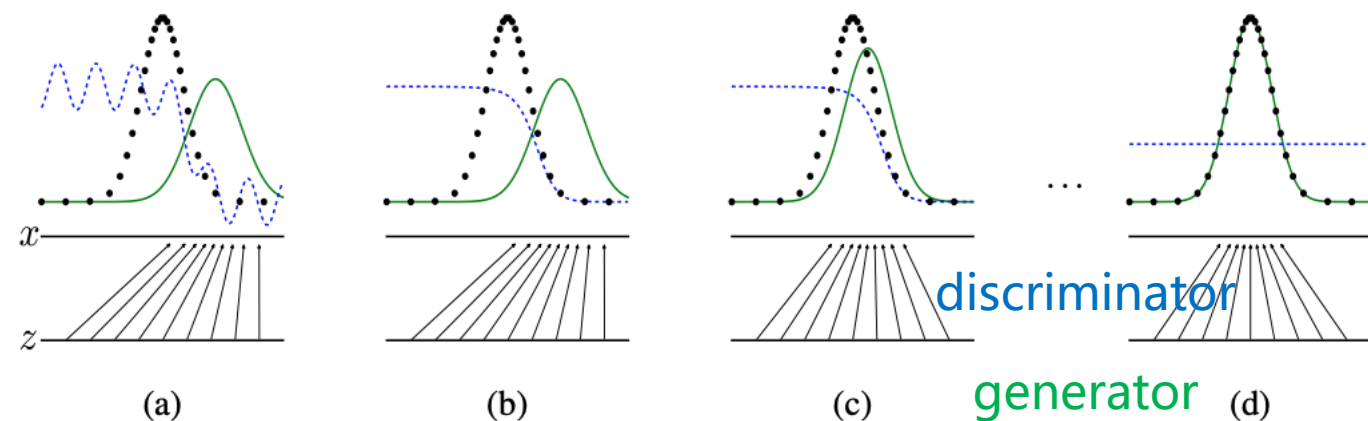


Ian Goodfellow

GAN



Bandeira J, et al.



Creswell A, et al. 2018

Step1:
Update D

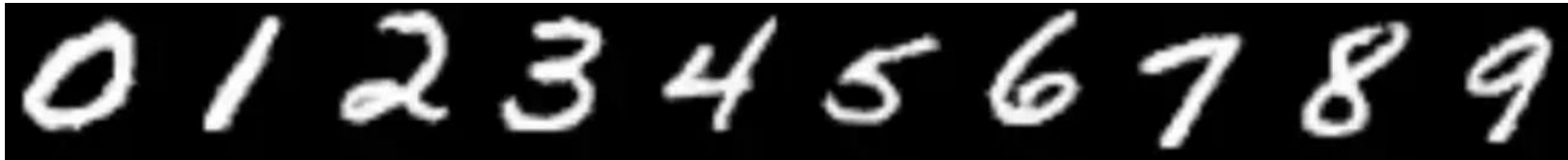
$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right]$$

Step2:
Update G

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)})))$$

InfoGAN

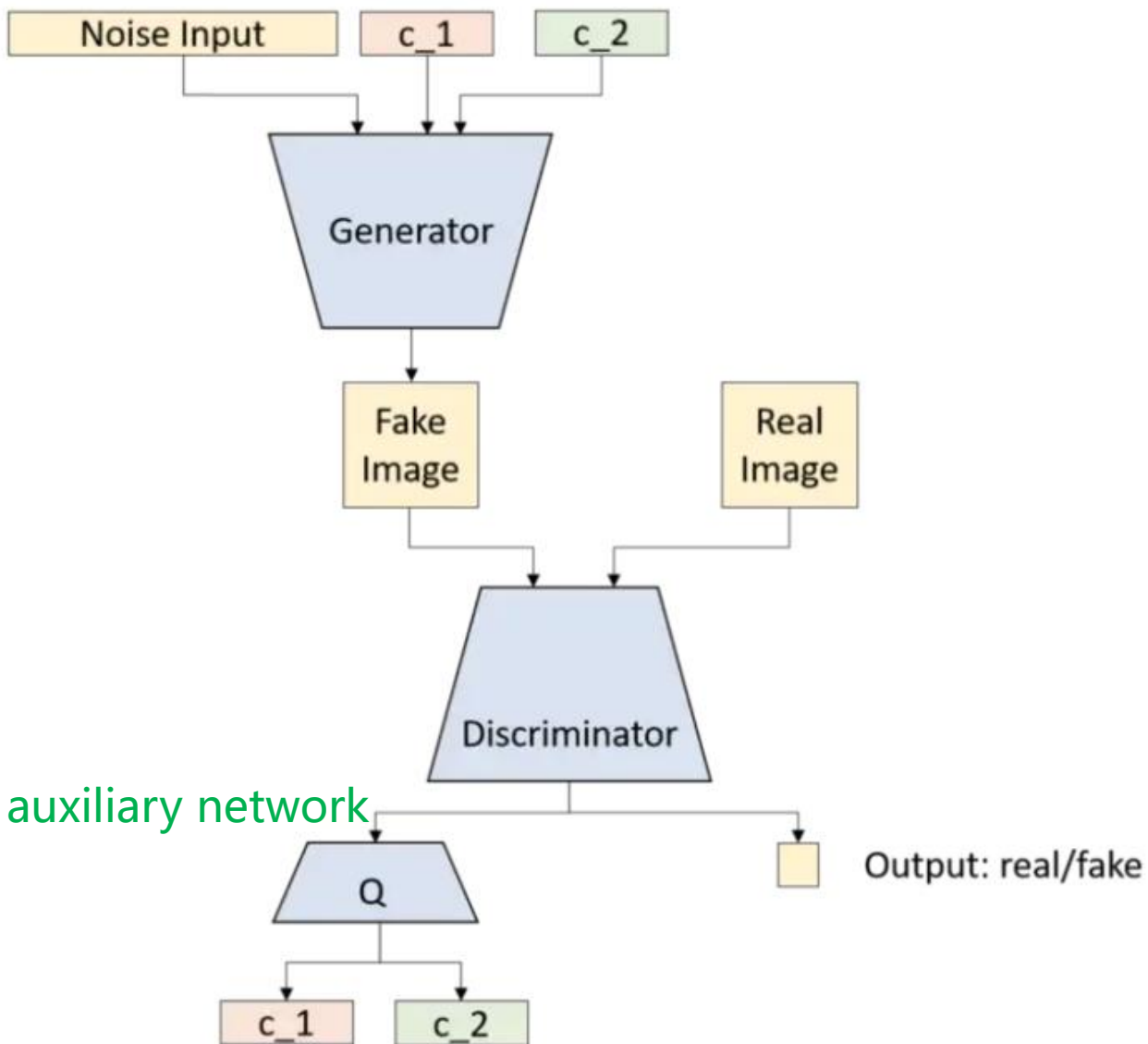
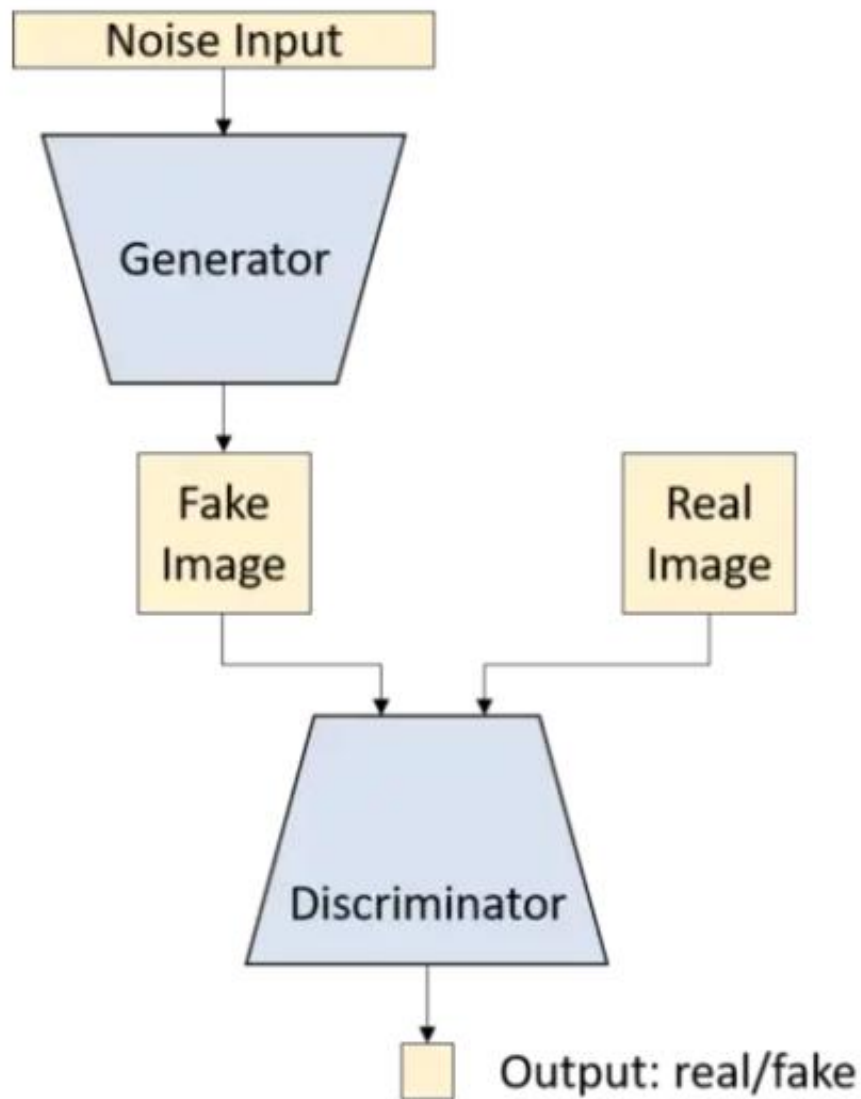
- GAN: no control over the images GANs produce



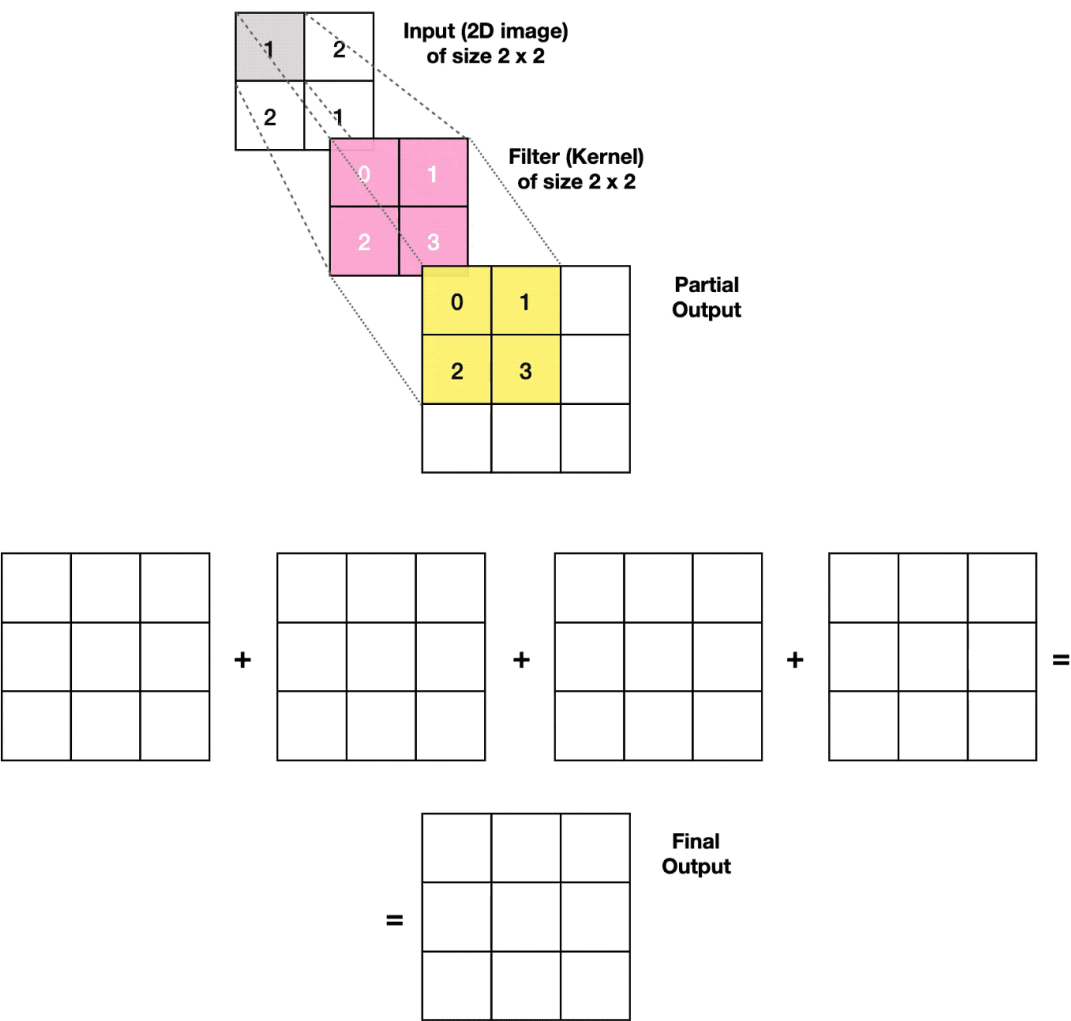
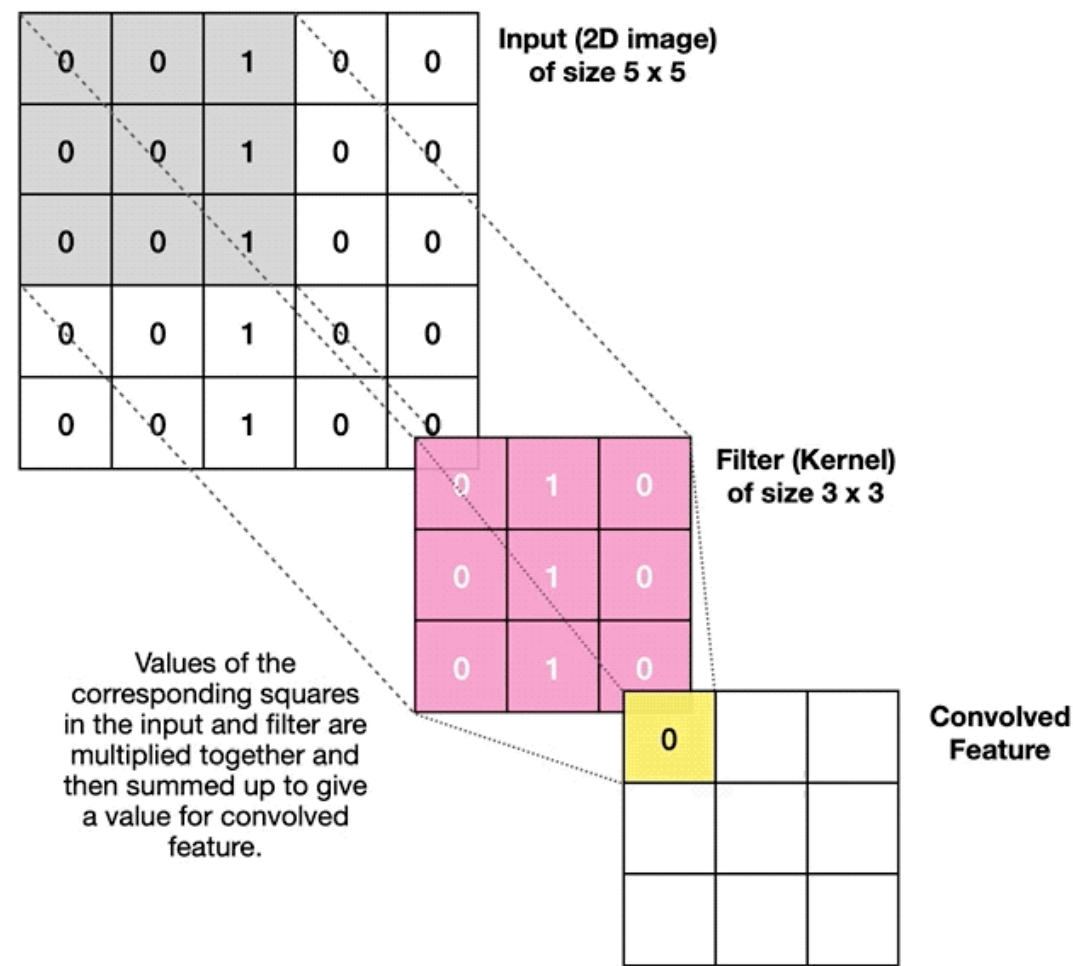
Can Not control which number GAN will generate

- InfoGAN: specific categorical features

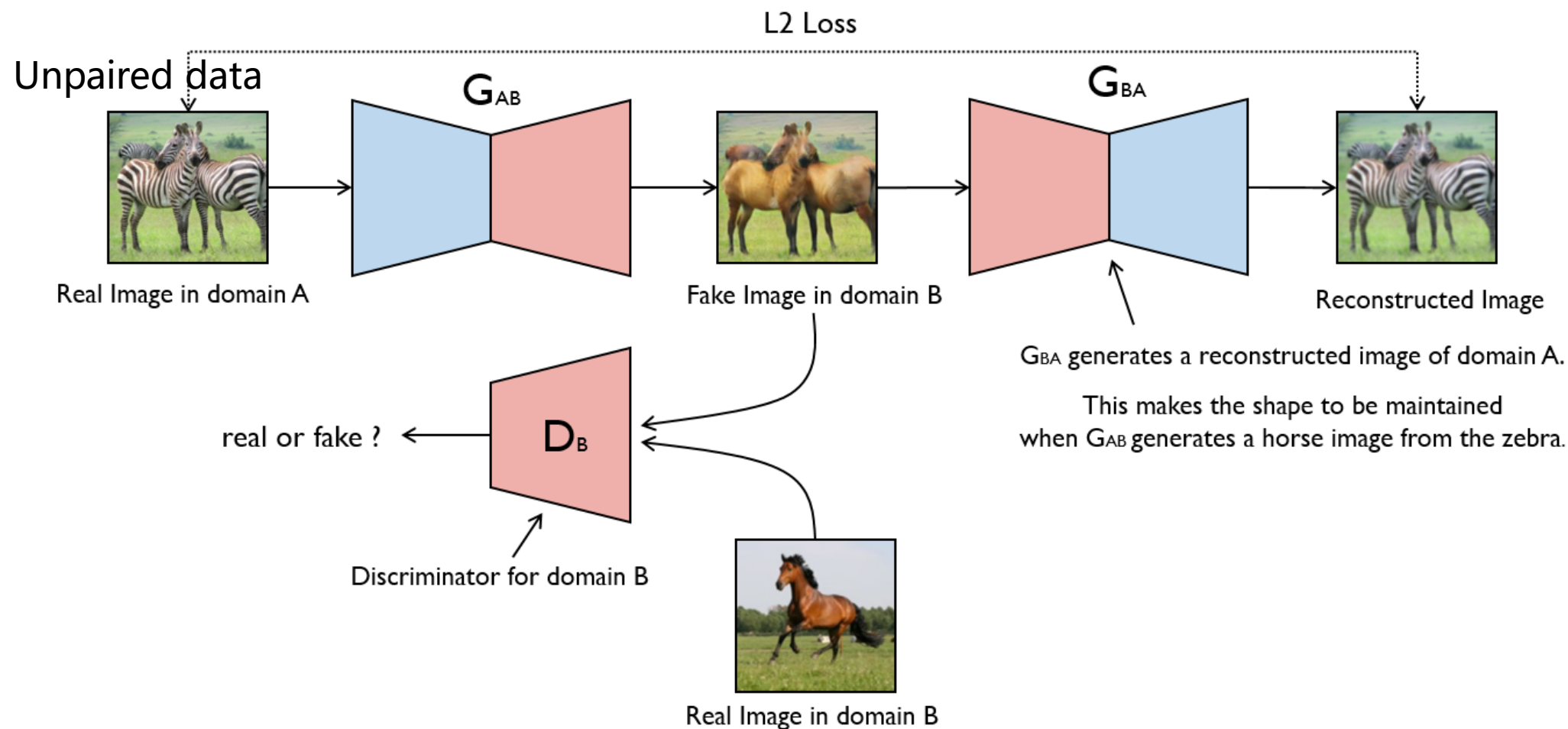
InfoGAN



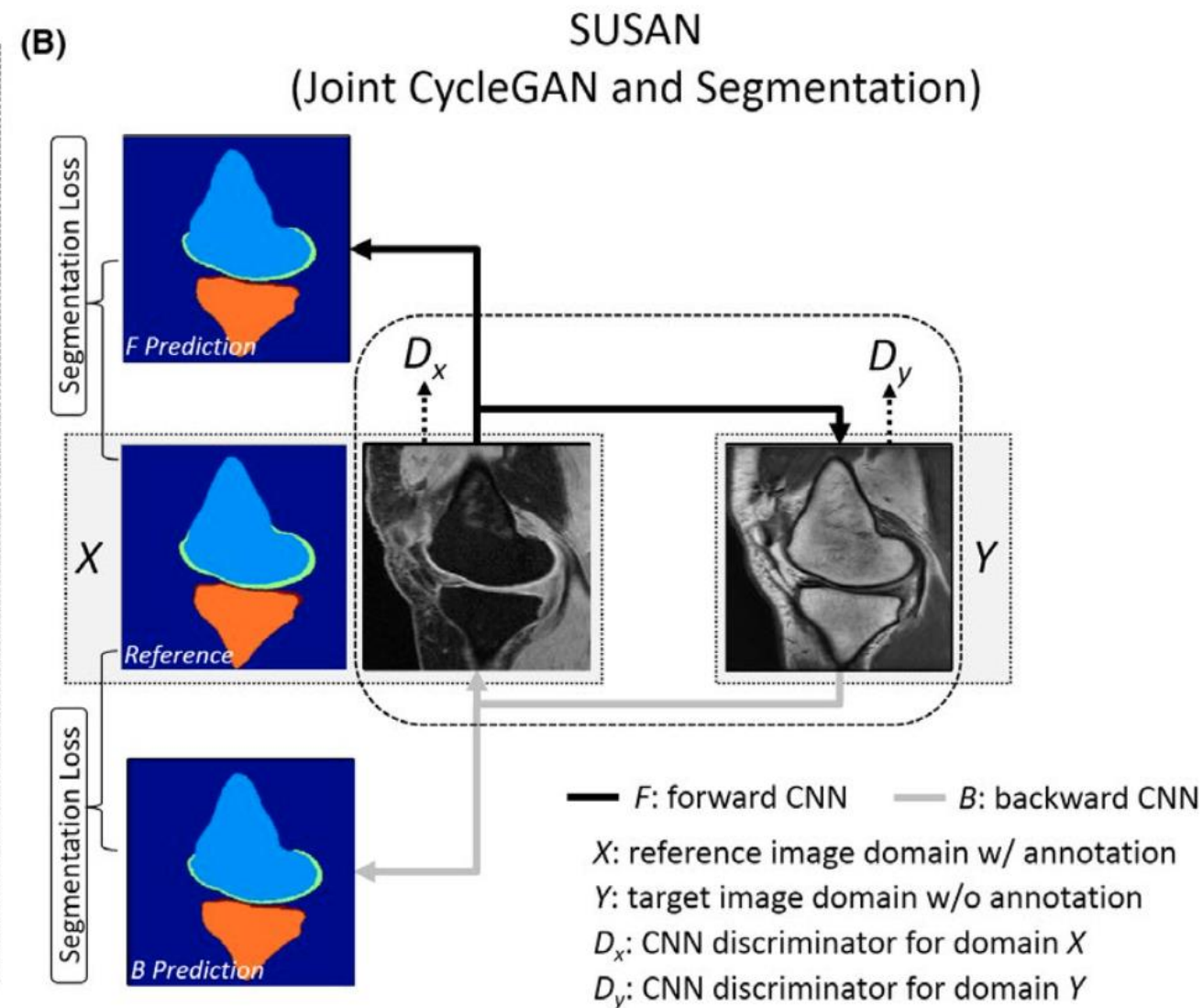
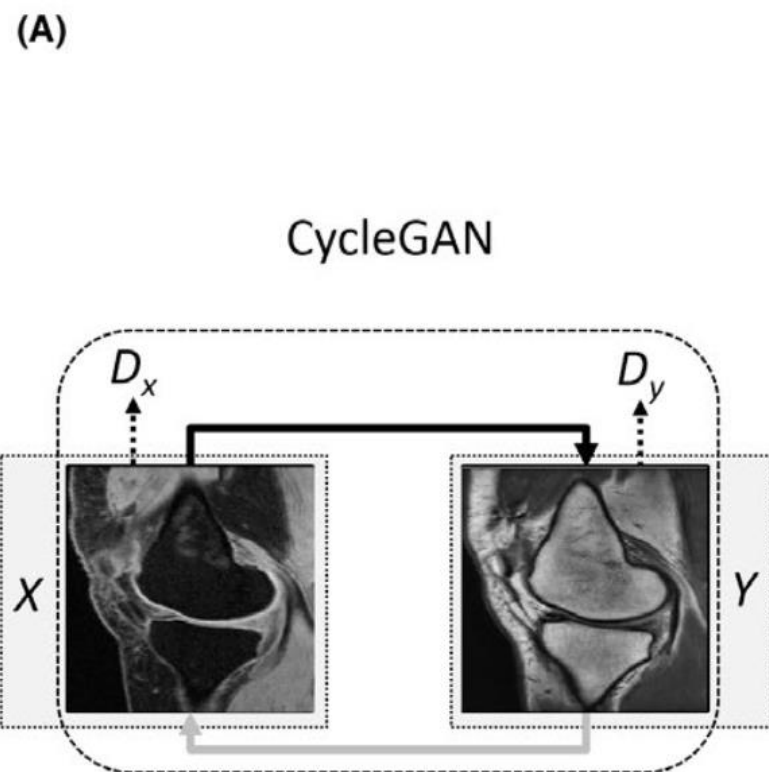
Deep Convolutional GAN (DCGAN)



Cycle GAN

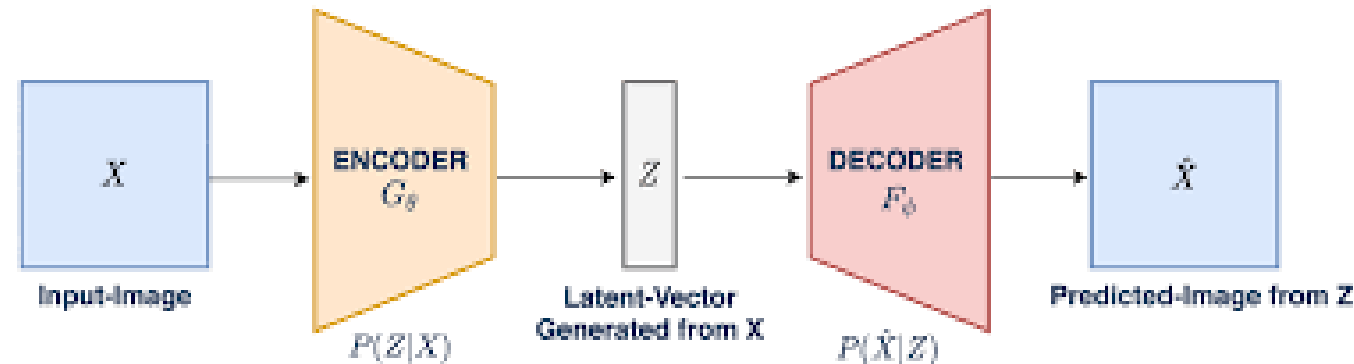


Example



Diffusion model

- GAN
 - training instability and generating diverse image problems
- VAE
 - surrogate loss

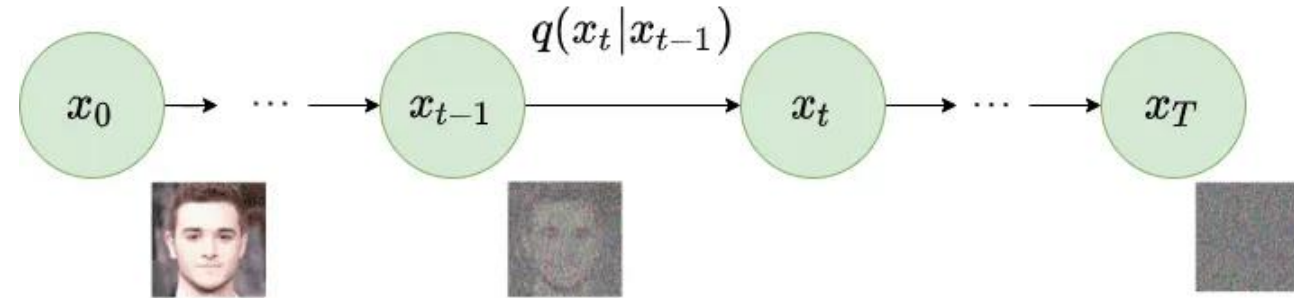


Diffusion model

- Forward diffusion process

- Markov chain
- Slowly and randomly add noise

- Initial image x_0 with the distribution $q(x)$
- Add Gaussian noise with variance β_t
- Producing a latent variable with distribution $q(x_t|x_{t-1})$



$$q(x_t|x_{t-1}) = N(x_t; \underbrace{\sqrt{1 - \beta_t}x_{t-1}}_{\text{mean}}, \beta_t)$$

$$q(x_T|x_0) = \prod_t^T q(x_t|x_{t-1})$$

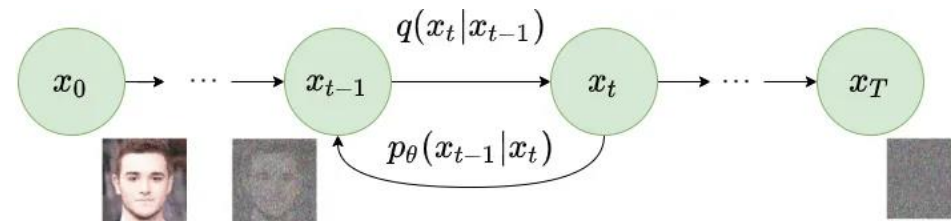
Stepwise $\rightarrow$ reparameterization trick $q(x_t|x_0)$

x_T is isotropic Gaussian distribution

Diffusion model

- Reverse diffusion process
 - Estimating $q(x_{t-1}|x_t)$ require the whole dataset $\rightarrow$ difficult
 - Neural network to parameterize the θ for a small β_t

$$p_{\theta}(x_{t-1}|x_t) = N(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$$



Train θ to predict the mean and variance in each step

Diffusion model

- Training
 - Find the reverse Markov transition
 - Maximize the likelihood of the training data
 - Minimize the variational upper bound of the negative log likelihood
 - $E(-\log p_{\theta}(x_0)) \leq E_q \left(-\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} \right) := L$
- Kullback-Leibler (KL) Divergence
 - How much one probability distribution differs from another one
 - $D_{KL}(p|q) = \int p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$

Diffusion model

- $L = L_0 + L_1 + \cdots + L_T$
- $L_{t-1} = D_{KL}(q(x_{t-1}|x_t, x_0) || p_{\theta}(x_{t-1}|x_t))$

$$p_{\theta}(x_{t-1}|x_t) = N(x_{t-1}; \mu_{\theta}(x_t, t), \sum_{\theta}(x_t, t))$$

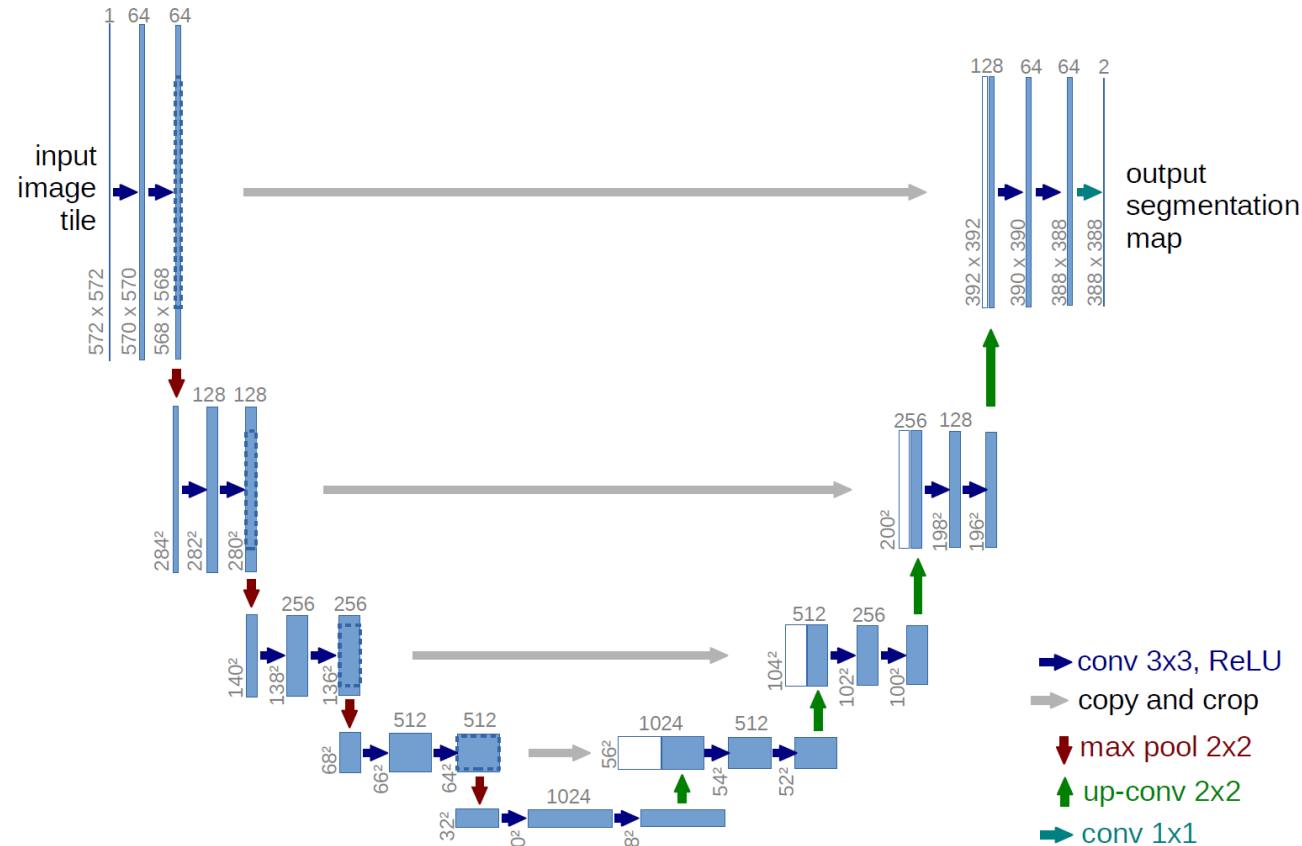
- Use the forward process variance
 - $\Sigma_{\theta}(x_t, t) = \beta_t I = \sigma^2 I$

$$p_{\theta}(x_{t-1}|x_t) = N(x_{t-1}; \mu_{\theta}(x_t, t), \sigma^2 I)$$

$$L_{t-1} \propto |\mu_t(x_t, x_0) - \mu_{\theta}(x_t, t)|_2$$

Diffusion Model

- Input and output have the same dimension
- U-Net like structure



Diffusion Model

- Markov chain: latent variables depend on the previous (or following) timestep
- transition distributions in the Markov chain are Gaussian
- asymptotically distributed as an isotropic Gaussian for sufficiently large T .
- variance schedule was fixed or learned
- Flexible backbone model: u-net or others
- Approximate the likelihood of the training data

Diffusion model

- Sampling

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

Diffusion model

- Score-based Diffusion model

$$\dot{\mathbf{x}} = -\beta(t)\mathbf{x} + g(t)\boldsymbol{\eta}(t)$$



$$x_0 \sim p_0(x), \quad \epsilon \sim N(0, I)$$

Conditional Diffusion

- For a paired data $\{x, y\}$
- We want the model $p(x|y)$
- Condition:
 - We want an university in the Harry Potter style



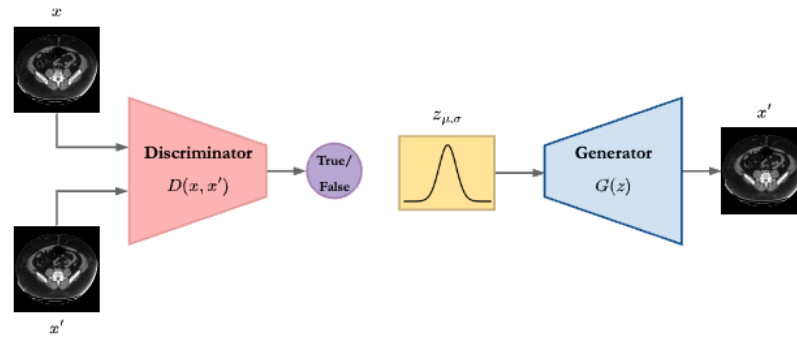
Conditional Diffusion

- Conditional signal
 - Different shape
 - not homogeneous
- Solution
 - Attention
 - We want an **university** in the **Harry Potter** style

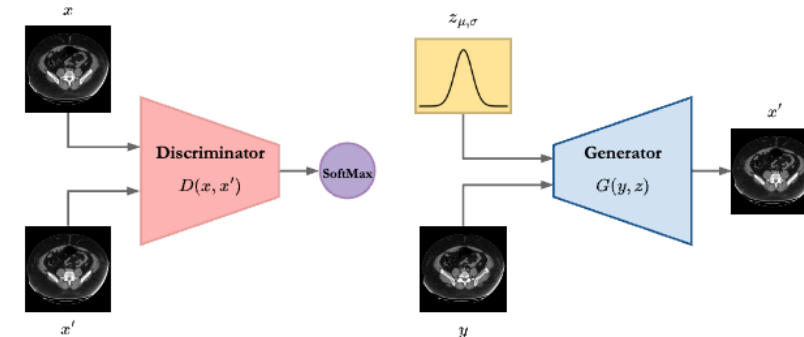


Diffusion Model

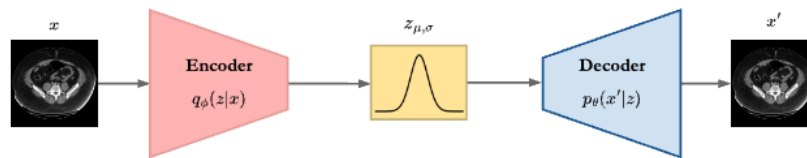
- Differences in the Generative Models



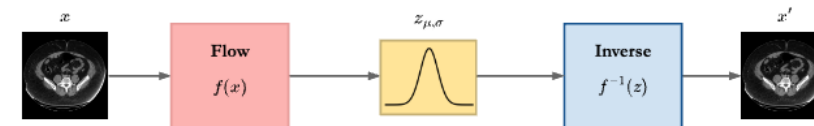
(a) GAN



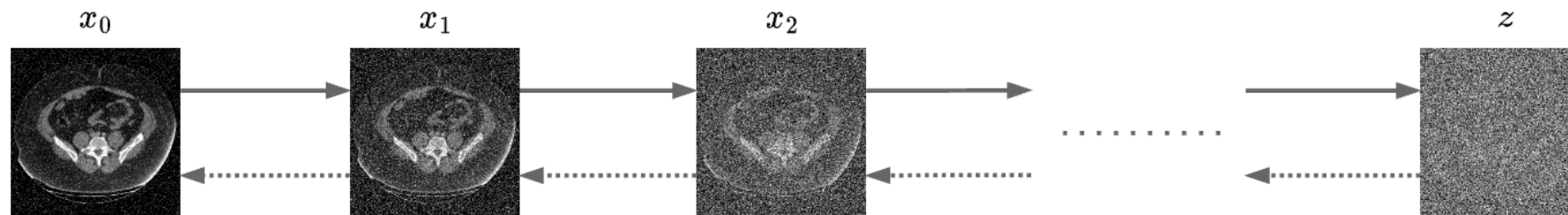
(b) Energy-based Models



(c) VAE



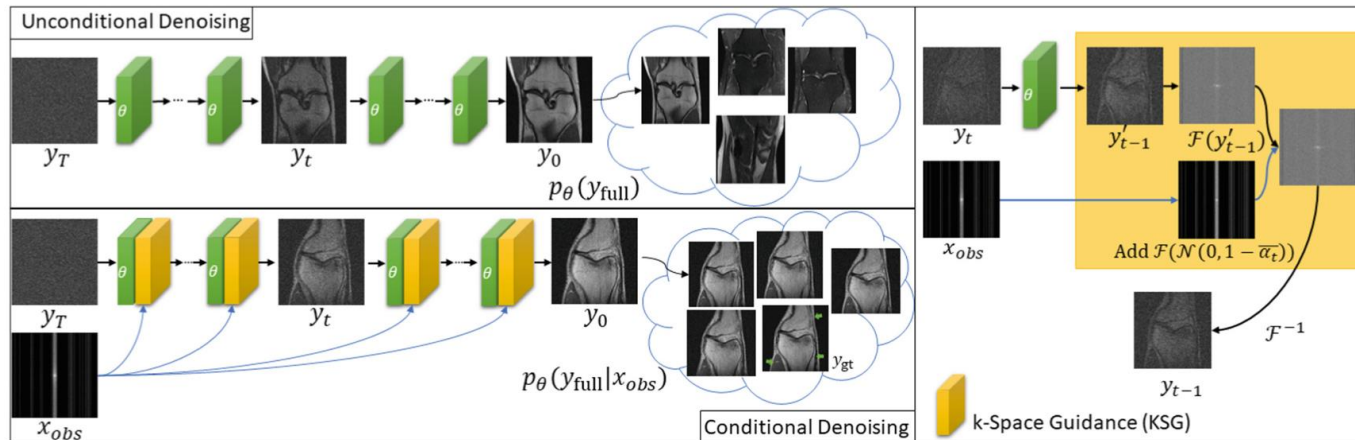
(d) Flow-based models



(e) Diffusion Models

Diffusion Model In Medical Image

- MRI image reconstruction



- K-Space Guidance
- Coarse-to-Fine Sampling

$$y'_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(y_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(y_t, t) \right) + \sigma_t \mathbf{z}, \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}).$$

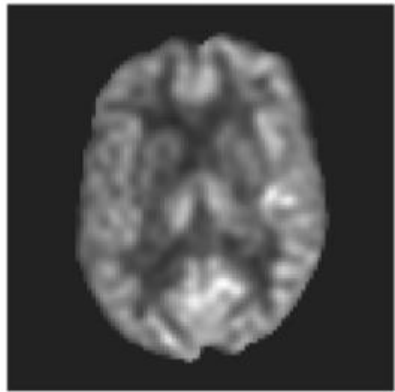
$$y_t = \mathcal{F}^{-1}((1 - \mathcal{M})\mathcal{F}y'_t + \mathcal{M}x_{\text{obs},t}),$$

$$\text{where } x_{\text{obs},t} = x_{\text{obs}} + \mathcal{F}(\mathcal{N}(0, (1 - \bar{\alpha}_t)\mathbf{I})).$$

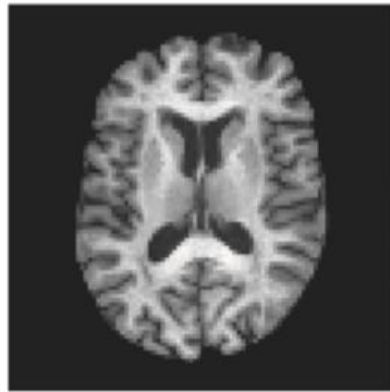
FastMRI				SKM-TEA			
D5C5 [19]	OUCR [6]	DiffuseRecon	GT	D5C5 [19]	OUCR [6]	DiffuseRecon	GT
6x							
29.61/.6974	31.34/.7472	35.30/.8622	Variance	30.03/.6160	30.50/.6765	31.28/.7011	Variance
8x							
25.61/.5167	27.84/.6148	30.42/.7087	Variance	31.59/.7399	31.97/.7513	33.26/.7896	Variance

Diffusion Model In Medical Image

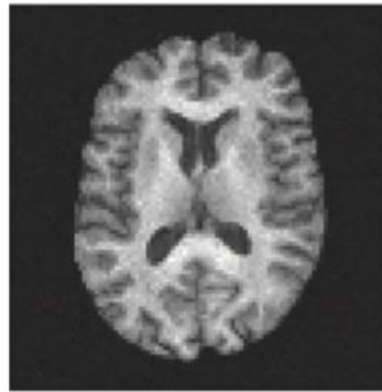
- MRI synthesis



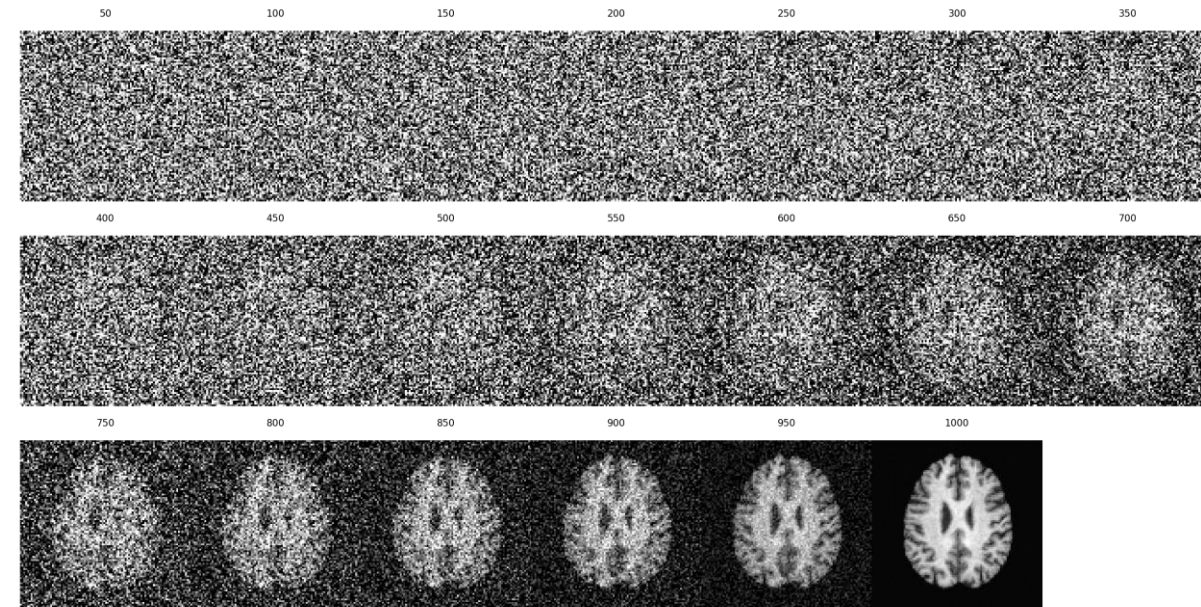
ASL



Real T1w

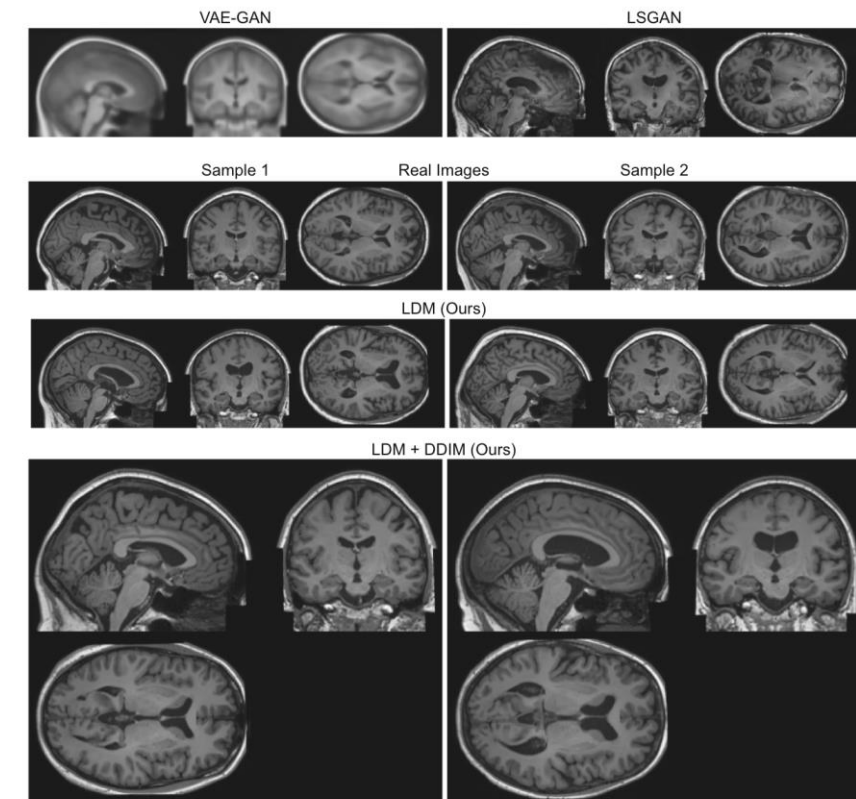
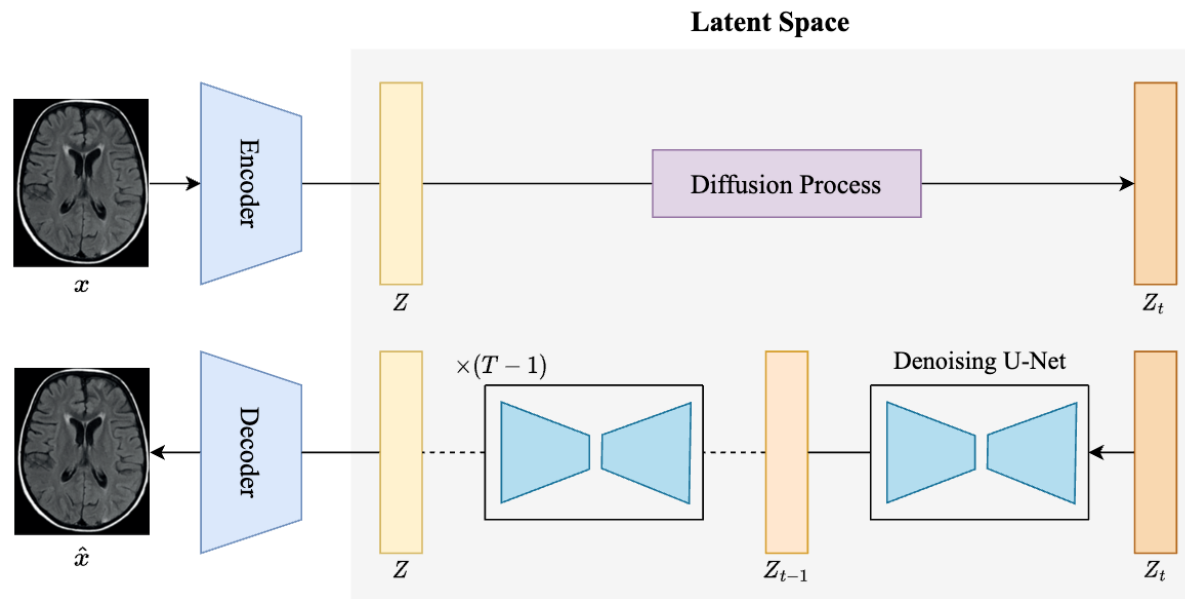


T1w generated
by ASL



Diffusion Model In Medical Image

- Latent Diffusion Model In Medical
- Synthetic high-resolution MRI images



Based on the LDM structure, a large number of high-resolution MRI images can be efficiently synthesized